

Politecnico di Torino

Master's Degree in Mechatronics Engineering

Academic Year 2025/2026

Graduation Session December 2026

Vision-Based Row Following Navigation for Horticultural Robots with Automatically Labeled Semantic Segmentation

Supervisors:

Marcello Chiaberge

Alessandro Navone

Umberto Albertin

Candidate:

Emanuele Pruiti

Abstract

Precision agriculture aims to increase crop yield and the efficiency of field operations while reducing waste. At the same time, the growing world population increases the demand for agricultural production, while the shortage of skilled labor and the rising cost of farm management push agriculture towards autonomous robots. Row-based crops are among the most studied applications of these robots.

Navigating between crop rows requires precise positioning, because the robot must stay within the corridor without damaging the plants. Satellite positioning alone does not guarantee this precision, since dense canopies and greenhouse structures degrade the GNSS signal and its accuracy. In addition, RTK corrections and 3D LiDAR sensors are expensive, which limits the large-scale adoption of agricultural robots. Camera-based methods are cheaper, but classical pipelines based on color thresholds are sensitive to illumination changes and weeds. Segmentation networks handle these conditions better, but they need large datasets annotated manually at pixel level, which take a long time to produce, and they must run fast on onboard hardware.

In this thesis, we propose a low-cost solution based on a monocular RGB camera and composed of two stages: segmentation and path extraction. In the first stage, a semantic segmentation network predicts the path between the rows. In the second stage, a path extractor estimates from the predicted mask the centerline of this path, which the robot follows to move along the row. The extractor divides the region of interest into horizontal strips, associates the corridor intervals from the bottom of the image upwards, and fits the left and right edges separately with iteratively reweighted least squares, so that short gaps and local errors in the predicted mask have a limited effect on the result. The centerline is the mean of the two edge lines.

The FCN, DeepLabV3, and LR-ASPP deep learning architectures were compared on a manually annotated dataset of lettuce images. LR-ASPP was selected for its speed: its accuracy is about 1% lower than that of the other two networks, but its inference is 7 to 11 times faster, providing lightweight execution on the onboard computer.

To reduce the annotation effort, we investigate whether manual annotation

can be replaced by labels generated automatically by SAM 3 from text prompts, without manual correction. In operating conditions, on the images taken between parallel rows, networks trained on SAM 3 labels reach a path Intersection over Union about 6% lower than the network trained on manual labels. Since the path is the only class used for navigation, the robot uses a binary network trained on the SAM 3 labels of lettuce, tomato, and cucumber crops, so the tomato and cucumber images required no manual annotation.

The images were collected in two experimental campaigns, and a third campaign was dedicated to the tests on the robot. The proposed navigation method was compared with three state-of-the-art solutions. The tests showed that the robot can autonomously navigate in horticultural rows with a monocular RGB camera and a segmentation neural network trained using only automatic labels without manual correction. These results represent a possible robust solution for precise motion inside the crop rows and a lower risk of damaging the crops, paving the way for further horticultural operations.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures VII

1 Introduction 1
- 1.1 Context and motivation . . . 1
- 1.2 Problem and approach . . . 2
- 1.3 Objectives of the thesis . . . 3
- 1.4 Organization of the work . . . 3

2 State of the Art 5
- 2.1 Segmentation for row navigation . . . 5
- 2.2 Segmentation architectures . . . 6
- 2.3 Annotation of the training images . . . 6
- 2.4 Extraction of the navigation reference . . . 7
  - 2.4.1 Column histograms . . . 7
  - 2.4.2 Horizontal strips and interval midpoints . . . 7
  - 2.4.3 RANSAC line fitting . . . 7
  - 2.4.4 Robust line fitting with IRLS . . . 7
- 2.5 Control and evaluation . . . 8
- 2.6 Positioning of this thesis . . . 8

3 Deep Learning Architectures for Semantic Segmentation 9
- 3.1 Principles of segmentation networks . . . 9
- 3.2 Fully Convolutional Networks (FCN) . . . 10
  - 3.2.1 Original architecture . . . 10
  - 3.2.2 TorchVision implementation . . . 11
- 3.3 DeepLabV3 . . . 11
  - 3.3.1 Atrous Spatial Pyramid Pooling . . . 11
  - 3.3.2 TorchVision implementation . . . 12
- 3.4 MobileNetV3 and LR-ASPP . . . 13
  - 3.4.1 MobileNetV3-Large backbone . . . 13
  - 3.4.2 LR-ASPP head . . . 15

3.5 SAM 3 . . . 16
3.6 Summary . . . 16

4 Dataset Preparation and Comparative Model Training 18
4.1 Image acquisition and class definition . . . 18
4.1.1 Image sources and acquisition conditions . . . 18
4.1.2 Semantic classes and labeling targets . . . 20
4.2 Dataset preparation . . . 20
4.2.1 Lettuce dataset with manual annotation . . . 20
4.2.2 Automated zero-shot annotation via SAM 3 . . . 20
4.2.3 Tomato and cucumber images . . . 21
4.2.4 Dataset partitioning . . . 22
4.3 Architecture comparison: FCN, DeepLabV3, and LR-ASPP . . . 22
4.3.1 Model selection and pretrained weights . . . 22
4.3.2 Training configuration and loss formulation . . . 23
4.3.3 Evaluation metrics and results . . . 24
4.3.4 Selection of LR-ASPP . . . 25
4.4 Manual versus SAM 3 labels . . . 25
4.4.1 Training configurations and evaluation . . . 25
4.4.2 Results on the lettuce test images . . . 27
4.4.3 Extension to the multi-crop dataset and deployed network . 29

5 Extraction of the Local Navigation Reference 31
5.1 Input mask and line models . . . 31
5.2 Strip-IRLS . . . 32
5.2.1 Region of interest and mask cleaning . . . 32
5.2.2 Horizontal strips and intervals . . . 33
5.2.3 Bottom-up association and chain selection . . . 34
5.2.4 Separate fitting of the corridor edges . . . 36
5.2.5 Validation, confidence, and output . . . 39
5.2.6 Setting the parameters . . . 41
5.3 EdgeNet, an edge regression network . . . 43
5.4 Baseline extractors . . . 46
5.4.1 Center points . . . 46
5.4.2 Column histogram . . . 47
5.4.3 RANSAC edges . . . 47
5.5 Structural differences and limitations . . . 48

6 Robot Platform and Integration 50
6.1 Robot hardware and sensors . . . 50
6.1.1 Robot platform . . . 50

6.1.2 Camera configuration . . . 52
6.1.3 Computing platform . . . 52
6.2 Software architecture . . . 52
6.2.1 ROS integration . . . 52
6.2.2 Control interface . . . 53
6.2.3 Controller node . . . 53
6.3 Complete pipeline . . . 60
6.4 Deployment and testing . . . 60
6.4.1 Timing and latency . . . 60
6.4.2 Processor-in-the-Loop test . . . 61

7 Experimental Validation 64
7.1 Offline evaluation . . . 64
7.1.1 Segmentation metrics . . . 64
7.1.2 Geometric reference accuracy . . . 65
7.2 Field validation . . . 74
7.2.1 Test conditions . . . 74
7.2.2 Trajectory measurement . . . 74
7.2.3 Protocol . . . 75
7.2.4 Results . . . 75
7.3 Discussion . . . 80
7.3.1 Segmentation performance . . . 80
7.3.2 Geometric extraction trade-offs . . . 80
7.3.3 EdgeNet: first results . . . 81
7.3.4 Field tests . . . 82

8 Conclusions and Future Work 83
8.1 Main contributions . . . 83
8.2 Conclusions on segmentation and annotation . . . 84
8.3 Conclusions on path extraction . . . 85
8.4 Conclusions on the field tests . . . 86
8.5 Limitations . . . 86
8.6 Future work . . . 88
8.6.1 Dataset and annotation . . . 88
8.6.2 Segmentation and model deployment . . . 88
8.6.3 Geometric extraction . . . 89
8.6.4 System integration and field validation . . . 89
8.7 Closing remarks . . . 90

Bibliography 91

List of Figures

3.1 Skip architecture of the original FCN. The stride-32 predictions are combined with predictions from `pool4` (FCN-16s) and `pool3` (FCN-8s). Adapted from Long et al. [7]. 11
3.2 DeepLabV3 with the ASPP module at output stride 16, with rates 6, 12, and 18. The TorchVision model used in this thesis works at output stride 8, with rates 12, 24, and 36. Adapted from Chen et al. [8]. 12
3.3 MobileNetV3 block: 1×1 expansion, depthwise convolution, squeeze-and-excitation, and 1×1 projection, with the residual connection. NL denotes the nonlinearity, ReLU or hard swish depending on the layer. Adapted from Howard et al. [9]. 14
3.4 LR-ASPP head on the MobileNetV3 backbone, with 19 output classes. Adapted from Howard et al. [9]. 15
3.5 SAM 3 architecture. The detector segments the instances of the prompted concept in each image, and the tracker propagates the masks between video frames. Only the detector is used in this thesis. Adapted from Carion et al. [11]. 16

4.1 Sample field images across crops and corresponding training masks 19
4.2 SAM 3 zero-shot autolabeling and mask composition workflow . . . 21
4.3 Training and validation loss per epoch of FCN, DeepLabV3, and LR-ASPP on the manual lettuce dataset. The best checkpoint is the epoch with the minimum validation loss. The backbone is unfrozen after epoch 10. 26

5.1 Example of the ROI and of the mask cleaning of Strip-IRLS on an example mask, with the parameters used on the robot. The rows above row 64 are excluded, and the dark gray regions inside the ROI are removed by the cleaning. 33

5.2 Strips and intervals on the mask of Figure 5.1, after the cleaning of Section 5.2.1, with the parameters used on the robot ($N = 16$, $h = 10$ px, ROI from row 64). The blue lines are the strip boundaries, and the orange boxes are the intervals found in each strip. 34
5.3 Chains built from the intervals of Figure 5.2. (a) The two chains grown from the seeds of strip 0, in purple from the blob and in orange from the corridor, with their number of intervals, samples and total cost. The dots are the samples, the endpoints that are not clipped. The dashed red boxes are intervals that are not in any chain. (b) The selected chain, the one with more samples. The yellow dots are its samples, used in the edge fits. The regions in light red are not in the selected chain and are not used in the fits. . 36
5.4 Reweighted fit of the two edges of Strip-IRLS on the samples of the selected chain of Figure 5.3b, at the start and after iterations 1, 3 and 10. The points are the samples of the chain, colored by their weight, the blue and green lines are the fits of the left and right edge, and the red line is the centerline, their mean. The three upper samples of the right edge, on the region attached to the corridor, pull the first fit toward the vertical. Their weights fall to 0.21, 0.10 and 0.09 only after several iterations, and after the tenth the line follows the edge. 39
5.5 Architecture of *EdgeNet*. Each box is a feature map, colored by the layer that produces it and labeled with channels × height × width. The two maps of a block have the same size and share one label. The mask is averaged over 2×2 blocks, and two channels with the x and y coordinates of each pixel, scaled to $[-1,1]$ as in CoordConv [20], are added, so the convolutions also see where each pixel is. Each of the four blocks halves the size with a stride-2 convolution and then applies a second convolution. The $96 \times 7 \times 7$ map is flattened and passed to two fully connected layers, the first followed by ReLU. A sigmoid turns the last output into p_{valid}. 44
5.6 Masks and edge lines of *EdgeNet*. (a)–(c) Synthetic validation masks with the edge lines of their labels. (d)–(f) Masks of the field tests (Section 7.2, *EdgeNet* Run 1), with the edge lines estimated by *EdgeNet*. The dots are the positions at rows 156 and 223. The darker rows above row 64 are not used for the IoU. 46

6.1 Side view of Demetra, with the wheel diameter, the wheelbase L, the camera height h_{cam} and the adjustable tilt θ of the camera. . . . 51

6.2 Top view of the opposite-phase 4WS kinematics of Demetra, with the equivalent bicycle in orange, the ICR and the turning radius R of the mid-axle point M (Equation (6.1)). 51
6.3 ROS 2 node graph on the robot, as shown by `rqt_graph`. Ellipses are nodes and rectangles are topics. 53
6.4 Frame of the debug video recorded during a field run with Strip-IRLS. From left to right: segmentation (the segmentation before the morphological cleaning), region of interest (the cleaned segmentation, on which the path extractor operates) and path (the extracted path drawn on the camera image). 53
6.5 Measured lateral error e_k and filtered error $\hat{e}_k$ during 1.5 s of a field run with Strip-IRLS. The filtered error was computed from the recorded measurements with Equation (6.3) and the smoothing factor of Equation (6.5) for the recorded time between frames. . . . 55
6.6 Response of the filter of Equation (6.3) to a step of the input, with $\alpha_{\text{ref}} = 0.20$. At 15 frames per second, the factor converted with Equation (6.5) gives the same response in time as at 30 frames per second (dashed line). 56
6.7 Angular velocity target ω^* of Equation (6.9) as a function of the command error e_{cmd}. (a) Effect of the confidence, with the gain used on the robot, $K_p = 1.2$. The curves are for $C = 1$, $C = 0.60$ (threshold between `ok` and `degraded`) and $C = 0.38$ (threshold below which the status is `lost`). (b) Effect of the gain, with $C = 1$. The command saturates at 36 px with $K_p = 1.8$, at 54 px with $K_p = 1.2$ and at 72 px with $K_p = 0.9$. In both plots the command is zero inside the deadband ($|e_{\text{cmd}}| < 4$ px) and is limited to ± 0.58 rad/s. The right axis gives the corresponding steering angle δ at $v = 0.20$ m/s, where Equation (6.2) becomes $\delta = \arctan(\omega)$, so 0.58 rad/s corresponds to 30°. 58
6.8 Complete pipeline on the robot. 60
6.9 Processor-in-the-Loop test. Top: frame of the simulation node shifted with Equation (6.13), with the edges (blue and green), the centerline (red) and the lateral error at y_{ref} (magenta) extracted offline by the network and Strip-IRLS. Bottom: the raised robot during the test, at similar lateral errors. (a) and (b) Lateral error to the left and to the right: the wheels steer toward the center of the row. (c) Error below 1 px: the simulated robot is back at the center and the wheels are straight. The mirrored vegetation comes from the border that the simulation node adds before the shift. 63

7.1 Visual comparison of segmentation outputs. (a) Four of ten test images of the manual lettuce dataset, taken at evenly spaced positions of the test split: RGB image, manual ground truth, and the predictions of the LR-ASPP network trained on the manual labels and of the three LR-ASPP networks trained only on SAM 3 lettuce labels. In the third image the SAM 3 networks mark the whole floor as path, including the strips beside the central corridor, and in the fourth, where the manual ground truth has no path, they mark the soil in front of the robot. (b) Tomato and cucumber images, which have no ground truth: RGB image and prediction of the binary network installed on the robot, as a qualitative comparison only. . . 66
7.2 Lines estimated by the five extractors on two masks with disturbances, chosen among the masks where Strip-IRLS has an error below 4 px and at least two other extractors fail. (a) is a mask of the synthetic scenario `hole_bottom_tall`, and (b) is the real frame `pomodori2_01051`, whose true centerline comes from the hand annotation. The magenta line is the estimated centerline, the diamond its position at y_{ref} (dashed line), and the yellow circle the true centerline at y_{ref}. Below each panel are the status and the error at y_{ref}. A panel without a line is `lost`. 67
7.3 Lateral distance from the row centerline in the three short runs of each extractor, positive to the left of the robot, from $Y = 13.0$ to $Y = 4.0$ m. The gray band is not used in Table 7.7: there the runs still depend on the start position. The robot moves toward the station, from left to right in the plot. The black dashed line is the mean of the runs of the other four extractors. The vertical axis is centered on zero, with the same scale in all panels. In this stretch the reference is the straight segment between the points measured by hand at $Y = 1.55$ m and $Y = 18.09$ m. 77
7.4 Runs along the full row, one per extractor. (a) Trajectories measured by the total station and centerline through the reference points measured by hand. The robot moves away from the station, so its left is toward negative X. (b) Lateral distance from the centerline, positive to the left of the robot. The vertical axis is centered on zero and inverted, so that the curves have the same orientation as in (a). 78

7.5 Frames recorded by the controller during the short runs. In each example the left image is the camera image with the edges (blue and green) and the centerline (red) estimated by the extractor, and the right image is the region of interest, the cleaned mask on which the extractor operates (Chapter 6), with the same lines. Column histogram estimates only a vertical centerline. The magenta segment is the lateral error. The status and the measured lateral error e_k (positive to the right) are those recorded by the controller for the same frame, and Y is interpolated from the positions measured by the total station. The text that the controller writes on these images was removed. (a) Sparse plants in the middle of the short runs. (b), (c) Column histogram with the centerline at the image border and status `ok`. (d) Strip-IRLS near the end of the stretch, with the station and the end of the row in view. 79

Chapter 1

Introduction

▶ **DA COMPLETARE: Contesto di tesi e motivazione applicativa**

Inquadrare l'attività svolta nell'ambito della tesi di laurea magistrale:

- Collaborazione con il gruppo di ricerca di riferimento (Politecnico di Torino).
- Esigenze dell'agricoltura di precisione: diserbo meccanico selettivo, monitoraggio delle colture e riduzione dell'impatto ambientale senza dipendenza da segnali GNSS RTK sotto chioma o in serra.
- Obiettivo di realizzare una soluzione autonoma, modulare e fruibile su calcolatori embedded a basso costo (famiglia NVIDIA Jetson).

1.1 Context and motivation

Global population growth and rising food demand exert increasing pressure on agricultural productivity, while the sector faces persistent shortages of skilled labor and the rising cost of farm management [1, 2]. Addressing these demands requires higher automation in field operations such as weeding, spraying, and crop monitoring. Row-based crops are among the most studied applications of agricultural robots and account for about 75% of the planted cropland in the United States [3]. Autonomous ground vehicles designed for these settings must navigate along inter-row corridors reliably without damaging crops or compacting productive soil beds.

Outdoor agricultural navigation historically relied on Global Navigation Satellite Systems (GNSS) augmented by Real-Time Kinematic (RTK) corrections. However, satellite positioning has limitations in the field: thick crop canopies cause multi-path reflections and signal obstruction [1, 3]. Greenhouse structures also obstruct satellite reception. Furthermore, a GNSS system with RTK corrections is costly [3].

Alternative exteroceptive sensors such as 3D LiDAR provide direct spatial measurements, but their high cost limits the large-scale adoption of agricultural robots [1, 3]. Monocular RGB cameras offer a cost-effective alternative that captures detailed chromatic and texture cues directly from the scene.

Early vision-based guidance pipelines extracted navigation lines through handcrafted image processing techniques. These approaches compute color vegetation indices to isolate plant matter from soil, apply intensity thresholding, and detect corridor edges using line-fitting algorithms or Hough transforms [1, 4]. Although computationally lightweight, these calibrated pipelines degrade when ambient field conditions shift. Variable sunlight angles, cast shadows from foliage, soil moisture gradients, specular reflections on wet leaves, and weed infestations frequently disrupt fixed thresholding rules and generate false boundary estimates.

Deep learning methods, in particular convolutional neural networks for semantic segmentation, overcome some of these limitations. They can detect crop rows despite variations in light levels, occlusions due to weeds, and variations in the background [2].

However, deep neural networks introduce a practical problem: supervised training requires extensive pixel-level ground truth. Manually labeling agricultural scenes is labor-intensive and must cover multiple crop types, growth stages, and ambient conditions. In addition, running neural networks on compact onboard computers requires balancing segmentation accuracy against processing speed and power consumption, as steering commands must be updated fast enough to guide the vehicle in real time.

This thesis addresses these challenges through a modular pipeline that combines automatic dataset annotation with SAM 3, real-time onboard semantic segmentation, and deterministic geometric reference extraction for local robot guidance.

1.2 Problem and approach

The robot must drive along the path between two crop rows with local guidance from the color images of one camera: it keeps its position relative to the rows visible in the image and does not build or use a map of the field.

Figure 6.8 in Chapter 6 shows the steps that run on the robot for each camera frame:

1. **Segmentation.** An LR-ASPP network with a MobileNetV3 backbone takes the camera image and predicts a binary mask of the path between the rows. The masks used to train it were generated by SAM 3 without manual correction. SAM 3 runs only offline, to label the training images, and not on the robot.

2. **Path extraction.** The path extractor reduces the mask to a straight centerline of the corridor in image coordinates. The predicted mask can contain holes

and false path regions, so the extractor also returns a status, `ok`, `degraded` or `lost`. Chapter 5 compares five extractors with the same input and output, and all five were tested in closed loop on the robot.

3. **Control.** The controller steers the robot to keep it at the center of the path between the rows. It computes the lateral error of the centerline from the image center at a fixed image row, filters it, and converts it into an angular velocity command at constant linear speed. If no valid centerline is found for more than 0.5 s, the robot stops.

1.3 Objectives of the thesis

The main objectives of this work are:

- Evaluate whether manual annotation can be replaced by labels generated automatically by SAM 3 from text prompts, without manual correction, by comparing networks trained on SAM 3 labels with a network trained on manual masks.
- Select a segmentation architecture balancing accuracy and inference speed for robotic deployment, through offline evaluation of FCN, DeepLabV3, and LR-ASPP.
- Evaluate the effect of output label configurations (four-class, three-class, and binary path) on path segmentation accuracy.
- Develop extractors that convert segmentation masks into local image-space navigation references: a geometric one that fits the two corridor edges (Strip-IRLS) and a first version of an edge regression network (EdgeNet). Compare them with three baseline algorithms.
- Integrate the selected perception and geometric components on a robot platform for field validation.

1.4 Organization of the work

The thesis is organized as follows:

Chapter 2 reviews vision-based methods for agricultural row navigation, covering classical and learning-based segmentation approaches, geometric extraction techniques, and experimental validation in related work.

Chapter 3 presents the segmentation networks evaluated to choose the one used on the robot, FCN, DeepLabV3 and LR-ASPP with a MobileNetV3 backbone, and SAM 3, which generates the training labels.

Chapter 4 describes dataset preparation with manual annotation and automatic SAM 3 labeling, empirical comparisons among the three architectures on lettuce, and evaluations of supervision strategies across label configurations.

Chapter 5 presents the two extractors developed in this work, which convert corridor masks into navigation references: Strip-IRLS, based on horizontal-strip interval selection and independent edge fitting, and EdgeNet, a network that regresses the corridor edges, still at an early stage. Three baseline algorithms are described for comparison.

Chapter 6 describes the robot and its camera, the complete pipeline that runs on the robot from the camera image to the steering command, its integration in ROS, the timing on the onboard computer, and the Processor-in-the-Loop test.

Chapter 7 compares the five extractors offline, on synthetic masks and on manually annotated frames, and reports the closed-loop tests of the robot in the field, with the trajectory measured by a total station.

Chapter 8 summarizes the main findings, discusses limitations, and identifies directions for future work.

Chapter 2

State of the Art

This chapter reviews the works related to the parts of this thesis: the segmentation network, the annotation of the images used to train it, the extraction of the navigation reference from the mask, and the controller that steers the robot with this reference. Each extraction method is linked to the extractor of Chapter 5 that is based on it or compared with it.

2.1 Segmentation for row navigation

Classical pipelines combine image filtering, thresholding, and application-specific geometric rules. Mendez et al. convert vineyard images to grayscale and apply Gaussian filtering and Otsu thresholding. They keep the largest white blob, which is the sky above the row, and find the row center from it (Section 2.4.1) [4, Section 3.4]. Such methods can be effective in the conditions for which their parameters were selected. However, fixed thresholds are sensitive to changes in illumination, and the complexity of the field and the variety of crops make it difficult to segment the image from low-level features alone [1].

Semantic segmentation assigns a class to each pixel, and the choice of the classes decides what the geometric stage can recover. De Silva et al. train U-Net to predict crop-row masks in sugar-beet images. In their labels each crop row is a single white line 6 pixels wide, not a set of individual plants [2, Sections 3.3 and 4, Figure 5]. Navone et al. predict a binary vegetation mask from RGB images with a MobileNetV3 backbone and an LR-ASPP head, and then combine this mask with depth to extract the reference [5, 3]. Li et al. segment the inter-row path instead of the plants [6].

2.2 Segmentation architectures

Fully Convolutional Networks (FCN) convert the fully connected layers of a classification network into convolutions, so the network outputs a map of class scores at a much lower resolution than the input image. A deconvolution layer then upsamples this map to one prediction per pixel [7, Sections 3.1 and 4.1]. DeepLabV3 uses atrous convolutions and atrous spatial pyramid pooling to capture multi-scale context without decreasing the feature resolution [8]. For embedded applications with limited computing resources, Howard et al. [9] introduced MobileNetV3, whose blocks are built on depthwise convolutions, together with the Lite Reduced Atrous Spatial Pyramid Pooling (LR-ASPP) segmentation head, which uses a large pooling kernel and only one 1×1 convolution in its pooling branch. Chapter 3 describes the three networks.

Transfer learning reuses most of the layers of a network trained on a similar task and replaces the top ones. It speeds up training considerably and requires significantly less training data. The reused layers can first be frozen and then fine-tuned, that is, trained further on the new task with a reduced learning rate [10, Chapter 11]. Long et al. obtain their segmentation networks by fine-tuning classifiers pretrained on ILSVRC [7, Section 4]. In agriculture, Li et al. train their network on 2000 maize images and use the resulting weights as the pretrained weights for separate datasets of 1000 tomato, cucumber, and wheat images [6, Abstract, Sections 2.1.1 and 2.2.2]. Navone et al. initialize their segmentation model with ImageNet weights and train it on synthetic AgriSeg images and 100 real crop images [3, Section 4.1, Table 1]. In this thesis, the three networks start from the pretrained segmentation weights of TorchVision and are fine-tuned in two stages (Section 4.3.2).

2.3 Annotation of the training images

Manual pixel-level annotation is slow and tedious, and a dataset that covers several crops, growth stages, and lighting conditions needs many annotated images. The first two Segment Anything models, SAM and SAM 2, segment a single object for each prompt, given as points, boxes, or masks, so the object to segment has to be indicated in each image, for example with a click. SAM 3 segments all the instances of a concept, given as a short noun phrase, as image exemplars, or as both [11, Section 1]. In this thesis, SAM 3 labels the training images offline from text prompts, and its masks are used without manual correction (Sections 3.5 and 4.2.2).

2.4 Extraction of the navigation reference

The geometric stage reduces a mask to a quantity the controller can use. In the methods reviewed below, this is an image column, one or more straight lines, or a curve. Chapter 5 describes five extractors: *Strip-IRLS* and *EdgeNet*, developed in this work, and three baselines, *Center points*, *Column histogram*, and *RANSAC edges*. Each subsection below describes one family of methods and the extractors related to it.

2.4.1 Column histograms

Histogram methods aggregate mask pixels by image column. Mendez et al. estimate the row center from the five columns with the largest pixel counts in the thresholded sky region [4, Section 3.4.3]. In SegMin, Navone et al. sum along the columns a vegetation mask combined over recent frames and refined with depth, and take the column with the least vegetation [3, Section 3]. *Column histogram* (Section 5.4.2) is a simplified version of SegMin.

2.4.2 Horizontal strips and interval midpoints

Li et al. combine a Faster-U-net inter-row-path mask with an adaptive multi-ROI procedure. Horizontal regions supply the midpoints of the path, which are used as control points for a multisegment cubic B-spline [6, Section 2.2.3].

Center points (Section 5.4.1) follows this scheme and fits one straight line to the interval midpoints. *Strip-IRLS* uses the same horizontal strips but fits the two edges of the corridor separately.

2.4.3 RANSAC line fitting

RANSAC is among the line fitting algorithms used to extract navigation lines in agricultural vision [1]. Wu et al. extract crop-row anchor points from the segmentation masks, group them with DBSCAN clustering, and fit navigation lines with RANSAC [12]. *RANSAC edges* (Section 5.4.3) applies RANSAC to the row extremes of the path corridor instead of to points of the crop rows.

2.4.4 Robust line fitting with IRLS

Kneip et al. detect crop–ground transitions in the scanlines of a stereo elevation map, then fit a linear crop-edge model using a Huber M-estimator and iteratively reweighted least squares [13, Sections 5–7].

Strip-IRLS (Section 5.2.4) also fits lines by iteratively reweighted least squares, but in image space, on the two edges of the path corridor, and with a weight function different from the Huber function of Kneip et al.

2.5 Control and evaluation

Navigation systems convert the extracted reference into motion commands. Navone et al. and Mendez et al. both compute the angular velocity from the horizontal offset of the row center from the image center. Navone et al. also reduce the linear velocity as the offset grows and smooth both commands with an exponential moving average [3, Section 3.4]. Mendez et al. keep the linear velocity constant, use a proportional gain, and switch to a second mode based on the ratio of sky pixels on the two sides of the row center [4, Section 3.5]. The controller of this thesis also filters the error with an exponential moving average and uses a proportional steering law (Section 6.2.3).

Mendez et al. report closed-loop vineyard trials and measure the centering error from the distances to the two sides of the row, given by a LiDAR that is not used for navigation [4, Sections 3.6 and 4]. Navone et al. compare the trajectory with the central path of the row in simulation. In crop fields, they measure the lateral displacement with a LiDAR and the heading with an attitude and heading reference system [3, Sections 4.3–4.4]. In the field tests of this thesis, the position of the robot is measured with a total station (Chapter 7).

2.6 Positioning of this thesis

This thesis builds a row-following system without GNSS, with one RGB camera and the onboard computer of the robot. Its main choices are the following:

1. **Automatic labels.** SAM 3 generated, from text prompts, the masks used to train the segmentation network that runs on the robot. The masks were used without manual correction (Section 4.2.2).

2. **Path segmentation.** The network segments the path between the rows and not the plants.

3. **Two path extractors.** *Strip-IRLS* fits the two edges of the corridor separately instead of using the midpoints of the path intervals (Section 5.2). *EdgeNet*, still at an early stage, is a small network that regresses the two edges directly from the mask (Section 5.3). Both are compared with three baselines adapted from the methods of Section 2.4: *Center points*, *Column histogram*, and *RANSAC edges* (Section 5.4).

Chapter 3

Deep Learning Architectures for Semantic Segmentation

Semantic segmentation classifies each pixel of an image according to the class of the object it belongs to. Different objects of the same class are not distinguished [10, Chapter 12]. In this thesis, the class used for navigation is the path between the crop rows. All the classes are listed in Chapter 4.

This chapter describes the three segmentation networks compared in Chapter 4 (FCN, DeepLabV3, and LR-ASPP with a MobileNetV3-Large backbone) and SAM 3, which generated the labels of the networks trained without manual annotation, including the network that runs on the robot. Section 3.1 introduces the concepts shared by the three networks.

3.1 Principles of segmentation networks

For each pixel, a segmentation network outputs one score for each class. These scores are called logits, and the softmax function converts them into the estimated probabilities of the classes. The predicted class of the pixel is the one with the highest probability. The network is trained by minimizing the cross entropy, which penalizes the model when it estimates a low probability for the target class [10, Chapter 4]. Each pixel is treated as one instance, as in Long et al., whose loss is a sum of per-pixel terms over the output [7, Sections 3 and 4]. Chapter 4 adds a weight for each class to this loss (Section 4.3.2).

A classification network reduces the spatial resolution of its feature maps with layers of stride greater than 1. Chen et al. call *output stride* the ratio of the input image resolution to the resolution of the final output. In the classification networks they consider, it is 32 [8, Section 3.1]. At this resolution, a network may recognize that an object is in a region of the image without locating its boundaries

precisely [10, Chapter 12]. The networks of this chapter use two ways to recover spatial detail:

- **Atrous (dilated) convolution.** An atrous convolution with rate r is equivalent to a convolution with a filter in which $r - 1$ zeros are inserted between consecutive filter values. To obtain an output stride of 16 instead of 32, Chen et al. set to 1 the stride of the last layer that reduces the resolution and replace the following convolutions with atrous convolutions of rate 2. The feature maps become denser without extra parameters [8, Section 3.1]. DeepLabV3 and the TorchVision backbones of FCN and LR-ASPP use atrous convolution.
- **Features from lower layers.** In the encoder-decoder structure, the encoder gradually reduces the spatial size of the feature maps, and the decoder gradually recovers the object details and the spatial size [8, Section 2]. The original FCN (Section 3.2.1) and LR-ASPP (Section 3.4.2) add predictions or features from lower, finer layers to the coarse ones.

In the TorchVision v0.20.1 implementations used in this thesis, each network has a *backbone*, a classification network without its final layers, which extracts the feature maps, and a *segmentation head*, which computes the logits from them. The logits are then resized to the input resolution with bilinear interpolation [14, 15, 16].

3.2 Fully Convolutional Networks (FCN)

3.2.1 Original architecture

Long et al. view the fully connected layers of a classification network as convolutions whose kernels cover their entire input region. The resulting network accepts inputs of any size and outputs a map of class scores [7, Section 3.1]. With the VGG-16 classifier, this map has a stride of 32, which limits the detail of the upsampled output [7, Section 4.2]. Long et al. therefore combine the final prediction layer with lower layers of finer stride (Figure 3.1):

- **FCN-32s** upsamples the stride-32 predictions to the image resolution in one step.
- **FCN-16s** adds a 1×1 convolution on the stride-16 layer `pool4` to produce additional class predictions. It sums them with the stride-32 predictions upsampled by 2, and upsamples the result to the image.
- **FCN-8s** repeats the same step with the stride-8 layer `pool3`.

The upsampling layers are transposed convolutions, also called deconvolutions. The 2× upsampling layers are initialized to bilinear interpolation and then learned [7, Sections 3.3 and 4.2].

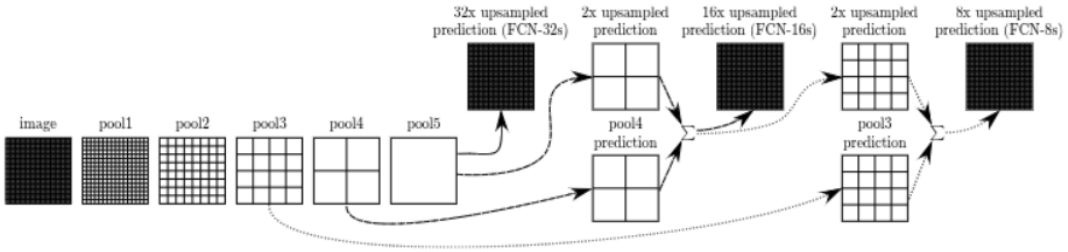


Figure 3.1: Skip architecture of the original FCN. The stride-32 predictions are combined with predictions from `pool4` (FCN-16s) and `pool3` (FCN-8s). Adapted from Long et al. [7].

3.2.2 TorchVision implementation

The `fcn_resnet50` model of TorchVision does not use the skip architecture. Its backbone is ResNet-50 [17], whose residual blocks add the input of the block to the output of its layers through a shortcut connection. In the bottleneck blocks of ResNet-50, a 1×1 convolution reduces the number of channels, a 3×3 convolution works on the reduced channels, and a second 1×1 convolution restores them [17, Sections 3.2 and 4.1]. ResNet-50 has four stages, `layer1` to `layer4` in TorchVision. TorchVision replaces the stride of `layer3` and `layer4` with atrous convolutions, so the backbone has an output stride of 8 instead of 32 [14].

The segmentation head, `FCNHead`, takes the 2048 channels of `layer4`. It applies a 3×3 convolution to 512 channels, batch normalization, ReLU, dropout with probability 0.1, and a 1×1 convolution that gives the logits. A second `FCNHead`, the auxiliary classifier, takes the 1024 channels of `layer3`. Its output enters only the training loss (Section 4.3.2) [14].

3.3 DeepLabV3

3.3.1 Atrous Spatial Pyramid Pooling

DeepLabV3 applies atrous convolution in the last blocks of a ResNet and adds an Atrous Spatial Pyramid Pooling (ASPP) module on the resulting feature map [8, Section 3]. ASPP applies atrous convolutions with different rates in parallel to the same feature map, so that the features are resampled at several scales. As the rate grows, fewer filter weights fall on the feature map and more on the padding

zeros. With a rate close to the size of the feature map, a 3×3 filter acts as a 1×1 filter. To add global context, ASPP therefore also includes image-level features, obtained by global average pooling, a 1×1 convolution, and bilinear upsampling. The module has five branches, all with 256 filters and batch normalization:

- one 1×1 convolution,
- three 3×3 atrous convolutions with rates 6, 12, and 18 at output stride 16, doubled at output stride 8,
- the image-level features.

The outputs of the five branches are concatenated and pass through a 1×1 convolution with 256 filters, followed by the final 1×1 convolution that gives the logits [8, Section 3.3]. Figure 3.2 shows the module. Block1 to Block4 in the figure are the four ResNet stages (`layer1` to `layer4` in TorchVision).

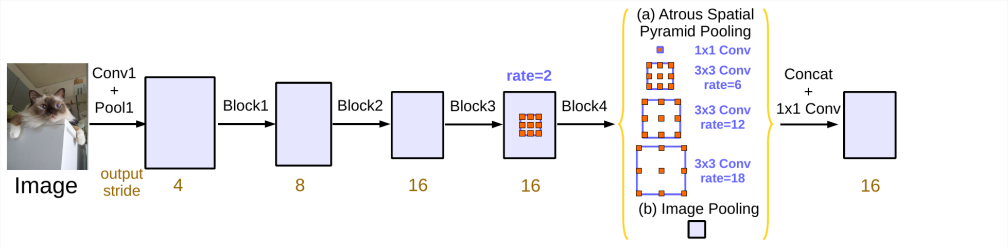


Figure 3.2: DeepLabV3 with the ASPP module at output stride 16, with rates 6, 12, and 18. The TorchVision model used in this thesis works at output stride 8, with rates 12, 24, and 36. Adapted from Chen et al. [8].

3.3.2 TorchVision implementation

The `deeplabv3_resnet50` model of TorchVision uses the same backbone as `fcn_resnet50`, the ResNet-50 with output stride 8 (Section 3.2.2). Its ASPP rates are 12, 24, and 36, the rates of Chen et al. for output stride 8. The 1×1 projection of the concatenated branches is followed by batch normalization, ReLU, and dropout with probability 0.5. A 3×3 convolution with 256 channels, batch normalization, and ReLU follow, and a 1×1 convolution gives the logits. Like FCN, the model has an auxiliary `FCNHead` on `layer3`, whose output enters only the training loss [15].

3.4 MobileNetV3 and LR-ASPP

Howard et al. designed MobileNetV3 for mobile phone CPUs, with a network architecture search that takes into account the latency of each candidate model [9, Abstract and Section 4.1]. They released two versions, MobileNetV3-Large and MobileNetV3-Small, for high and low resource use cases [9, Sections 1 and 5.4]. This thesis uses the Large version. Together with MobileNetV3, they proposed the Lite Reduced ASPP (LR-ASPP) segmentation head [9, Section 6.4].

3.4.1 MobileNetV3-Large backbone

The blocks of MobileNetV3 combine layers introduced by earlier networks [9, Section 3]:

- **Depthwise separable convolution.** A light depthwise convolution applies a single spatial filter to each input channel, and a heavier 1×1 pointwise convolution combines the channels to generate new features [9, Section 3], [10, Chapter 12].
- **Inverted residual with linear bottleneck.** A 1×1 convolution expands the channels, a depthwise convolution works in the expanded space, and a 1×1 projection reduces the channels again. The input and output of the block are narrow and have no nonlinearity. The residual connection is used only when the input and output have the same number of channels. This is the opposite of the ResNet bottleneck of Section 3.2.2, which first reduces and then restores the channels.
- **Squeeze-and-excitation (SE).** In some of the blocks, an SE module is placed after the depthwise convolution, so it acts on the expanded representation [9, Section 3 and Table 1]. An SE module applies global average pooling to obtain one value per channel, compresses these values in a small layer with ReLU, and computes with a second layer one weight between 0 and 1 per channel. The channels are then multiplied by these weights [10, Chapter 12]. In MobileNetV3, the small layer has 1/4 of the channels of the expansion layer [9, Section 5.3].

Figure 3.3 shows the resulting block. MobileNetV3 also replaces the sigmoid and the swish nonlinearity with piecewise-linear *hard* versions built on ReLU6. Howard et al. found no discernible difference in accuracy, and the hard versions are cheaper to compute on mobile devices. The hard swish is used only in the second half of the network [9, Section 5.2].

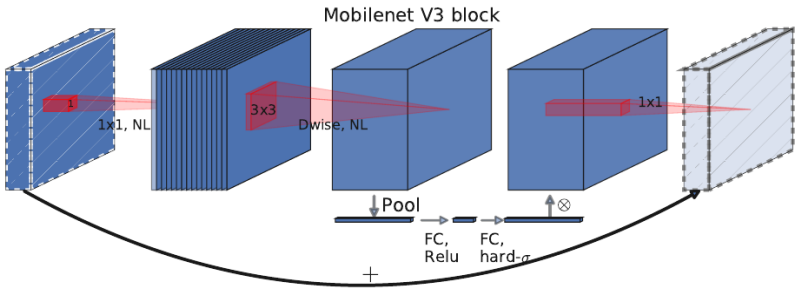


Figure 3.3: MobileNetV3 block: 1×1 expansion, depthwise convolution, squeeze-and-excitation, and 1×1 projection, with the residual connection. NL denotes the nonlinearity, ReLU or hard swish depending on the layer. Adapted from Howard et al. [9].

3.4.2 LR-ASPP head

LR-ASPP derives from R-ASPP, a reduced version of ASPP (Section 3.3.1) that keeps only two branches, a 1×1 convolution and global average pooling. In LR-ASPP, the pooling branch works like an SE module and contains only one 1×1 convolution. Atrous convolution in the last block of MobileNetV3 gives denser features, and a skip connection from low-level features adds detail [9, Section 6.4]. Figure 3.4 shows the head.

In the TorchVision model `lraspp_mobilenet_v3_large`, the last stage of the backbone uses atrous convolution, so its last layer, with 960 channels, has output stride 16 instead of 32. The low-level features are the 40 channels of the backbone at output stride 8. The head works as follows [16, 18]:

1. A 1×1 convolution with batch normalization and ReLU reduces the 960 channels to 128.

2. A second branch applies global average pooling to the 960 channels, a 1×1 convolution to 128 channels, and a sigmoid. It gives one weight per channel.

3. The output of the first branch is multiplied channel by channel by these weights and upsampled with bilinear interpolation to the size of the low-level features.

4. Two 1×1 convolutions compute logits from this result and from the low-level features, and the two logit maps are summed.

Unlike Figure 3.4, which uses a 49×49 pooling kernel, TorchVision pools over the whole feature map.

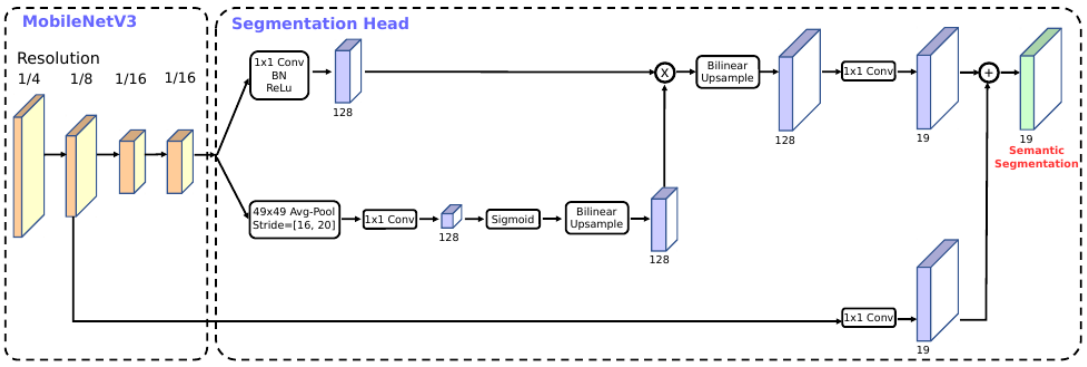


Figure 3.4: LR-ASPP head on the MobileNetV3 backbone, with 19 output classes. Adapted from Howard et al. [9].

3.5 SAM 3

SAM 3 (Section 2.3) consists of a detector for images and a tracker with a memory for videos. Both take vision-language inputs from a shared Perception Encoder (PE) backbone [11, Section 3]. Figure 3.5 shows the architecture. The labels of this thesis were generated from text prompts on single images, so only the detector is used.

The detector follows the DETR paradigm, in which a set of learned object queries proposes the objects. PE encodes the image and the text prompt. A fusion encoder conditions the image embeddings on the prompt tokens through cross-attention. In the decoder, the object queries cross-attend to the conditioned image embeddings, and each query predicts whether its object corresponds to the prompt, together with a box and a mask. A learned global token, the presence token, predicts only whether the concept is present in the image. Each query then only has to locate its object, and its final score is the product of its own score and the presence score [11, Section 3].

SAM 3 has about 850 million parameters, of which about 450 million are in the vision encoder and 300 million in the text encoder [11, Appendix C.1]. Its reported time of 30 ms for one image refers to an NVIDIA H200 data-center GPU [11, Section 1]. SAM 3 was therefore run offline (Section 4.2.2). The robot runs an LR-ASPP network trained on its labels (Section 4.4.3).

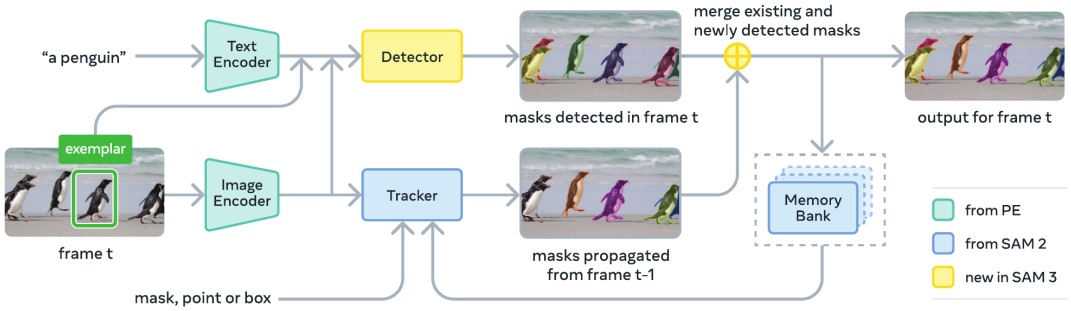


Figure 3.5: SAM 3 architecture. The detector segments the instances of the prompted concept in each image, and the tracker propagates the masks between video frames. Only the detector is used in this thesis. Adapted from Carion et al. [11].

3.6 Summary

Table 3.1 summarizes the four models of this chapter.

Table 3.1: Models described in this chapter.

Model	Backbone	Head	Role in this thesis
FCN	ResNet-50, output stride 8	`FCNHead`: 3×3 and 1×1 convolutions	Compared in Chapter 4
DeepLabV3	ResNet-50, output stride 8	ASPP with rates 12, 24, 36 and image pooling	Compared in Chapter 4
LR-ASPP	MobileNetV3-Large, output stride 16	Pooling gate on stride-16 features, logits summed with stride-8 features	Compared in Chapter 4, network on the robot
SAM 3	Perception Encoder (image and text)	DETR-like detector with presence token	Offline labeling for the networks trained without manual annotation

Chapter 4

Dataset Preparation and Comparative Model Training

This chapter describes the datasets and two experiments on the segmentation network. The first compares FCN, DeepLabV3, and LR-ASPP, fine-tuned from pretrained weights on a manually annotated lettuce dataset, and selects LR-ASPP for the robot. Manual annotation was done only for lettuce, because it is time-consuming. The second experiment tests whether LR-ASPP networks trained on masks generated by SAM 3, without manual correction, can replace manual annotation. The binary labeling was then extended to all crops, and a network trained on this collection is the one deployed on the robot (Section 4.4.3).

4.1 Image acquisition and class definition

4.1.1 Image sources and acquisition conditions

▶ **DA COMPLETARE: Origine delle immagini e condizioni di acquisizione**

Specificare la sorgente dei dati e le modalità di raccolta:

- Dispositivo di ripresa utilizzato (risoluzione e ottica), quota di acquisizione e angolazione.
- Condizioni operative nei campi (aziende agricole o serre sperimentali, condizioni meteo e fasce orarie).
- Numero complessivo di immagini raccolte per lattuga, pomodoro e cetriolo.



Figure 4.1: Sample field images across crops and corresponding training masks

4.1.2 Semantic classes and labeling targets

The segmentation labels in this thesis use up to four semantic classes: background, plants, path (the traversable corridor between two crop rows), and covering material (plastic mulch film). The manual lettuce annotations use all four classes (Section 4.2.1).

The labels generated with SAM 3 (Section 4.2.2) follow three configurations, each used to train a separate network:

- **Four-class configuration:** background, plants, path, and covering material, matching the four classes of the manual lettuce annotations.
- **Three-class configuration:** background, plants, and path. No prompt targets the covering material.
- **Binary configuration:** background and path. This configuration is used both for the lettuce-only binary network and for the multi-crop dataset.

For row following, the path mask is the input of the geometric extraction in Chapter 5.

4.2 Dataset preparation

4.2.1 Lettuce dataset with manual annotation

The manually annotated lettuce dataset, `Data`, contains 762 image–mask pairs from eight folders, with masks prepared from the manual annotations in `annotations.json`. These masks are the ground truth for all the evaluations of this chapter. They contain four classes (background, plants, path, and covering material), and the path class marks only the corridor the robot should follow, not other visible paths. The SAM 3 labels of the same images form the datasets `Data_insalataSAM_4class`, `Data_insalataSAM_3class`, and `Data_insalataSAM_2class` (Section 4.2.2).

4.2.2 Automated zero-shot annotation via SAM 3

Manual segmentation takes a lot of human work, so this experiment tests whether SAM 3 can replace it. SAM 3 was run offline with text prompts, and its masks were used for training without inspection, filtering, or correction. Manual masks were available for lettuce and were more accurate, but the unedited masks were kept on purpose, to measure how well a network performs when the annotation is fully automatic.

All autolabel masks were generated using `SAM3SemanticPredictor` with the local `sam3.pt` checkpoint and a confidence threshold of 0.25. For each text prompt, SAM 3 returned zero or more candidate masks, and pixels with mask values greater than 0.5 were assigned to the corresponding class. When masks of different prompts overlapped, the class was assigned according to a fixed prompt order, listed below for each dataset: later prompts were written only on pixels still labeled as background, so earlier classes were never overwritten. Pixels not assigned by any prompt remained background. The prompts and their order are:

- `Data_insalataSAM_4class` (lettuce images): `plants` (class 1), then `plastic mulch film` (class 3), then `central farm path` (class 2)
- `Data_insalataSAM_3class` (lettuce images): `plants` (class 1), then `central farm path` (class 2)
- `Data_insalataSAM_2class` (lettuce images): `central farm path` only (class 1)
- `Data_allPlantsSAM2` (images of lettuce, tomato, and cucumber crops): `central path` only (class 1).

The SAM 3 masks are not compared directly with the manual annotations. Their quality enters the evaluation only through the networks trained on them (Section 4.4.1).



Figure 4.2: SAM 3 zero-shot autolabeling and mask composition workflow

4.2.3 Tomato and cucumber images

The tomato and cucumber images were labeled only by SAM 3 and are used only in the multi-crop dataset `Data_allPlantsSAM2`, on which the binary network

deployed on the robot is trained. They have no manual ground truth, so they are not part of the quantitative evaluations of Section 4.4.

4.2.4 Dataset partitioning

The lettuce networks use a random 80/10/10 split of the 762 pairs: 609 training, 76 validation, and 77 test images. The training split updates the weights, the validation split gives the loss at the end of each epoch and selects the best checkpoint, and the test split is used only for the final scores. The three SAM 3 lettuce datasets contain the same 762 images in the same order and use the same split file as the manual model, so the 77 test images are the same for all four lettuce networks and none of them is used for training. For the multi-crop network evaluated in Section 4.4.3, the 77 lettuce test images were removed from `Data_allPlantsSAM2`, and the remaining 1882 of 1959 pairs were split 80/10/10.

The split is random at the level of single images. Each lettuce folder is one video recorded along a different path and contributes images to both the training and the test split, so the networks are trained on other photos of the same paths as the test images. The IoU values may therefore overestimate the performance on new paths or fields. This applies to all the networks of the comparison.

The realistic scenario uses only the 42 test images of `insalata_grande1`, `insalata_grande2`, and `insalata_media1`. These folders were recorded between parallel rows, where the robot operates, from viewpoints the robot can reach. The other five folders were recorded from viewpoints the robot cannot reach and can show more than one path, while the manual masks label only the path the robot should follow. The realistic scenario is a subset of the test split, not a separate test set.

4.3 Architecture comparison: FCN, DeepLabV3, and LR-ASPP

4.3.1 Model selection and pretrained weights

FCN, DeepLabV3, and LR-ASPP were chosen among the implementations available in TorchVision, for practical reasons and not after a comparison with every available architecture. Each network has a backbone, which extracts image features, and a segmentation head, which gives the class scores. All three start from pretrained parameters, and only their output layers are adapted to the target classes.

For FCN and DeepLabV3, the selected implementations use a ResNet-50 backbone with pretrained segmentation weights. After these weights are loaded, the final convolution of the main classifier is replaced by a new 1×1 convolution with

one output channel per target class (4 channels). The final convolution of the auxiliary classifier is replaced in the same way.

LR-ASPP is initialized in the same way. The implementation loads the default pretrained weights of the complete LR-ASPP model with a MobileNetV3-Large backbone. Its segmentation head ends with two 1×1 convolutions, one applied to the low-level features and one to the high-level features [16]. Both are replaced by new convolutions with 4 output channels. LR-ASPP has no auxiliary classifier.

The default weights of the three models were trained on a subset of COCO restricted to the 20 categories of Pascal VOC [14, 15, 16]. All three networks therefore start from pretrained segmentation weights of the same origin, and only their final output convolutions are newly initialized.

4.3.2 Training configuration and loss formulation

Loss function and class weighting

To keep the classes with many pixels from dominating the training, the models use a weighted cross-entropy loss $\mathcal{L}_{\mathrm{WCE}}$, in which each pixel is multiplied by the weight w_c of its ground-truth class c. The weights are computed on the training split only. With N_c the number of pixels of class c in all the training masks and C the number of classes (four in this comparison),

$$\mathrm{inv}_c = \frac{\sum_{j=0}^{C-1} N_j}{C \cdot N_c}, \qquad w_c = \frac{\mathrm{inv}_c}{\frac{1}{C}\sum_{j=0}^{C-1} \mathrm{inv}_j}. \tag{4.1}$$

A rare class receives a large weight and a frequent class a small one, and the weights have mean 1.

For FCN and DeepLabV3, which incorporate an auxiliary classifier on the `layer3` features (output stride 8 in the dilated TorchVision backbone), the total training objective combines the primary and auxiliary losses with an auxiliary weighting factor of 0.4:

$$\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{WCE}}(\mathrm{main}) + 0.4 \cdot \mathcal{L}_{\mathrm{WCE}}(\mathrm{aux}). \tag{4.2}$$

For LR-ASPP, which lacks an auxiliary head, training relies solely on the primary loss $\mathcal{L}_{\mathrm{WCE}}(\mathrm{main})$.

Data augmentation and two-stage optimization

Data augmentation is applied only to the training subset: random horizontal flips with probability 0.5, random rotations between $-30°$ and $30°$, and, with probability 0.8, a color jitter of brightness and contrast (factors in $[0.8, 1.2]$) and saturation (factors in $[0.85, 1.15]$) to handle variable outdoor illumination. The validation and

test sets are not augmented. Images are not resized, and all images are normalized with the ImageNet mean and standard deviation.

The networks were trained on an NVIDIA GeForce RTX 2080 SUPER with 8 GB of memory, with a batch size of 2.

Optimization proceeds across 200 epochs in two distinct phases:

1. **Stage 1 (Warmup, Epochs 1–10):** Backbone parameters are frozen by setting `requires_grad = False`. Only the segmentation heads (and auxiliary classifiers when present) are optimized using AdamW with an initial learning rate of 10^{-3} and weight decay of 10^{-4}. A cosine-annealing scheduler with $T_{\max} = 200$ and $\eta_{\min} = 10^{-6}$ updates the learning rate after each epoch. In the first epochs, the newly initialized output layers make large errors, and the resulting large gradients could damage the pretrained weights. Freezing the reused layers during these epochs gives the new layers time to learn reasonable weights [10, Chapter 11].

2. **Stage 2 (Differential Fine-Tuning, Epochs 11–200):** At epoch 10, the backbone is unfrozen. A new AdamW optimizer is instantiated with decoupled parameter groups: the backbone is assigned a learning rate of 10^{-5} and the segmentation heads a learning rate of 10^{-4}. The lower learning rate of the unfrozen backbone limits changes to its pretrained weights [10, Chapter 11]. The cosine-annealing scheduler is re-initialized across the remaining 190 epochs ($T_{\max} = 190, \eta_{\min} = 10^{-6}$).

Early stopping was not used: all three networks were trained for the full 200 epochs, so that the complete training and validation loss curves could be recorded. After each epoch, the model is evaluated on the unaugmented validation set. The weights are saved whenever the validation loss falls below its previous minimum, and the weights of the final epoch are saved separately.

4.3.3 Evaluation metrics and results

The three networks are evaluated on the 77 test images with the intersection over union of each class c,

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{4.3}$$

where TP_c, FP_c, and FN_c are the true positive, false positive, and false negative pixels of class c. The mIoU is the mean over the four classes, background included. The inference time is measured on the training GPU, not on the robot (Section 6.4.1), one image at a time at its original resolution, and covers the forward pass and the argmax. The epoch column of Table 4.1 gives the epoch with the lowest validation loss, whose weights are evaluated.

FCN and DeepLabV3 reach the same mIoU (0.9253), 1.08 percentage points above LR-ASPP, and LR-ASPP has the lowest IoU in every class (Table 4.2). LR-ASPP is 7.2 times faster than FCN and 10.6 times faster than DeepLabV3.

For FCN and DeepLabV3, the validation loss reaches its minimum early (Figure 4.3). After that, both networks start to overfit, and their validation loss rises while the training loss keeps decreasing. For LR-ASPP, the validation loss reaches a plateau around epoch 75 and does not rise until epoch 200. The losses of FCN and DeepLabV3 include the auxiliary term of Eq. (4.2), while those of LR-ASPP do not, so the absolute values are not comparable across networks.

Table 4.1: Architecture comparison on the 77 test images of the manual lettuce dataset. Parameter counts refer to the trained four-class models and include the auxiliary classifier of FCN and DeepLabV3, which is used only during training.

Model	Parameters (M)	Epoch	mIoU	Path IoU	Mean time (ms)	FPS
FCN	35.3	38	0.9253	0.9654	59.10	16.92
DeepLabV3	42.0	43	0.9253	0.9686	86.90	11.51
LR-ASPP	3.2	162	0.9145	0.9532	8.21	121.87

Table 4.2: Per-class IoU on the same 77 test images.

Model	Background	Plants	Path	Covering material
FCN	0.9493	0.9385	0.9654	0.8479
DeepLabV3	0.9494	0.9384	0.9686	0.8448
LR-ASPP	0.9439	0.9262	0.9532	0.8346

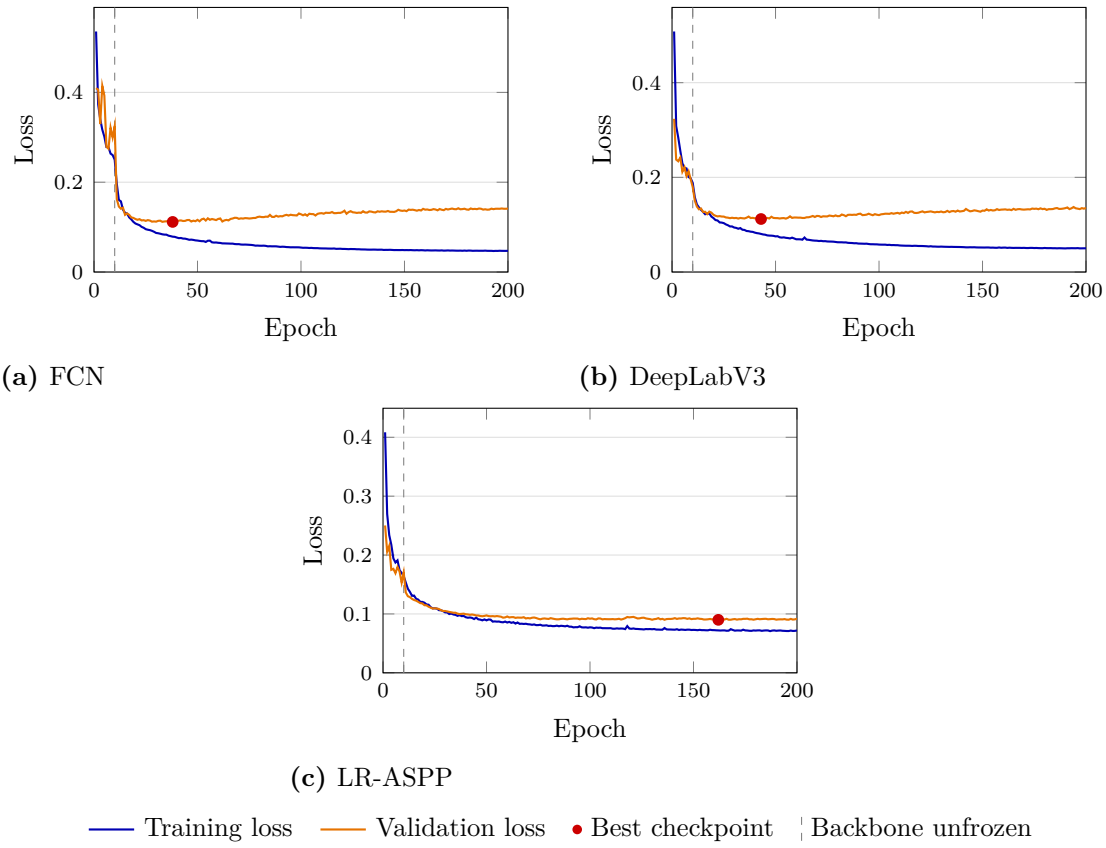
4.3.4 Selection of LR-ASPP

LR-ASPP was selected for its speed. Its mIoU is 1.08 percentage points below that of FCN and DeepLabV3, but it is 7 to 11 times faster. The SAM 3-supervised networks of the next section therefore all use LR-ASPP.

4.4 Manual versus SAM 3 labels

4.4.1 Training configurations and evaluation

Table 4.3 lists the four LR-ASPP networks of this comparison: the manual baseline, trained on the manual labels, and three networks trained on SAM 3 labels, called



(a) FCN **(b)** DeepLabV3

(c) LR-ASPP

Figure 4.3: Training and validation loss per epoch of FCN, DeepLabV3, and LR-ASPP on the manual lettuce dataset. The best checkpoint is the epoch with the minimum validation loss. The backbone is unfrozen after epoch 10.

SAM 3-supervised below. All four are trained on lettuce images only, so that their predictions can be evaluated against the manual ground truth. The extension to the multi-crop dataset is described in Section 4.4.3.

Table 4.3: LR-ASPP training configurations in the supervision comparison. All networks are trained on the same lettuce images and evaluated against the manual lettuce ground truth. The epoch is that of the evaluated weights, those with the lowest validation loss.

Configuration	Labels	Output classes	Epoch
Manual baseline	Manual	Background, plants, path, covering material	162
SAM 3-supervised four-class	SAM 3	Background, plants, path, covering material	30
SAM 3-supervised three-class	SAM 3	Background, plants, path	38
SAM 3-supervised binary	SAM 3	Background, path	18

The SAM 3-supervised networks reach their minimum validation loss much earlier than the manual baseline, between epochs 18 and 38 against epoch 162. A possible explanation, not verified in this work, is the noise of the SAM 3 labels, which are also the labels of their validation split. Arpit et al. [19] observed that a network trained with partly random labels first learns the patterns shared by the real data and only then fits the noisy labels, so its validation accuracy reaches its maximum before the training accuracy becomes high. Their labels were randomly corrupted, while the SAM 3 errors depend on the prompts and on the scene, so their result is only an analogy.

Each network is evaluated with the IoU of Eq. (4.3) against the manual labels, on two subsets of the 77 test images:

- **Complete test set (77 images):** all eight lettuce folders, including viewpoints that the robot cannot reach.
- **Realistic scenario (42 images):** the test images of the three folders recorded between parallel rows (Section 4.2.4).

4.4.2 Results on the lettuce test images

Each SAM 3-supervised network is compared with the manual baseline on the classes the two networks share. The background is not compared for the three-class and binary networks. In the three-class labels it also contains the covering material, and in the binary labels it also contains the plants and the covering material, so it does not match the background class of the manual masks. Table 4.4 compares the

four-class networks on all classes, Table 4.5 compares the plants and path classes of the three-class network, and Table 4.6 compares the path class of the binary network.

Table 4.4: IoU of the four-class networks on the test set.

Scenario	Model	Background	Plants	Path	Covering material	mIoU
Complete test set	Manual baseline	0.9439	0.9262	0.9532	0.8346	0.9145
	SAM 3-supervised	0.4631	0.8081	0.4124	0.1719	0.4639
Realistic scenario	Manual baseline	0.9203	0.9485	0.9587	0.8178	0.9113
	SAM 3-supervised	0.3449	0.8757	0.8997	0.2495	0.5925

Table 4.5: IoU of the three-class network on plants and path. The mean is computed over these two classes only.

Scenario	Model	Plants IoU	Path IoU	Mean IoU
Complete test set	Manual baseline	0.9262	0.9532	0.9397
	SAM 3-supervised	0.8130	0.4124	0.6127
Realistic scenario	Manual baseline	0.9485	0.9587	0.9536
	SAM 3-supervised	0.8779	0.8966	0.8873

Table 4.6: Path IoU of the binary network trained on lettuce only.

Scenario	Model	Path IoU
Complete test set	Manual baseline	0.9532
	SAM 3-supervised	0.3917
Realistic scenario	Manual baseline	0.9587
	SAM 3-supervised	0.9013

In the four-class network, the IoU of the covering material and of the background is low. Visual inspection of the SAM 3 masks showed that SAM 3 segmented the covering material poorly in general and also assigned the greenhouse plastic cover to this class, which is background in the manual masks. With an mIoU of 0.5925 in the realistic scenario, against 0.9113 for the manual baseline, the four-class SAM 3 labels do not replace manual annotation when all four classes are needed.

All SAM 3-supervised networks lose much of their path accuracy on the complete test set (path IoU 0.39–0.41), while the manual model does not (0.9532). The drop

comes from the five folders outside the operational scenario, whose images were taken from viewpoints the robot cannot reach. In these images the manual masks label only the path the robot should follow. Visual inspection showed that the SAM 3 path labels also cover other parts of the ground, such as adjacent paths and soil outside the corridor, especially where the path between the rows is not clearly visible. The networks learn this wider path class, and the IoU counts the additional ground pixels as false positives. The complete-test values therefore combine segmentation errors and this difference between the SAM 3 and manual labels. The two effects were not separated quantitatively.

In the realistic scenario, the path IoU of the three SAM 3-supervised networks is between 0.8966 and 0.9013, 5.7 to 6.2 percentage points below the manual baseline (0.9587). The three SAM 3-supervised networks differ from each other by less than 0.5 percentage points, so these data do not show that removing the plant and covering material classes improves path segmentation. The geometric extraction uses only the path class (Chapter 5), so the binary configuration was chosen for the robot. Whether its accuracy is sufficient for row following is assessed in the offline and field tests of Chapter 7.

4.4.3 Extension to the multi-crop dataset and deployed network

After the lettuce study, the binary labeling was extended to the complete image collection, with images of lettuce, tomato, and cucumber crops, labeled entirely by SAM 3 with the `central path` prompt and without manual correction. The binary LR-ASPP network trained on this collection is the one deployed on the robot and used in the following chapters (Chapters 6 and 7).

The training data of the deployed network include the lettuce test images, so its IoU on them would not be an independent measure. A second network with the same configuration was therefore trained on the collection without these images (Table 4.7). Its results estimate the accuracy of the configuration, not of the exact deployed weights. On the complete test set it is clearly better than the lettuce-only binary network (0.6369 against 0.3917).

Table 4.7: Path IoU of the multi-crop binary network, trained without the lettuce test images.

Scenario	Model	Path IoU
Complete test set	Manual baseline	0.9532
	SAM 3-supervised binary, multi-crop	0.6369
Realistic scenario	Manual baseline	0.9587
	SAM 3-supervised binary, multi-crop	0.9071

Chapter 5

Extraction of the Local Navigation Reference

This chapter describes five path extractors, two of which were developed in this work. The first, **Strip-IRLS**, is the main one. It extracts the path intervals in horizontal strips of the predicted mask. Going upward from the bottom of the image, it links in each strip the interval that continues the corridor found below, and it keeps the chain that gives the most edge samples. It then fits the left and right edges of the corridor separately with iteratively reweighted least squares. **EdgeNet**, still at an early stage, is a small network that takes the same mask and regresses the two corridor edges directly. The other three are geometric baselines: **Center points**, which fits one line to the midpoints of the intervals, **Column histogram**, which takes the column with the most path pixels, and **RANSAC edges**, which fits the two edges with RANSAC on the row extremes of the corridor. All five extractors take the same input mask and return the same outputs, a straight centerline, a confidence and a status, `ok`, `degraded` or `lost`, so the controller node can run any of them. The controller does not use the centerline of a `lost` result (Chapter 6). The extractor is chosen at start-up with a ROS parameter. All five were tested on the robot. Section 5.1 defines the input mask and the line models, Sections 5.2 to 5.4 describe the extractors, and Section 5.5 compares their structure and lists the limitations of **Strip-IRLS**.

5.1 Input mask and line models

All five extractors take the mask M predicted by the deployed binary network (Chapter 4), of height H and width W, with x increasing to the right and y downward. On the robot, the 640×480 camera image is resized to 224×224 pixels before the segmentation network, so $H = W = 224$ (Chapter 6). Path pixels have

value 1 and all other pixels have value 0. Lines are written as functions of y, which avoids infinite slopes for vertical lines. All distances are in pixels of the input mask.

Strip-IRLS, *EdgeNet* and *RANSAC edges* model the two corridor edges as separate straight lines, each with its own slope and intercept,

$$x_L(y) = m_L y + q_L, \qquad x_R(y) = m_R y + q_R, \tag{5.1}$$

and take the centerline as their mean,

$$x_C(y) = \frac{x_L(y) + x_R(y)}{2} = m_C y + q_C, \qquad m_C = \frac{m_L + m_R}{2}, \quad q_C = \frac{q_L + q_R}{2}. \tag{5.2}$$

The edges need not be parallel, so the modeled width can decrease toward the top of the image because of perspective. The slopes and intercepts are not constrained, so the lines can follow the corridor for any position and heading of the robot inside the row. *Strip-IRLS* then rejects the edges that do not converge upward (Section 5.2.5). *Center points* fits the centerline $x_C(y)$ directly to the midpoints of the corridor in each strip, without the edges. *Column histogram* returns a vertical centerline, with $m_C = 0$.

5.2 Strip-IRLS

Strip-IRLS works in five stages:

- Select a lower region of interest and clean the path mask (Section 5.2.1).
- Extract the path intervals in horizontal strips (Section 5.2.2).
- Associate intervals upward from each candidate seed and keep the best chain (Section 5.2.3).
- Fit the two edges separately (Section 5.2.4).
- Validate the edge geometry, assign a status from a confidence score, and return the centerline (Section 5.2.5).

5.2.1 Region of interest and mask cleaning

Like the three baselines of Section 5.4, *Strip-IRLS* uses only the lower part of the mask, the region of interest (ROI). The upper rows, where the path is not visible or cannot be distinguished, are excluded (Figure 5.1). The ROI starts at row $\lfloor \rho H \rfloor$ and ends at the bottom of the image. The ratio ρ must be chosen based on the position of the camera and on the path, which together set the highest row where the path appears in the image. The ROI should contain the path and as few other

rows as possible (Section 5.2.6). On the robot, $\rho = 0.30$, chosen on the development data (Section 7.1.2), gives row 67. When needed, *Strip-IRLS* extends the ROI slightly upward, so that it is filled exactly by strips of equal height (Section 5.2.2). On the robot, the ROI is extended from row 67 to row 64.

The path pixels inside the ROI are cleaned before the intervals are extracted. An opening with a 5×5 elliptical kernel removes thin protrusions and small isolated regions, then a closing with a 9×9 rectangular kernel fills small holes and gaps. The three baselines apply the same opening and closing. Finally, the connected components with fewer than 500 pixels are removed. In Figure 5.1, the dark gray regions inside the ROI are the pixels removed by the cleaning. Only the remaining path pixels, the cleaned mask $\widetilde{M}$, are passed to the interval extraction.

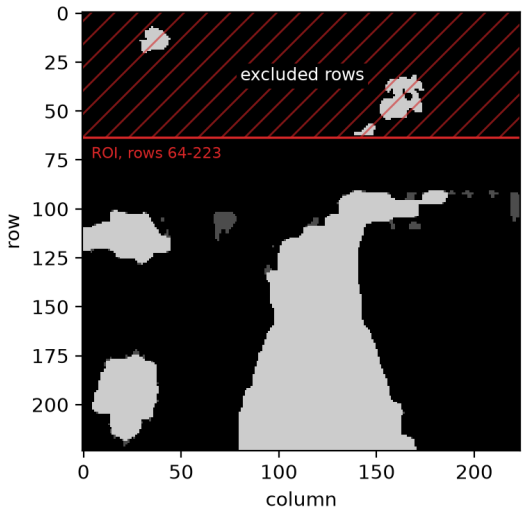


Figure 5.1: Example of the ROI and of the mask cleaning of Strip-IRLS on an example mask, with the parameters used on the robot. The rows above row 64 are excluded, and the dark gray regions inside the ROI are removed by the cleaning.

5.2.2 Horizontal strips and intervals

The ROI is divided into N strips of equal height, indexed from the bottom. N is a parameter of the extractor, set to 16 on the robot. The strip height is $h = \lceil (H - \lfloor \rho H \rfloor)/N \rceil$, which is the smallest height for which N strips cover the rows from $\lfloor \rho H \rfloor$ to the bottom of the image. When the number of these rows is not a multiple of N, the N strips cover a few more rows, so the ROI is extended upward and starts at $y_0 = H - Nh$. On the robot, the 157 rows from 67 to 223 are not a multiple of 16, so $h = 10$ px and the ROI starts at row 64. The extension

depends only on H, ρ and N, so the ROI is the same for every frame. Strip s covers the image rows from $H-(s+1)h$ to $H-sh-1$, and its center is at row $H-(s+0.5)h$. On the robot, strip 0 covers rows 214 to 223.

In each strip, a column is active if it contains at least one path pixel of $\widetilde{M}$. Runs of active columns separated by at most 30 inactive columns are merged into one interval $[\ell_i, r_i]$ of width $d_i = r_i - \ell_i + 1$, so that small holes and shadows inside the corridor do not split it.

An endpoint within 2 px of the image border is flagged as clipped. There the corridor leaves the image, so the endpoint is the image border and not the edge of the corridor. Two kinds of intervals are discarded: those narrower than 8 px, and those clipped on both sides when path pixels cover less than 70% of the strip.

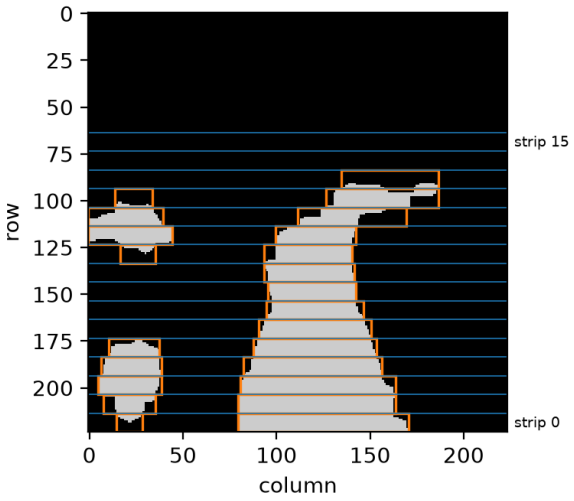


Figure 5.2: Strips and intervals on the mask of Figure 5.1, after the cleaning of Section 5.2.1, with the parameters used on the robot ($N = 16$, $h = 10$ px, ROI from row 64). The blue lines are the strip boundaries, and the orange boxes are the intervals found in each strip.

5.2.3 Bottom-up association and chain selection

The association decides which intervals belong to the corridor. The endpoints of these intervals are the samples of the two edge fits (Section 5.2.4). It works in two steps: one or more chains of intervals are grown upward from the bottom of the ROI, then one chain is selected.

Growing the chains. A chain starts from a seed, an interval of the bottom six strips, where the corridor is closest to the robot. On the robot, these six strips cover 60 image rows. From the seed, the chain is grown upward, one strip at a time. In each strip, each interval is compared with the last accepted interval $[\ell_p, r_p]$, of width d_p, through the boundary displacement $\delta_i = \max(|\ell_i - \ell_p|, |r_i - r_p|)$, which is the larger shift of its two endpoints. An interval is a candidate only if this shift is within the jump limit $\Delta_{\max}$. The limit is 28 px, or half the width of the last accepted interval when this is wider than 56 px. Among the candidates, the accepted interval is the one with the lowest cost J_i, which adds to the shift a penalty on the change of width:

$$\Delta_{\max} = \max(28,\, 0.5\, d_p), \qquad J_i = \delta_i + 0.4\, |d_i - d_p|. \tag{5.3}$$

The accepted interval becomes the reference for the next strip. The choice is greedy: an accepted interval is never revised later. A strip without candidates is skipped, and the reference does not change. The chain ends at the top of the ROI or at the fourth consecutive skipped strip, so on the robot it can cross a gap of up to 30 rows. No information is carried from one frame to the next.

The seeds are tried from strip 0 upward, and a chain is grown from each seed that does not already belong to an earlier chain. A chain grown from an interval of an earlier chain would start from the same reference and accept the same intervals, so it would only repeat the upper part of that chain. When the first chain contains all the intervals of the bottom six strips, there is only one chain. Other chains are grown when some of these intervals are not in the first chain, for example the intervals of a blob or of an adjacent corridor next to the path.

Selecting a chain. The complete chains are compared. The selected chain is the one that gives the most samples to the edge fits. The samples are the endpoints not clipped at the image border. An interval inside the image gives two samples, an interval clipped on one side gives one, and an interval clipped on both sides gives none. A chain along the image border therefore has fewer samples than a chain of the same length inside the corridor. Among chains with the same number of samples, the one with the lowest sum of the costs J_i is selected. If the bottom six strips contain no interval, or the selected chain has fewer than three intervals, the result is `lost`.

Figure 5.3 shows the association on the mask of Figure 5.2. The blob at the bottom left has more than 500 pixels, so the cleaning does not remove it, and it gives one interval in each of strips 0 to 4. Strip 0 contains two intervals, and the one on the left, in the blob, is tried first as a seed. Its chain accepts the blob intervals up to strip 4. The only interval of strip 5 belongs to the corridor, and it is shifted by 113 px from the interval accepted in strip 4, well beyond the jump limit of 28 px. The same happens in the next strips, so the chain finds no other candidate and ends at strip 4. The corridor interval of strip 0 is not in this chain,

so a second chain is grown from it. This chain accepts one interval in each strip up to strip 13. Strips 14 and 15 contain no interval, and the chain ends at the top of the ROI. In the bottom six strips, no interval is left outside the two chains, so no other seed is available and no other chain is grown (Figure 5.3a). The corridor chain has 28 samples and the blob chain has 10, so the corridor chain is selected, although its total cost is higher, 122 against 26.8 (Figure 5.3b). The mask also contains a second region separate from the corridor, at the top left, in strips 9 to 12. Its intervals are above the bottom six strips, so they are not seeds. In each of these strips, the corridor chain compares them with its last accepted interval: they are shifted by 96 to 136 px, well beyond the jump limit of 28–29 px, and they are rejected. They are not in any chain, and their endpoints are not used in the edge fits.

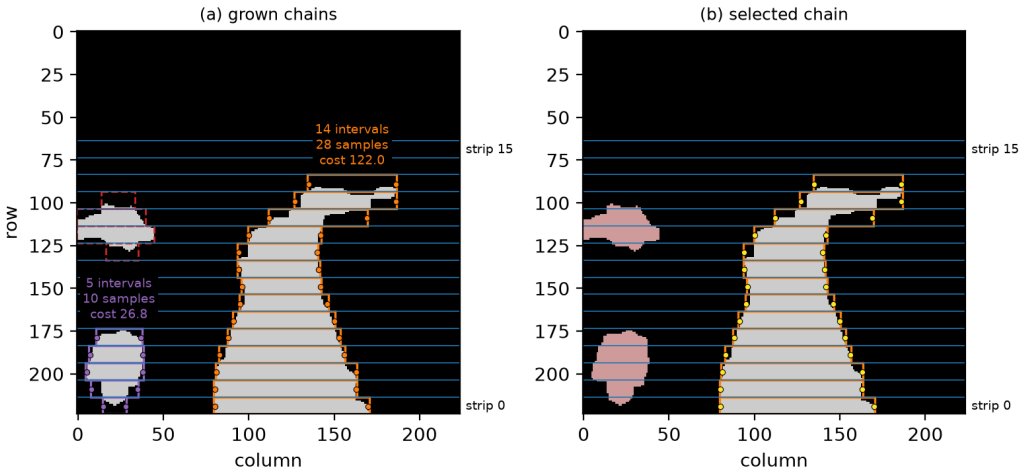


Figure 5.3: Chains built from the intervals of Figure 5.2. (a) The two chains grown from the seeds of strip 0, in purple from the blob and in orange from the corridor, with their number of intervals, samples and total cost. The dots are the samples, the endpoints that are not clipped. The dashed red boxes are intervals that are not in any chain. (b) The selected chain, the one with more samples. The yellow dots are its samples, used in the edge fits. The regions in light red are not in the selected chain and are not used in the fits.

5.2.4 Separate fitting of the corridor edges

The two edges are fitted separately. The left edge uses the left endpoints ℓ_i of the accepted intervals, and the right edge uses the right endpoints r_i. Clipped endpoints are not used, so an interval clipped on one side gives a sample only to the other edge, and the two edges can have samples in different strips. The row of

a sample is the center of its strip. In the equations below, a sample of one edge is the point (y_i, x_i), where y_i is its row and x_i is the endpoint, ℓ for the left edge and r for the right edge. In these equations i indexes the samples of one edge, not the intervals. Each edge needs at least two samples. If one edge has fewer, the result is `lost`, because the centerline is never estimated from a single edge. The edges are first fitted inside a crossing check, described at the end of this section, and then once more on the intervals that remain.

Each edge is fitted with iteratively reweighted least squares. The first fit uses uniform weights, and ten reweighting iterations follow. At each iteration, the distance of each sample from the current line gives its new weight, and the line is fitted again. A sample far from the line gets a low weight. How fast the weight decreases is set by the residual scale σ, a parameter of the extractor. A sample at distance σ from the line gets weight 0.5. On the robot, $\sigma = 18$ px (Section 5.2.6). At iteration t, the residuals and the new weights are

$$e_i^{(t)} = x_i - \left(m^{(t)} y_i + q^{(t)}\right), \qquad w_i^{(t+1)} = \frac{1}{1 + (e_i^{(t)}/\sigma)^2}, \tag{5.4}$$

and the line is fitted again by weighted least squares, with each weight applied to the residual before it is squared:

$$(m, q) = \arg\min_{m,q} \sum_i \left[w_i \Big(x_i - (m y_i + q) \Big) \right]^2 . \tag{5.5}$$

Each squared residual is therefore weighted by w_i^2. The weights are not accumulated across iterations, since they are computed again from the current line only. The reweighting reduces the effect of isolated outliers, but not of many samples from outside the corridor, because they lie on a consistent line and the fit follows them. Figure 5.4 shows the reweighting on the samples of the selected chain of Figure 5.3b.

The two edges of a corridor converge upward in the image, but in the rows where the chain has samples they are as far apart as the corridor is wide. Every accepted interval is at least $d_{\min} = 8$ px wide, the minimum interval width of Section 5.2.2. If the fitted edges come closer than $d_{\min}$ in these rows, the intervals above the row where this happens do not follow the corridor. They can come, for example, from a region attached to the corridor, and they bend the fits. The edges are therefore checked before the final fit. With $\Delta m = m_R - m_L$ and $\Delta q = q_R - q_L$, the modeled width is

$$D(y) = x_R(y) - x_L(y) = \Delta m\, y + \Delta q, \tag{5.6}$$

the signed horizontal distance between the two fitted lines at row y. It is negative where the lines have crossed. $D(y)$ is not the width of an interval, and it is defined at every row, also in the rows where the chain has no interval. The lines are fitted on the left and right endpoints. At the row of an interval $[\ell, r]$ not clipped on either

side, a good fit therefore gives D close to $r - \ell = d - 1$, not to the interval width d. When $\Delta m \neq 0$, $D(y)$ equals $d_{\min}$ at one value of y only,

$$y_{\times} = \frac{d_{\min} - \Delta q}{\Delta m}. \tag{5.7}$$

In general $y_{\times}$ is not an integer row. The edges converge upward when $\Delta m > 0$, in the code when $\Delta m > 10^{-6}$, and then $D(y) < d_{\min}$ above $y_{\times}$. If the edges converge upward and $y_{\times}$ is below the highest sample, the intervals whose strip center is above $y_{\times}$ are discarded, together with their samples on both edges, and both edges are fitted again. This crossing check is made at most three times. Each pass fits the edges, checks them, and makes at most one discard. After a discard in the third pass, the edges fitted on the remaining intervals are not checked again here. The check stops earlier in three cases:

- The check finds no crossing to treat: the edges do not converge upward, or $y_{\times}$ is not below the highest sample.
- The discard would leave fewer than three intervals. The intervals are then kept, and the check stops with the crossing still present.
- An edge has fewer than two samples, and the result is `lost`. The previous rule counts only the intervals, so this can also happen after a discard, when the remaining intervals are clipped on that side.

If the result is not `lost`, the final fit uses the remaining intervals. A crossing in the rows where the chain has samples can therefore still be present in three cases: when too few intervals would remain, after a discard in the third pass, or when the edges do not converge upward, which the check does not treat. These crossings are found by the width check of Section 5.2.5.

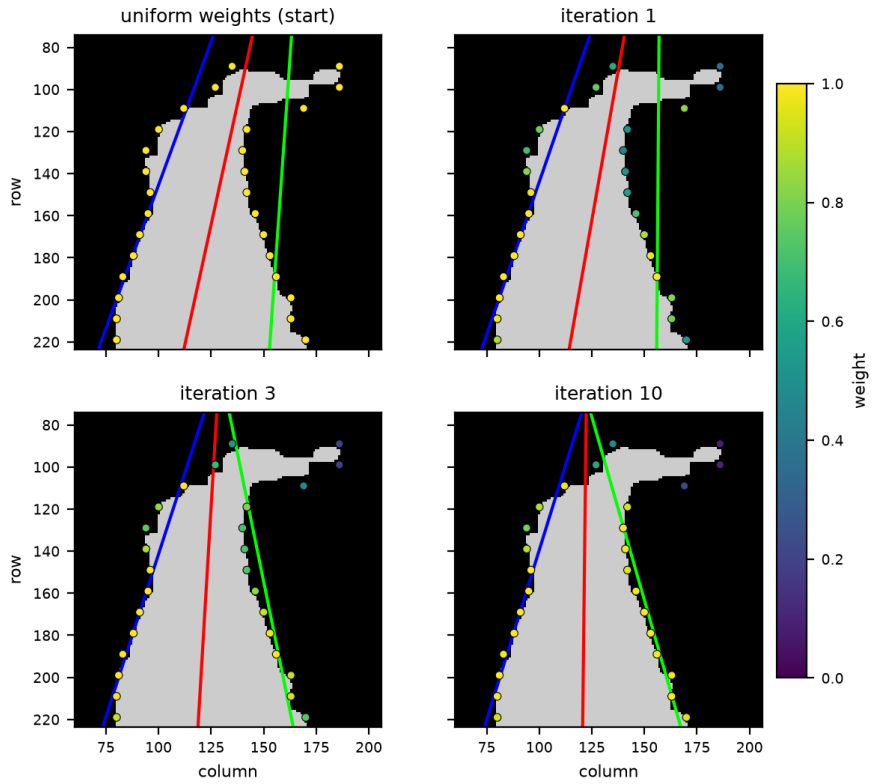


Figure 5.4: Reweighted fit of the two edges of Strip-IRLS on the samples of the selected chain of Figure 5.3b, at the start and after iterations 1, 3 and 10. The points are the samples of the chain, colored by their weight, the blue and green lines are the fits of the left and right edge, and the red line is the centerline, their mean. The three upper samples of the right edge, on the region attached to the corridor, pull the first fit toward the vertical. Their weights fall to 0.21, 0.10 and 0.09 only after several iterations, and after the tenth the line follows the edge.

5.2.5 Validation, confidence, and output

After the final fit, the width check tests the final edges. It uses the modeled width $D(y)$ of Eq. (5.6) and $y_\times$ of Eq. (5.7), both computed on the final edges, and the highest sample used in the final fit. When the edges converge upward, the valid range starts at the highest sample, or at $y_\times$ when $y_\times$ lies below the highest sample, and it ends at the bottom row of the image, row $H-1$. In these rows $D(y) \geq d_{\min}$. If the check passes, the valid range is used by the tracking score T of the confidence. The check fails in three cases:

- The edges do not converge upward, $\Delta m \leq 10^{-6}$ in the code. The robot moves between parallel rows, so because of perspective the edges of the corridor

converge upward. Parallel edges or edges that converge downward indicate a wrong fit.

- The edges converge upward and $y_\times$ lies more than $0.05H$, the crossing tolerance, below the highest sample. On the robot the tolerance is 11.2 px, a little more than the strip height of 10 px. The edges are then closer than $d_{\min}$ in rows where the chain has samples, which indicates a chain that joins different regions or a wrong fit.

- The distance from the top of the valid range to the bottom row of the image is less than 10% of the ROI height $H - y_0$. On the robot the ROI has 160 rows, so the limit is 16 px. The corridor is then seen in too few rows.

The first two cases find the crossings that the crossing check of Section 5.2.4 can leave in the final edges. A crossing at most $0.05H$ below the highest sample is accepted, and the valid range then starts at $y_\times$.

The confidence C combines four scores between 0 and 1. They use the seed and the accepted intervals of the selected chain that were not discarded by the crossing check of Section 5.2.4. Below they are called the intervals of the chain.

- **Tracking T.** The number of intervals of the chain whose strip center lies in the valid range, divided by the number of strips whose center lies in the valid range. T is 1 when every strip of the valid range has an interval of the chain, and it is lower when the chain skips strips. Intervals above the valid range are not counted, so that both counts refer to the same strips and $T \leq 1$. For example, when $y_\times$ lies below the highest sample within the tolerance, the valid range starts at $y_\times$ and the interval of the highest sample is not counted. Take a chain from strip 0 to strip 13 that skips two strips below strip 11. The crossing check discards the intervals of strips 12 and 13, and the valid range starts at the center of strip 11. The valid range then has 12 strips and 10 intervals, so $T = 10/12 \approx 0.83$. The two skipped strips lower T, while the two discarded intervals do not.

- **Fit F.** How close the samples are to the fitted edge lines. With $\bar{e}$ the mean horizontal distance of the samples of both edges from their lines, $F = \text{clip}(1 - \bar{e}/\sigma,0,1)$. F is 1 when the samples lie on the lines and falls to 0 when $\bar{e}$ reaches $\sigma = 18$ px. It is low when the edges of the mask are irregular or when some samples come from outside the corridor. The distances are not weighted, so a sample with a low weight in the fit still counts fully in $\bar{e}$.

- **Width regularity A.** With straight edges, the width of the corridor changes by the same amount from one strip to the next, so its second differences, the changes between consecutive changes of width, are zero. A uses the widths

$r_i - \ell_i$ of the intervals of the chain that are not clipped on either side, from the bottom up, since the width of a clipped interval is not the corridor width. The error E_D is the 90th percentile of the absolute second differences divided by the median width, and $A = \text{clip}(1 - E_D/0.30,0,1)$. A is 1 when the width changes by the same amount at every step and 0 when E_D reaches 0.30. With fewer than three unclipped intervals there is no second difference, so A is 0. For example, the widths 120, 114, 108, 102 and 96 px change by -6 px at every step, so $E_D = 0$ and $A = 1$. If a region attached to the corridor widens the third interval to 140 px, the changes are -6, $+26$, -38 and -6 px and the absolute second differences are 32, 64 and 32 px. Their 90th percentile is 57.6 px and the median width is 114 px, so $E_D = 0.51$ and $A = 0$. A skipped strip or an excluded clipped interval also gives a nonzero second difference when the width changes along the corridor, but a small one. Without the 108 px interval of the first example, the absolute second differences are 6 and 6 px and the median width is 108 px, so $E_D \approx 0.06$ and $A \approx 0.81$.

- **Gaps G.** The longest hole in the chain. With $g_{\max}$ the longest run of strips skipped between two intervals of the chain, $G = \text{clip}(1 - g_{\max}/3,0,1)$. G is 1 when the chain skips no strip and 0 when it skips three strips in a row, the longest gap the chain can cross. Every strip of the valid range without an interval of the chain lowers T, also below the seed. G depends only on the longest run of skipped strips between two intervals of the chain, so one long gap lowers G more than several short ones.

The confidence is their weighted sum:

$$C = 0.3125\,T + 0.3125\,F + 0.25\,A + 0.125\,G. \tag{5.8}$$

The result is `lost` if the width check fails, if $\bar{e} > 2\sigma$, or if $C < 0.38$. Otherwise it is `degraded` for $C < 0.60$ and `ok` for $C \geq 0.60$.

For a `lost` result, no centerline model is returned. Otherwise, the centerline model $x_C(y)$ is returned. The controller reads the lateral error from the model at the reference row $y_{\text{ref}} = 156$ (Section 6.2.3).

5.2.6 Setting the parameters

Table 5.1 lists the parameters of *Strip-IRLS* on the robot. Distances and areas are in pixels of the 224×224 mask, so the values depend on the camera mounting and on the resizing of Section 5.1. The ROI start and the merging gap were chosen on the development data, and the strips of 10 rows and the ten reweighting iterations were checked on the same data (Section 7.1.2). The other values were set by hand during the development, on the synthetic development masks. The main parameters are listed below.

Table 5.1: Parameters of Strip-IRLS used on the robot. Distances and areas are in pixels of the 224×224 mask.

Parameter	Value
Arguments of the extractor	
Path class	1
ROI start ratio ρ	0.30
Number of strips N	16
Minimum component area	500
Minimum interval width $d_{\min}$	8
Merging gap	30
Image-border margin	2
Minimum jump limit	28
Residual scale σ	18
Reweighting iterations	10
Seed window	6 strips
Consecutive skipped strips allowed	3
Crossing tolerance	$0.05H$
Minimum valid-range length	$0.10\,(H - y_0)$
Lost / ok thresholds on C	0.38 / 0.60
Constants in the code	
Opening / closing kernels	5×5 elliptical / 9×9 rectangular
Connectivity of the components	8
Occupancy for intervals clipped on both sides	0.70
Jump-limit factor on d_p	0.5
Width-change cost coefficient	0.4
Minimum intervals in the chain	3
Minimum samples per edge	2
Crossing-check passes	3
Convergence threshold on Δm	10^{-6}
Residual rejection factor	2 ($2\sigma = 36$)
Percentile of the second differences	90
Width-regularity scale	0.30
Weights of T, F, A, G in C	0.3125, 0.3125, 0.25, 0.125

- **ROI start ρ.** A smaller ρ adds rows above the path, whose regions can enter the chain. A larger ρ leaves fewer rows of the corridor for the fits.
- **Number of strips N.** Each strip gives at most one sample per edge. Thinner strips give more samples and a finer resolution along the edges, but a thin strip is emptied or split more easily by a hole in the mask. Thicker strips are less affected by holes, but they give fewer samples and a lower resolution. The seed window and the allowed skipped strips are counted in strips, so they have

to be changed with N to cover the same rows.

- **Merging gap.** It should be larger than the holes inside the corridor and smaller than the crop row between two adjacent corridors. If it is too large, a blob or an adjacent corridor is merged into the interval.
- **Minimum component area and minimum interval width $d_{\min}$.** They should be below the area of the parts of the corridor separated from the rest and below the narrowest width of the corridor. Smaller values keep blobs and noise that can enter a chain. $d_{\min}$ is also the threshold of the crossing check and of the width check, so a larger $d_{\min}$ shortens the valid range.
- **Jump limit $\Delta_{\max}$.** It should be above the shift of the corridor edges, also across three skipped strips, and below the shift to the regions next to the corridor. If it is too small, the chain stops on inclined edges. If it is too large, the regions next to the corridor become candidates.
- **Residual scale σ.** Samples much closer to the line than σ keep almost all their weight, and samples much farther lose most of it. σ should therefore be larger than the distance of the correct samples, which comes from the irregular edges of the mask, and smaller than the distance of the outliers. A smaller σ lowers F, so more results are `degraded` or `lost`. A larger σ keeps the outliers in the fit and raises the threshold 2σ.
- **Status thresholds.** Higher thresholds give more `lost` and `degraded` results, which the controller filters differently (Section 6.2.3).

The values are set for the camera, its mounting and the fields of this work. With another camera, mounting or crop, they may need to be adjusted to get the best results. This is one of the reasons for *EdgeNet* (Section 5.3).

5.3 EdgeNet, an edge regression network

EdgeNet is a first test of replacing the geometric steps of *Strip-IRLS* with a learned model. It takes the segmentation mask and not the image, and it regresses the two corridor edges. It has no geometric parameters to set by hand for each scenario.

The network is a small CNN of about 0.8 million parameters, shown in Figure 5.5. Its input is the 224×224 mask, and it has five outputs: the x positions of the left and right edges at rows 156 and 223, and the validity p_{valid}, the estimate of the network that the mask contains a corridor.

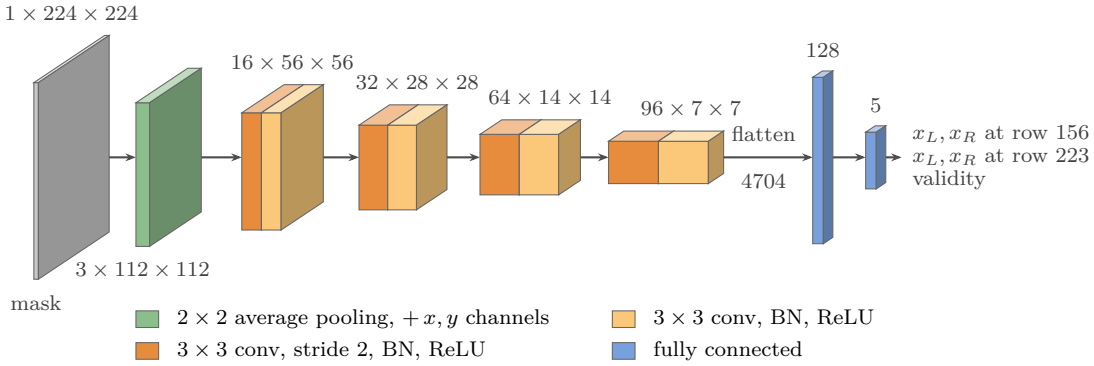


Figure 5.5: Architecture of *EdgeNet*. Each box is a feature map, colored by the layer that produces it and labeled with channels × height × width. The two maps of a block have the same size and share one label. The mask is averaged over 2×2 blocks, and two channels with the x and y coordinates of each pixel, scaled to $[-1,1]$ as in CoordConv [20], are added, so the convolutions also see where each pixel is. Each of the four blocks halves the size with a stride-2 convolution and then applies a second convolution. The $96 \times 7 \times 7$ map is flattened and passed to two fully connected layers, the first followed by ReLU. A sigmoid turns the last output into p_{valid}.

The four positions are given normalized as $(x - 112)/112$, so the columns 0 and 223 correspond to -1 and about 1. Each edge line passes through its two predicted points, and the centerline is the mean of the two lines, as in Eq. (5.2).

The confidence is $C = p_{\text{valid}} \cdot \text{IoU}$. The IoU compares the region between the two edge lines with the path pixels of the mask, not cleaned as in *Strip-IRLS*, in the rows from 64 to 223, the ROI of *Strip-IRLS*, which contains the row read by the controller:

$$\text{IoU} = \frac{N_{\text{in}}}{N_{\text{in}} + N_{\text{out}} + N_{\text{bg}}}, \tag{5.9}$$

where N_{in} is the number of path pixels between the lines, N_{out} the number of path pixels outside them, and N_{bg} the number of pixels between the lines that are not path. The IoU is 1 when the region between the lines covers exactly the path. Above the crossing of the lines the region between them is empty. The status is `lost` if $p_{\text{valid}} < 0.5$, if the distance between the two predicted edges at row 223 is below 8 px, or if $C < 0.38$. Otherwise it is `degraded` for $C < 0.60$ and `ok` for $C \geq 0.60$.

The network is trained only on synthetic masks. Its input has the same format as on the robot, and the labels are computed from the corridor model of the generator and not from an extractor, so they are exact, need no manual annotation and do not contain the errors of an algorithm. No segmentation network is used to produce

the training masks. The errors of the segmentation are drawn by the generator, with shapes and probabilities chosen in its code. The masks are therefore realistic only as far as these disturbances resemble the errors of the real segmentation. The generated masks look like the predicted ones, with irregular edges, holes, blobs, adjacent corridors and the end of the row.

The labels are the positions of the edge lines of the corridor model at rows 156 and 223, normalized as the outputs. The edge line of a wide or offset corridor can pass outside the image, and limiting it to the image would move the centerline. The labels are therefore clipped only to $[-2,2]$, from -112 to 336 px. In the training set, 36.8% of the valid masks have at least one label outside $[-1,1]$, and 8.2% have one clipped. The validity label is 1 when the corridor drawn before the disturbances has at least 800 pixels and at least one of its two edge lines is visible, more than 2 px from the image border, in at least 15 of the rows from 64 to 223. It is 0 for the other masks and for the masks without a corridor, so a corridor partly erased by the disturbances is still valid. About 14% of the 300 000 training masks have no valid corridor.

The loss is the sum of three terms: a smooth L1 loss on the four edge positions, twice the same loss on the centerline at row 156, which is the value used by the controller, and 0.2 times the binary cross-entropy of the validity. The first two terms are computed only on masks with a valid corridor.

Figure 5.6 shows three validation masks with their labels and three masks of the field tests with the lines estimated by the network. The network is evaluated on the synthetic test masks in Section 7.1.2 and in the field in Section 7.2.

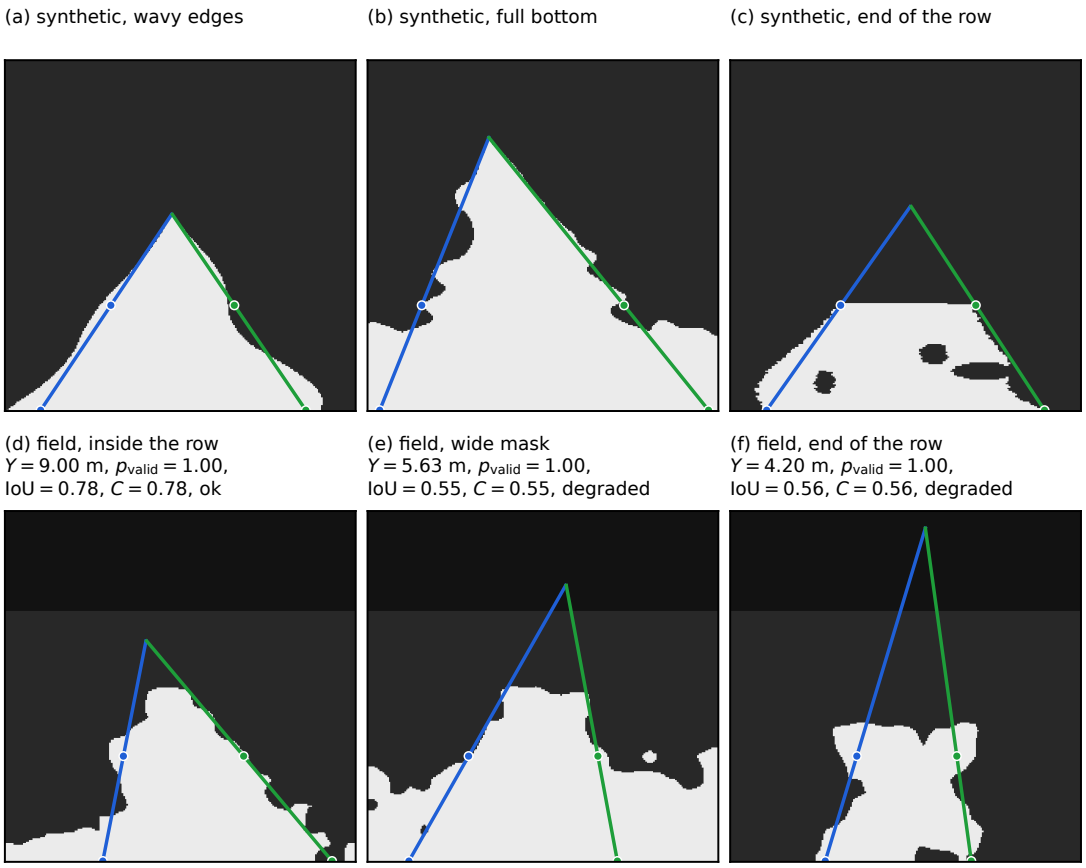


Figure 5.6: Masks and edge lines of *EdgeNet*. (a)–(c) Synthetic validation masks with the edge lines of their labels. (d)–(f) Masks of the field tests (Section 7.2, *EdgeNet* Run 1), with the edge lines estimated by *EdgeNet*. The dots are the positions at rows 156 and 223. The darker rows above row 64 are not used for the IoU.

5.4 Baseline extractors

Strip-IRLS and *EdgeNet* are compared with the three geometric baselines of this section.

5.4.1 Center points

Center points uses the same mask cleaning, ROI and interval extraction as *Strip-IRLS*, but with $N = 24$ strips. Each interval is reduced to its midpoint $c = (\ell + r)/2$ and its width $d = r - \ell + 1$. The seed is the interval of the bottom six strips with

the most path pixels. Intervals are associated upward with the cost

$$J_i = |c_i - c_p| + 0.4\,|d_i - d_p|, \tag{5.10}$$

where c_p and d_p are the midpoint and width of the last accepted interval, under the jump limit $|c_i - c_p| \leq \max(28, 0.5\,d_p)$. The stopping rule and the minimum of three intervals are those of *Strip-IRLS*.

One line is fitted to the midpoints with four iterations of the weights of Eq. (5.4), starting from weights proportional to the pixel count of each interval. Clipped intervals are not excluded. The confidence combines tracking, fit, width regularity and gap scores similar to those of *Strip-IRLS* (Section 5.2.5), with weights 0.30, 0.35, 0.25 and 0.10. The result is `lost` if the mean residual exceeds 2σ or if the confidence is below 0.38, and `degraded` if it is below 0.60. Section 5.5 discusses the effect of forming the midpoints before the fit.

5.4.2 Column histogram

Column histogram is a simplified version of SegMin (Section 2.4.1). It uses the path mask of a single frame instead of a plant mask accumulated over frames and refined with depth, it does not smooth the profile, and it takes the maximum instead of the minimum [3]. It cleans the ROI from row 67 as *Strip-IRLS*, removes the components with fewer than 500 pixels and counts the path pixels of each column. The target column x_{target} is the column with the most path pixels, or the center of the widest group of adjacent columns with the maximum count. If two groups have the same width, the result is `lost`. The confidence is the maximum count divided by the ROI height, with the thresholds 0.38 and 0.60. The output is the vertical line $x = x_{\text{target}}$, a lateral position without heading.

5.4.3 RANSAC edges

The works of Section 2.4.3 fit RANSAC lines to points of the crop rows. *RANSAC edges* fits them to the edges of the segmented corridor. It cleans the ROI from row 67 as *Strip-IRLS* and keeps one large connected component that starts low in the image. The leftmost and rightmost pixels of each row of the component are the edge points, except in rows where they are less than 10 px apart and at the image border.

Each edge is fitted with RANSAC [21]: in 100 trials, two random points define a line, and the points within 18 px of it are inliers. The line with the most inliers is refitted by least squares on its inliers, and the centerline is the mean of the two edge lines. The result is `lost` if an edge has fewer than 10 points or spans fewer than 20 rows, as at the end of a row, if the component has fewer than 1000 pixels in the ROI, or if the mean fraction of inliers is below 0.30. Otherwise a score

combines the fraction of inliers, their residual, the vertical extent of the component and the regularity of its width, with the thresholds 0.38 and 0.60.

5.5 Structural differences and limitations

Table 5.2 summarizes how the five extractors differ in their code. It does not describe measured performance, which is reported in Chapter 7.

Table 5.2: Structural comparison of the five extractors. Row extremes are the leftmost and rightmost pixels of the selected component in each ROI row. LS is a least-squares refit on the inliers.

Characteristic	Strip-IRLS	EdgeNet	Center points	Column histogram	RANSAC edges
Output	Line	Line	Line	Vertical line	Line
Heading estimate	Yes	Yes	Yes	No	Yes
First row used	64	0 (network), 64 (IoU)	56	67	67
Mask cleaning	Open, close, ≥500 px	None	Open, close, ≥500 px	Open, close, ≥500 px	Open, close, one component ≥500 px
Edges fitted separately	Yes	Regressed	No	N/A	Yes
Edge points	Chain endpoints	N/A	N/A	N/A	Row extremes
Clipped endpoints excluded	Yes	N/A	No	N/A	Yes
Bottom-up seed and jump limit	Yes	No	Yes	No	No
Line estimator	IRLS	CNN regression	IRLS	N/A	RANSAC, LS

The main structural difference between *Strip-IRLS* and *Center points* is where the two edges are combined: *Center points* combines them before the fit, in the midpoints, and *Strip-IRLS* after the fit, in the centerline. When a disturbance moves one edge, the midpoint moves by half of the displacement, so its distance from the fitted line is also halved. Both extractors use the weights of Eq. (5.4) with $\sigma = 18$ px. For example, an edge point displaced by 36 px gets weight 0.2 in the fit of its edge, while the corresponding midpoint, displaced by 18 px, gets weight 0.5, so the reweighting reduces the disturbance less in *Center points*. In addition, *Center points* does not exclude the clipped intervals, whose midpoint lies

between the image border and the other edge and is not the center of the corridor. On the synthetic masks with a widening on one side of the corridor, *Center points* has 11.5% gross errors and *Strip-IRLS* 3.1% (Section 7.1.2).

Strip-IRLS has three structural limitations:

- **Seed dependence.** The seeds are searched only in the bottom six strips. If these contain no interval, the result is `lost` even if the upper part of the corridor is segmented. If a hole wider than the merging gap and taller than these strips splits the bottom of the corridor, the chain follows only one part (Section 7.1.2). At a fork above these strips, the chain takes the branch with the lowest cost and never revises it.
- **Number of parameters.** *Strip-IRLS* has 16 arguments and 13 constants (Table 5.1), most of them in pixels of the mask, and several act together. They were set for the camera, its mounting and the fields of this work, and in a new scenario they may need to be adjusted to get the best results (Section 5.2.6).
- **Missing plants on one side.** The predicted path, and therefore the interval, can widen on that side. Within the jump limit it is accepted, and its endpoint on that side is a wrong sample that only the reweighting can reduce. Beyond the limit the strip is skipped. If the interval reaches the image border, its endpoint on that side gives no sample.

Chapter 6

Robot Platform and Integration

This chapter describes the robot used in the field tests, the software that runs on it, and the deployment of the segmentation network.

6.1 Robot hardware and sensors

6.1.1 Robot platform

The experimental platform is Demetra, a mobile robot prototype developed by the Roboto student team of the Politecnico di Torino. It has four-wheel steering (4WS) and four-wheel drive (4WD).

Each wheel has a DJI RoboMaster M3508 brushless motor with a DJI RoboMaster C620 speed controller, and its own CubeMars AK70-10 actuator for steering. A DJI RoboMaster Development Board Type C (STM32F407) controls the speed controllers and the steering actuators. It receives the linear velocity and the steering angle from the onboard computer and sends back the measured platform speed and steering angle. The wheelbase is $L = 40$ cm, the track width is $T = 34$ cm, the wheel diameter is 19 cm ($r_w = 9.5$ cm) and the chassis is 15 cm above the ground.

Steering is described by a kinematic bicycle model with opposite-phase steering (Figure 6.2), where the equivalent front and rear steering angles are $\delta_f = \delta$ and $\delta_r = -\delta$. With pure rolling, the turning radius of the mid-axle point M is

$$R = \frac{L}{2 \tan \delta}. \tag{6.1}$$

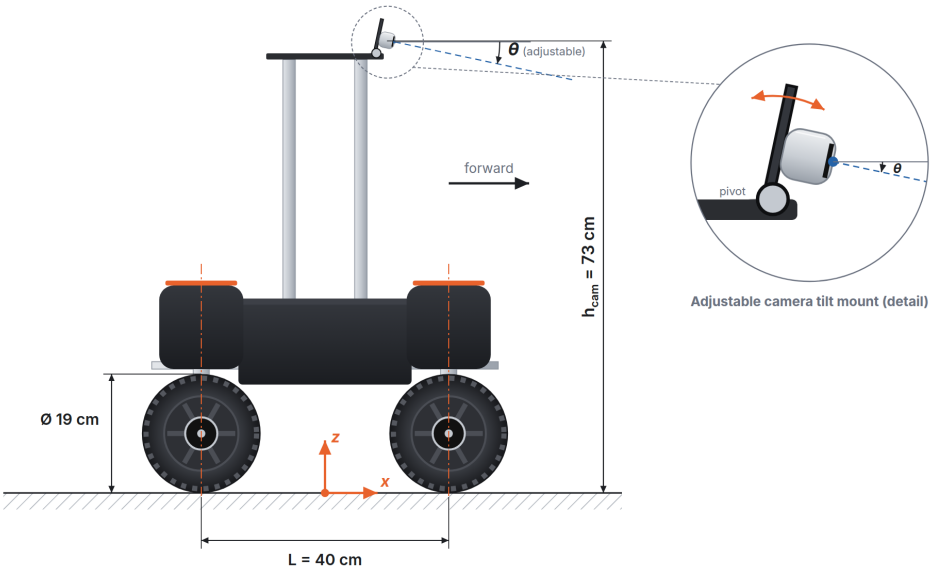


Figure 6.1: Side view of Demetra, with the wheel diameter, the wheelbase L, the camera height h_{cam} and the adjustable tilt θ of the camera.

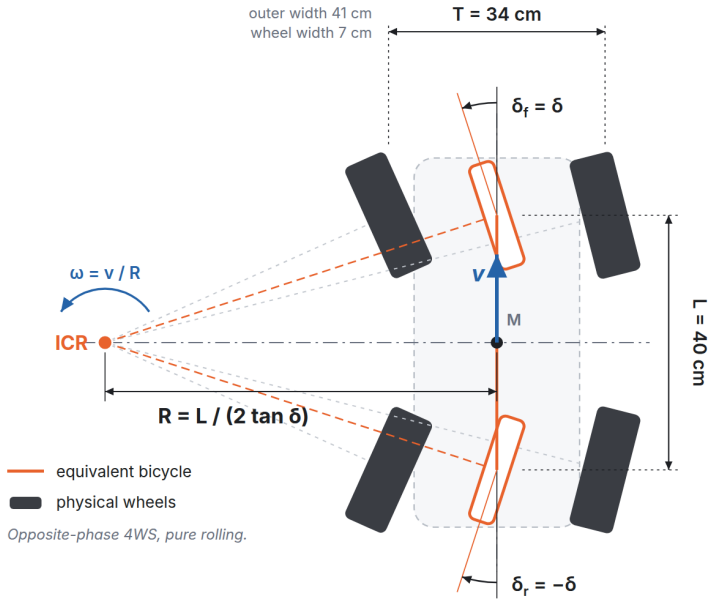


Figure 6.2: Top view of the opposite-phase 4WS kinematics of Demetra, with the equivalent bicycle in orange, the ICR and the turning radius R of the mid-axle point M (Equation (6.1)).

6.1.2 Camera configuration

The camera is an Intel RealSense D435, mounted on the front of the robot frame on a mount with adjustable tilt (Figure 6.1). The color stream is used at 640×480 pixels and 30 frames per second. The camera is tilted downward by a few degrees, so that the image shows mainly the path, which covers about the lower 70% of the image, as the region of interest of the extractors. The tilt is adjusted to the field.

6.1.3 Computing platform

The onboard computer is an NVIDIA Jetson AGX Orin Developer Kit (64 GB), with a 12-core ARM Cortex-A78AE CPU and an NVIDIA Ampere GPU with 2048 CUDA cores. It runs Ubuntu 22.04 with L4T R36.4.4. The software runs in a Docker container with access to the GPU and to the USB devices. Inside the container, ROS 2 Humble runs the camera driver, the controller node, and the serial driver.

6.2 Software architecture

6.2.1 ROS integration

The software pipeline is implemented in ROS 2. Figure 6.3 shows the node graph of the robot. Three nodes are active. The RealSense node (`/camera/camera`) publishes the compressed color images. The controller node (`/controller_node`) subscribes to these images. In the same callback, it runs the segmentation network and the path extractor, computes the velocity command (Section 6.2.3), and publishes it on `/cmd_vel`. The serial driver node (`/demetra_node`) receives the commands and sends them to the microcontroller. The controller node also publishes debug topics under `/debug`, which are used during configuration and are omitted from the figure. When the debug video is enabled, it also publishes three debug images (segmentation, region of interest and path), shown in Figure 6.4. The debug video makes the node skip camera frames (Section 6.4.1), so it should be disabled when the robot operates. The text log on `/debug/text` is always active. It carries important information on each frame and its cost is small, so the node still processes every camera frame.

The image subscription keeps only the last frame, so frames that arrive during the processing are not queued and the controller always processes the most recent image seen by the robot.

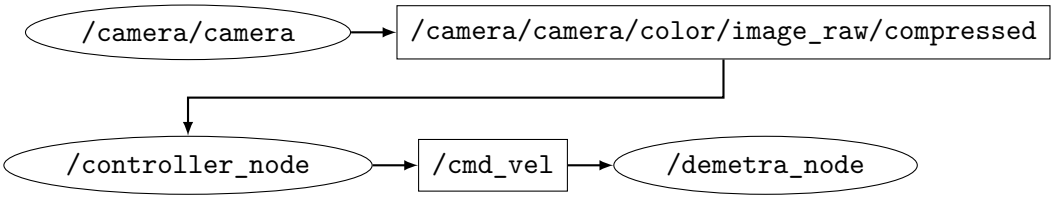


Figure 6.3: ROS 2 node graph on the robot, as shown by `rqt_graph`. Ellipses are nodes and rectangles are topics.

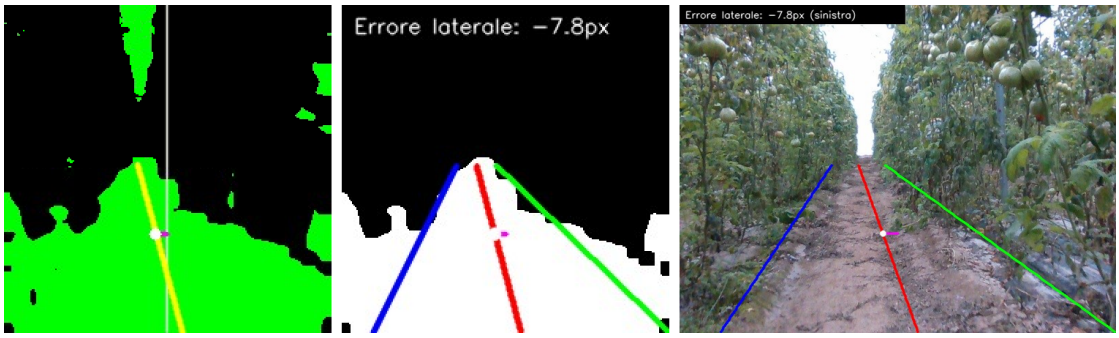


Figure 6.4: Frame of the debug video recorded during a field run with Strip-IRLS. From left to right: segmentation (the segmentation before the morphological cleaning), region of interest (the cleaned segmentation, on which the path extractor operates) and path (the extracted path drawn on the camera image).

6.2.2 Control interface

The serial driver node (`/demetra_node`) converts each command (v, ω) into the steering angle

$$\delta = \arctan\left(\frac{L\,\omega}{2\,v}\right), \tag{6.2}$$

limited to $\pm 30°$. Before the conversion, the node limits v to ± 1.5 m/s and ω to ± 6.0 rad/s. Every 10 ms it sends the last v and δ to the microcontroller. If no command arrives for 500 ms, it sets the speed to zero.

The microcontroller receives v and δ from the driver and actuates them with the steering actuators and the traction motors of the four wheels.

6.2.3 Controller node

For each frame, the controller node decodes the image, resizes it to 224×224 pixels and normalizes it with the ImageNet mean and standard deviation. The

segmentation network turns the image into the label mask, and the path extractor (Chapter 5) turns the mask into a centerline.

The lateral error e is the horizontal offset, in pixels, of the centerline from the image center at row $y_{\text{ref}} = 156$ of the $W \times H = 224 \times 224$ input. It is positive when the centerline is on the right. The error then passes through a filter that depends on the status returned by the path extractor (Section 5.2.5), a deadband and the steering law, which gives the angular velocity command.

Error filter. The error is smoothed with an exponential moving average (EMA). At each frame k, the filtered error is a weighted average of the previous filtered error $\hat{e}_{k-1}$ and of the new measurement e_k:

$$\hat{e}_k = (1 - \alpha)\, \hat{e}_{k-1} + \alpha\, e_k. \tag{6.3}$$

The smoothing factor α is the weight of the new value. With $\alpha = 0.20$ the filter keeps 80% of the previous value at each frame. If the measured error jumps to a new value and then stays constant, the filter covers about half of the step after three frames and 95.6% of it after 14 frames, which at 30 frames per second take 0.1 s and 0.47 s.

In practice the measured error changes at every frame, because of the noise of the segmentation and of the path extractor and because the robot moves. Replacing $\hat{e}_{k-1}$ with Equation (6.3) written for the previous frame, and repeating this for the older frames, gives

$$\hat{e}_k = \alpha\, e_k + \alpha(1 - \alpha)\, e_{k-1} + \alpha(1 - \alpha)^2\, e_{k-2} + \dots \tag{6.4}$$

Each measurement enters the filtered error with weight α, and its weight is multiplied by $1 - \alpha$ at every following frame. With $\alpha = 0.20$ the newest measurement weighs 0.20, the previous one $0.20 \cdot 0.8 = 0.16$ and the one before $0.20 \cdot 0.8^2 = 0.128$. The weights of the last n measurements add up to $1 - 0.8^n$, so more than 95% of the filtered value depends on the last 14 measurements. Within them, the most recent measurements count the most, while the oldest weigh almost nothing, for example $0.20 \cdot 0.8^{13} \approx 0.011$ for the fourteenth. A change that lasts one frame therefore enters the filtered error only with weight 0.20, while a change that lasts several frames enters all the recent measurements and is followed with a short delay. Figure 6.5 shows an example from a field run. In the 1.5 s shown, the mean difference between two consecutive frames is 10.8 px for the measured error and 4.2 px for the filtered error.

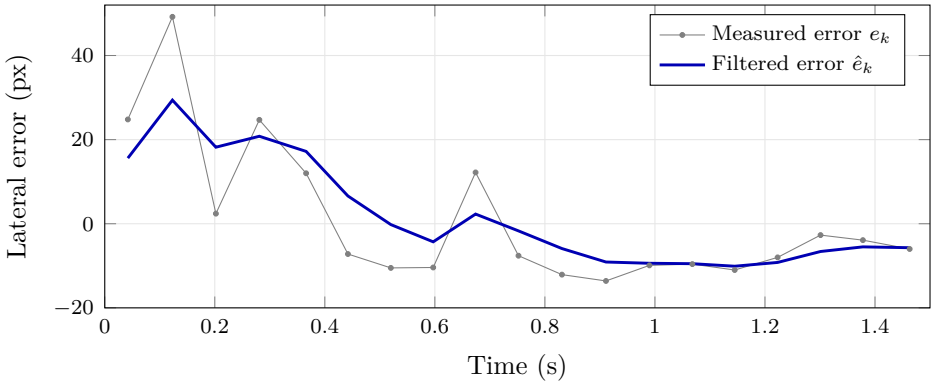


Figure 6.5: Measured lateral error e_k and filtered error $\hat{e}_k$ during 1.5 s of a field run with Strip-IRLS. The filtered error was computed from the recorded measurements with Equation (6.3) and the smoothing factor of Equation (6.5) for the recorded time between frames.

Smoothing independent of the frame rate. If the node runs slower, the same three frames last longer and the filter reacts later. With the debug video enabled, for example, the node processes 17 to 24 frames per second (Section 6.4.1). The smoothing factors are therefore defined as reference values α_{ref} for 30 frames per second. At each frame, the node measures the time Δt since the previous frame, limited to the interval from 1/120 to 0.25 s, and converts each reference factor into

$$\alpha = 1 - (1 - \alpha_{\text{ref}})^{30\,\Delta t}. \tag{6.5}$$

With $\Delta t = 1/30$ s, $\alpha = \alpha_{\text{ref}}$. At 15 frames per second, for example, $\alpha_{\text{ref}} = 0.20$ gives $\alpha = 1 - 0.8^2 = 0.36$, so the filter keeps 64% of the previous value at each frame, as in two frames at 30 frames per second. The filter therefore follows the same response in time, only with fewer updates (Table 6.1). Figure 6.6 shows the two responses.

Table 6.1: Fraction of a step of the input covered by the filter of Equation (6.3) with $\alpha_{\text{ref}} = 0.20$, at 30 and 15 frames per second. A dash marks a time with no frame at 15 frames per second.

Time (s)	0.033	0.067	0.100	0.133	0.167	0.200	0.233	0.467
30 fps, $\alpha = 0.20$	20.0%	36.0%	48.8%	59.0%	67.2%	73.8%	79.0%	95.6%
15 fps, $\alpha = 0.36$	–	36.0%	–	59.0%	–	73.8%	–	95.6%

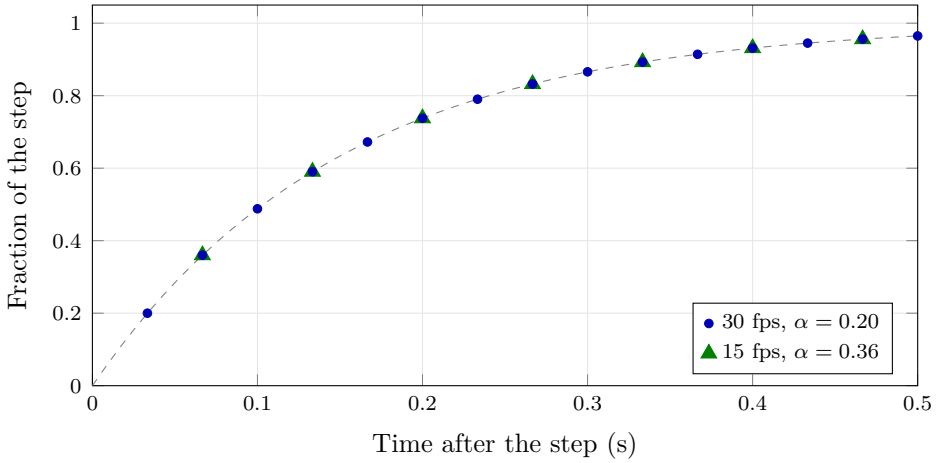


Figure 6.6: Response of the filter of Equation (6.3) to a step of the input, with $\alpha_{\text{ref}} = 0.20$. At 15 frames per second, the factor converted with Equation (6.5) gives the same response in time as at 30 frames per second (dashed line).

Parameters per status and jumps. When the status is `ok` or `degraded`, the filter updates the error with a reference smoothing factor α_{ref} and a jump threshold J that depend on the status:

$$(\alpha_{\text{ref}}, J) = \begin{cases} (0.20,\ 40 \text{ px}) & \text{if the status is } \texttt{ok}, \\ (0.18,\ 38 \text{ px}) & \text{if the status is } \texttt{degraded}. \end{cases} \tag{6.6}$$

In our case the values for the two statuses are close to each other and worked well. The filter keeps them separate, so that the values for `degraded` can be changed if the segmentation is worse. When the status is `lost`, the error is not updated and the robot stops after a short hold, described at the end of this section.

The jump is the distance $\Delta_k = |e_k - \hat{e}_{k-1}|$ between the new measurement and the previous filtered error. The filtered error is

$$\hat{e}_k = \begin{cases} (1-\alpha)\,\hat{e}_{k-1} + \alpha\, e_k & \text{if } \Delta_k \le J, \\ e_k & \text{if } \Delta_k > J,\ \Delta_{k-1} > J \text{ and } |e_k - e_{k-1}| \le J_c, \\ \hat{e}_{k-1} & \text{otherwise.} \end{cases} \tag{6.7}$$

A measurement within the threshold is smoothed with Equation (6.3). A jump on an isolated frame is ignored, so a segmentation error on a single frame does not reach the command. A jump present in two consecutive frames is taken directly only if the two measurements agree, that is, if they differ by at most the tolerance J_c. Otherwise both frames are ignored. Taking the measurement directly lets the filter follow a sudden real change of the centerline position, for example when an

obstacle under a wheel suddenly rotates the robot, without waiting for the slower response of Equation (6.3).

We chose J_c in offline tests, by applying the filter to recorded errors of Strip-IRLS. When two consecutive jumps agreed on the new position of the centerline, their measurements differed by at most 18 px, while in the other cases they differed by more than 50 px. $J_c = 25$ px accepts the first case with some margin and rejects the second.

Steering law. The steering law uses the command error

$$e_{\text{cmd}} = \begin{cases} 0 & \text{if } |\hat{e}_k| < 4 \text{ px}, \\ \hat{e}_k & \text{otherwise.} \end{cases} \tag{6.8}$$

This deadband avoids small oscillations of the steering around the center of the row. It acts only on the command, so the filter keeps its internal value $\hat{e}_k$. The angular velocity target is

$$\omega^* = \text{sat}_{\omega_{\max}}\left(-K_p\,(0.6 + 0.4\,C)\,\frac{e_{\text{cmd}}}{W/2}\right), \tag{6.9}$$

with $K_p = 1.2$, where $\text{sat}_{\omega_{\max}}$ limits the value to $\pm\omega_{\max} = \pm 0.58$ rad/s. Dividing by the half-width $W/2 = 112$ px gives the error as a fraction of the image, from 0 at the center to ± 1 at the edges, so K_p does not depend on the input resolution. The confidence $C \in [0, 1]$ of the path extractor reduces the gain when the result is less reliable. The effective proportional gain is $K_p(0.6 + 0.4\,C)$, which equals K_p with $C = 1$ and $0.75\,K_p$ with $C = 0.38$, the lowest value before `lost`. The minus sign makes the robot turn towards the centerline.

The limit $\omega_{\max}$ is chosen from v so that it matches the steering limit of 30° (Equation (6.2)): a larger ω would not turn the wheels further. The command therefore saturates for errors above

$$e_{\text{sat}} = \frac{\omega_{\max}\,(W/2)}{K_p\,(0.6 + 0.4\,C)}, \tag{6.10}$$

which is about 54 px with $C = 1$ (Figure 6.7a). The gain K_p sets how strongly the robot reacts to the error (Figure 6.7b). A larger K_p gives a larger command for the same error and reaches 30° at a smaller error. A smaller K_p gives a smoother response and reaches the limit later.

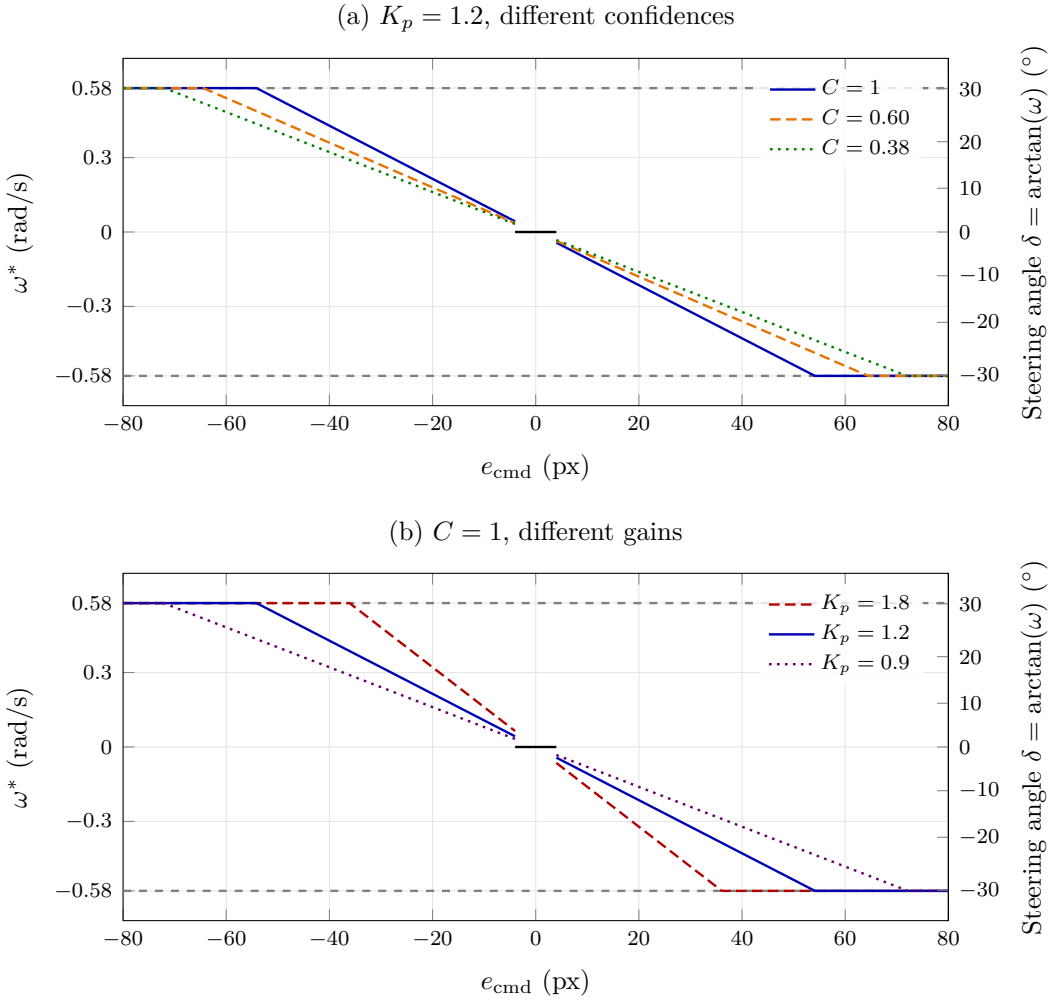


Figure 6.7: Angular velocity target ω^* of Equation (6.9) as a function of the command error e_{cmd}. (a) Effect of the confidence, with the gain used on the robot, $K_p = 1.2$. The curves are for $C = 1$, $C = 0.60$ (threshold between `ok` and `degraded`) and $C = 0.38$ (threshold below which the status is `lost`). (b) Effect of the gain, with $C = 1$. The command saturates at 36 px with $K_p = 1.8$, at 54 px with $K_p = 1.2$ and at 72 px with $K_p = 0.9$. In both plots the command is zero inside the deadband ($|e_{\text{cmd}}| < 4$ px) and is limited to ± 0.58 rad/s. The right axis gives the corresponding steering angle δ at $v = 0.20$ m/s, where Equation (6.2) becomes $\delta = \arctan(\omega)$, so 0.58 rad/s corresponds to 30°.

A second EMA smooths the angular velocity sent to the robot, $\omega_k = (1 - \alpha_w)\,\omega_{k-1} + \alpha_w\,\omega^*$, with $\alpha_w = 0.30$ at 30 frames per second (converted with Equation (6.5), so this filter is also independent of the frame rate). The linear velocity v is constant.

The values of K_p and $\omega_{\max}$ above hold for $v = 0.20$ m/s. The steering angle of Equation (6.2) depends on the ratio ω/v, so at a higher speed the same ω turns the wheels less. If only v is increased, the robot steers less for the same error and can no longer reach the maximum steering angle of 30°. At $v = 0.50$ m/s, for example, $\omega_{\max} = 0.58$ rad/s gives at most 13°.

This can be a desired behavior. The same steering angle gives the same turning radius R at any speed (Equation (6.1)), so the lateral acceleration v^2/R grows with the square of the speed, and steering less at a higher speed keeps it lower.

If instead the same steering angle is wanted for each error, $\omega_{\max}$ and K_p can be multiplied by the same factor as v:

$$\omega_{\max} = \frac{2\,v\tan 30^\circ}{L} \approx 2.89\,v, \qquad K_p = 1.2\,\frac{v}{0.20\ \text{m/s}}. \tag{6.11}$$

In this way $\omega_{\max}$ still corresponds to 30° and each error gives the same steering angle as at 0.20 m/s. At $v = 0.30$ m/s, for example, $\omega_{\max} = 0.87$ rad/s and $K_p = 1.8$.

Missing centerline and stop. When the status is `lost` or no centerline is available, the node keeps the last filtered error for up to $t_{\text{hold}} = 0.5$ s. After this time it publishes a zero velocity command. After a stop, the error filter restarts from the first new measurement and the filter of the angular velocity from zero. Table 6.2 lists the parameters used on the robot.

Table 6.2: Parameters of the error filter and of the steering law used on Demetra.

Parameter	Symbol	Value
Input resolution ($W \times H$)	–	224×224 px
Reference row	y_{ref}	156 px
Reference frame rate of the smoothing factors	–	30 fps
Smoothing factor, status `ok`	α_{ok}	0.20
Smoothing factor, status `degraded`	α_{deg}	0.18
Jump threshold, status `ok`	J_{ok}	40 px
Jump threshold, status `degraded`	J_{deg}	38 px
Frames that confirm a jump	–	2
Coherence tolerance of two jumps	J_c	25 px
Deadband	–	±4 px
Proportional gain	K_p	1.2
Maximum angular velocity	$\omega_{\max}$	0.58 rad/s
Smoothing factor of the angular velocity	α_w	0.30
Linear velocity	v	0.20 m/s
Hold time	t_{hold}	0.5 s

6.3 Complete pipeline

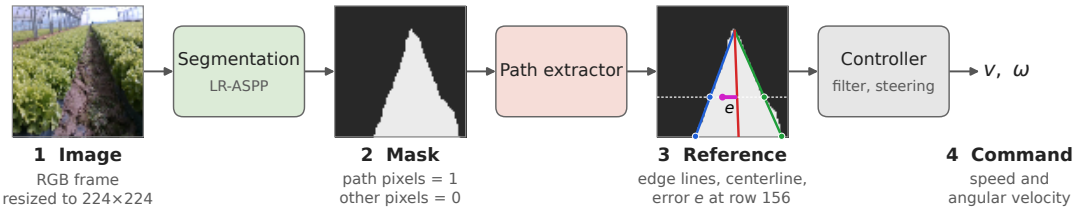


Figure 6.8: Complete pipeline on the robot.

Figure 6.8 summarizes the steps that run on the robot for each camera frame. The controller node resizes the color image to 224×224 pixels, and the segmentation network turns it into the path mask (Chapter 4). The path extractor turns the mask into the centerline (Chapter 5). The controller computes the lateral error e at row $y_{\text{ref}} = 156$, filters it and gives the command (v, ω) (Section 6.2.3). The serial driver converts the command into the speed and the steering angle sent to the microcontroller (Section 6.2.2).

6.4 Deployment and testing

6.4.1 Timing and latency

The inference times in Table 4.1 were measured on the training GPU at the original image resolution, so they do not describe the onboard model. The times in this section were measured on the Jetson AGX Orin in the MAXN power mode, with the clock frequencies fixed at their maximum by `jetson_clocks`, so that the runs are comparable. For each path extractor, the controller node processed the same recording of a row, played at its original speed, once with the debug video disabled and once with it enabled. For each run, 100 callbacks were measured. Table 6.3 lists the mean and standard deviation of the time of the path extractor and of the whole callback.

Table 6.3: Time of the path extractor and of the whole callback on the Jetson AGX Orin (mean ± standard deviation, ms). The inference takes about 5.8 ms, and the other steps of the callback (decoding, resizing, filter, command and log) about 4.7 ms without the debug video.

Extractor	Path extractor	Callback, debug off	Callback, debug on
Strip-IRLS	11.07 ± 0.21	21.62 ± 0.24	53.73 ± 0.69
EdgeNet	4.68 ± 0.18	14.72 ± 0.22	47.64 ± 0.71
RANSAC edges	15.52 ± 0.24	26.24 ± 0.28	57.75 ± 0.77
Center points	7.27 ± 0.27	17.97 ± 0.30	47.40 ± 0.72
Column histogram	0.91 ± 0.04	11.30 ± 0.14	41.45 ± 0.31

Without the debug video, the callback takes from 11.3 ms with Column histogram to 26.2 ms with RANSAC edges, less than the 33.3 ms frame period of the camera. The node therefore processes every camera frame, and at $v = 0.20$ m/s the robot moves about 0.67 cm between two processed frames. The debug video adds 29 to 33 ms, and every callback becomes longer than the frame period. The node then processes 17 to 24 frames per second and skips the others, so the robot moves 0.8 to 1.2 cm between two processed frames. The smoothing factors are corrected for the time between frames (Equation (6.5)), so the response of the filters in time does not change. The difference between the two cases is small, so the field tests of Chapter 7 were run with the debug video enabled, to record the debug images. In normal operation it should still be disabled, so that every frame is used.

6.4.2 Processor-in-the-Loop test

Before the field tests, the complete pipeline was tested in the laboratory with the robot raised, so that the wheels could steer without moving the robot. The purpose of the test was to check that the whole chain, from the image to the steering of the wheels, gives steering commands coherent with the position of the row in the image. A simulation node replaces the camera. It publishes a recorded frame of a row on the camera topic at 30 frames per second, and the controller node runs on these frames without changes. Both the simulation node and the controller node ran on the Jetson AGX Orin of the robot, and the driver sent the commands to the real steering actuators. The simulation node keeps the lateral offset d and the heading ψ of a simulated robot with respect to the row. The offset d is positive when the robot is to the right of the center of the row, and ψ is positive when the robot points to the left. At each step of $\Delta t = 1/30$ s, the node converts the velocity command into the steering angle δ with Equation (6.2), limited to $\pm 30°$,

and updates the pose with the kinematic model of the robot:

$$\psi_k = \psi_{k-1} + \frac{2\,v \tan\delta}{L}\,\Delta t, \qquad d_k = d_{k-1} - v \sin\psi_k\,\Delta t. \tag{6.12}$$

It then shifts the frame horizontally by

$$s = -\frac{f}{D}\,d + f\,\psi \tag{6.13}$$

pixels, positive to the right, where $f = 465$ px is the nominal focal length of the camera and $D = 1.25$ m is the assumed distance of the image row where the error is measured. A lateral offset therefore moves the row in the image by $f/D \approx 372$ px per meter, and a rotation by 465 px per radian. During the test, a lateral offset was added from the keyboard. The simulated pose was computed from the velocity command, assuming that the wheels follow the commanded steering angle, and the wheels visibly reached the commanded angle. Figure 6.9 shows three frames of the test.

The test checked the integration of the perception, control and actuation chain. The wheels turned forward, and after each displacement they steered towards the center of the row while the simulated offset returned to the center without oscillating. However, the test is not a realistic simulation of row navigation and does not validate physical path tracking, because the scene is a single shifted frame, the projection is approximate and the robot does not move on the ground. The network segments every frame, but always the same image shifted laterally. The segmentation and the path extraction along a real row were instead tested by playing recordings of the rows to the controller node, as in Section 6.4.1. A visual check of the debug video showed that most of the extracted centerlines were correct.

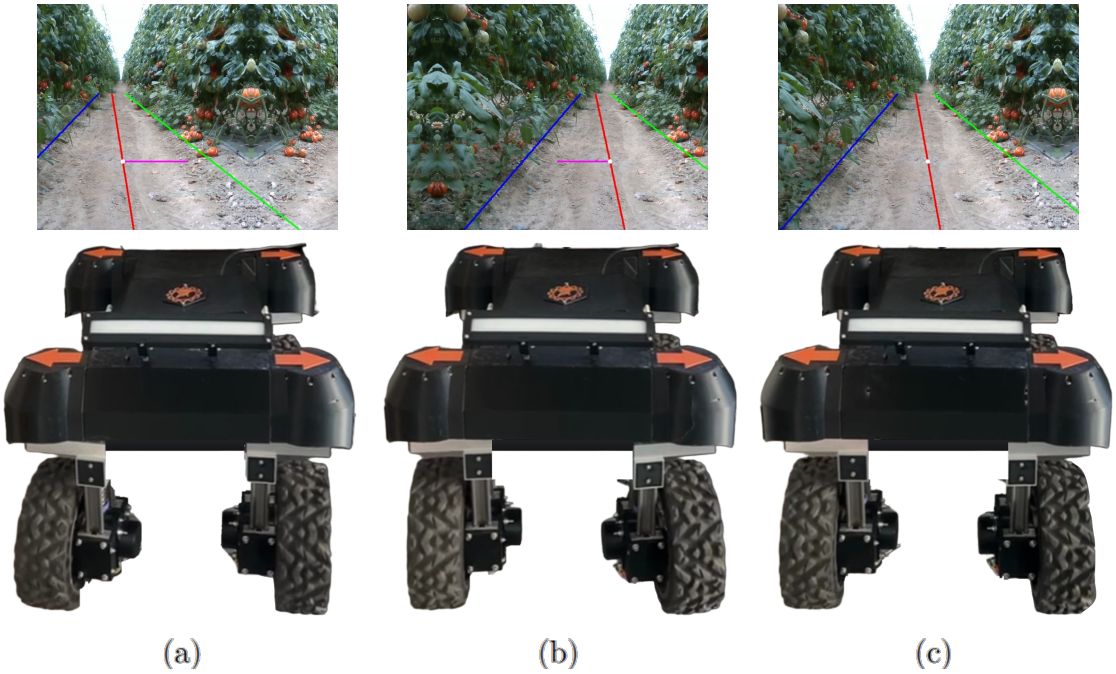


Figure 6.9: Processor-in-the-Loop test. Top: frame of the simulation node shifted with Equation (6.13), with the edges (blue and green), the centerline (red) and the lateral error at y_{ref} (magenta) extracted offline by the network and Strip-IRLS. Bottom: the raised robot during the test, at similar lateral errors. (a) and (b) Lateral error to the left and to the right: the wheels steer toward the center of the row. (c) Error below 1 px: the simulated robot is back at the center and the wheels are straight. The mirrored vegetation comes from the border that the simulation node adds before the shift.

Chapter 7

Experimental Validation

This chapter reports the experimental evaluation of the segmentation models, geometric extractors, and integrated navigation system. It covers the offline evaluation on recorded images and the closed-loop tests of the robot in a row of tomato plants.

7.1 Offline evaluation

7.1.1 Segmentation metrics

Segmentation performance is measured using intersection over union (IoU) for each semantic class. For a class c, IoU is defined as:

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{7.1}$$

where true positives, false positives, and false negatives are computed against the manual reference masks.

Mean IoU (mIoU) averages the per-class IoU values. For the four-class configuration, mIoU includes background, plants, path, and covering material. Path IoU is the task-specific metric most relevant for navigation.

Architecture comparison results

The architecture comparison in Section 4.3.3 reports, on the 77 test images of the manual lettuce dataset, a path IoU of 0.9532 for LR-ASPP, against 0.9654 for FCN and 0.9686 for DeepLabV3. The recorded GPU inference times are 8.21 ms, 59.10 ms, and 86.90 ms. LR-ASPP accepts an mIoU about 1.1 percentage points lower in exchange for 7.2 to 10.6 times shorter recorded times.

Supervision strategy results

The supervision comparison is reported in Section 4.4.2. On the test split, the manual model reaches a path IoU of 0.9532 on the complete test set and 0.9587 in the realistic scenario. The three networks trained only on SAM 3 lettuce labels reach path IoU values between 0.8966 and 0.9013 in the realistic scenario and between 0.3917 and 0.4124 on the complete test set. The 4-class network reaches an mIoU of 0.5925 in the realistic scenario. A network with the deployed multi-crop configuration, trained on the complete SAM 3 collection without the lettuce test images, reaches 0.9071 in the realistic scenario and 0.6369 on the complete test set. The deployed weights themselves have no independent IoU (Section 4.4.3).

7.1.2 Geometric reference accuracy

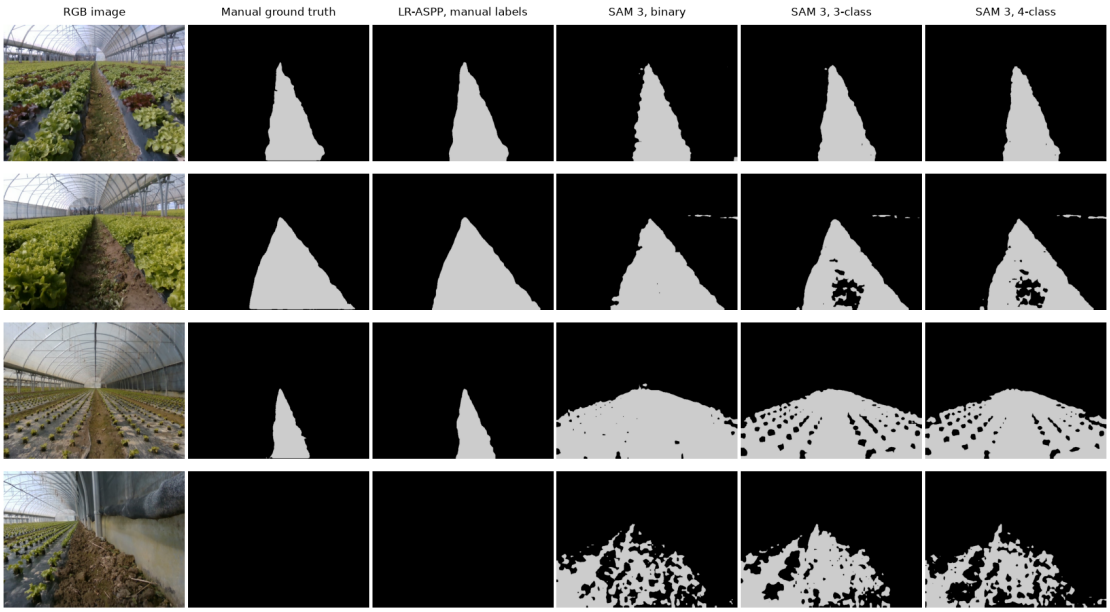
The two extractors developed in this work, Strip-IRLS and EdgeNet, and the three baselines of Chapter 5, which are the five extractors tested on the robot, are compared offline on the same masks with the parameters of that chapter. Table 7.1 lists the two data sets.

Table 7.1: Data sets of the offline comparison of the extractors.

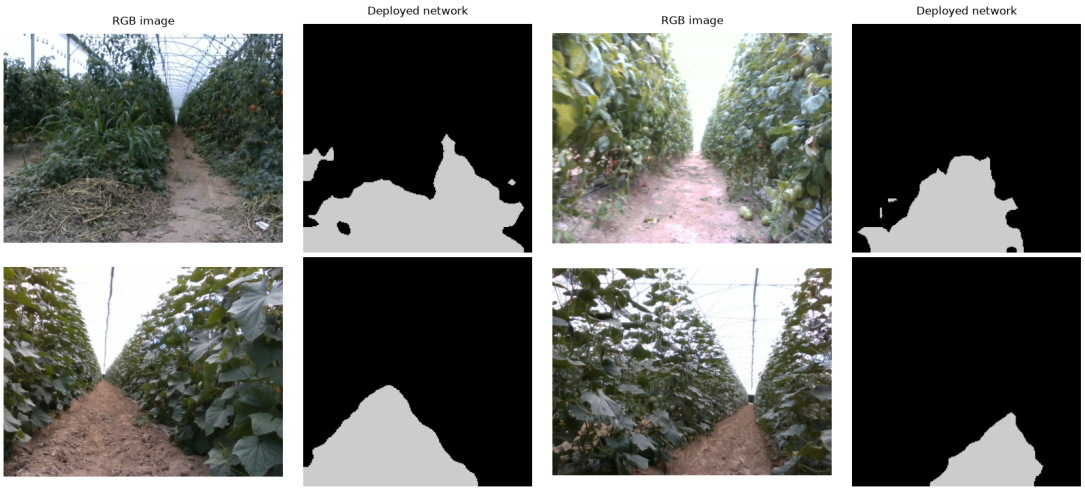
Set	Masks	Ground truth	Use
Synthetic	2500 (25 scenarios)	Exact	Failure cases: straight corridors with one disturbance, adjacent corridors, curve
Annotated real frames	255 of 308	Edges marked by hand	Accuracy on real masks

The synthetic masks have 224×224 pixels. Each one is a straight corridor in perspective, defined by its bottom center, vanishing point, bottom half-width and apex row, with irregular edges (a slow undulation of a few pixels and a fine grain of 1–2 px). The set has 21 base scenarios with one disturbance each, two of them without a corridor (an empty mask and a mask that is all path), and 4 scenarios with adjacent corridors or a curved corridor, layouts that were not present in the test field. Table 7.2 lists the scenarios. The masks generated with seed 0 were used during the development of the extractors, and the results reported here are computed on a separate set generated with seed 2027.

The annotated frames are taken from 14 sequences: `pomodori2` and 13 sequences of the SAM 3 dataset (Chapter 4), 7 of tomato, 2 of cucumber and 4 of lettuce. `pomodori2` is a recording of 9465 compressed images of the topic `/camera/camera/color/image_raw/compressed`, taken in a greenhouse with tomato plants. Its masks are 100 masks predicted on frames evenly spaced in the



(a) Lettuce test images



(b) Tomato (top) and cucumber (bottom)

Figure 7.1: Visual comparison of segmentation outputs. (a) Four of ten test images of the manual lettuce dataset, taken at evenly spaced positions of the test split: RGB image, manual ground truth, and the predictions of the LR-ASPP network trained on the manual labels and of the three LR-ASPP networks trained only on SAM 3 lettuce labels. In the third image the SAM 3 networks mark the whole floor as path, including the strips beside the central corridor, and in the fourth, where the manual ground truth has no path, they mark the soil in front of the robot. (b) Tomato and cucumber images, which have no ground truth: RGB image and prediction of the binary network installed on the robot, as a qualitative comparison only.

recording and segmented as in the controller node (Chapter 6). The masks of the SAM 3 sequences were predicted by the network deployed on the robot and reduced from 640×480 to 224×224 pixels with nearest-neighbor interpolation. These images belong to the training data of the network, so their masks are probably cleaner than those of images not seen in training. 348 masks without path pixels and four lettuce sequences in which the network predicted almost no path (7 further masks) were removed before the frames were selected. The annotation is described in Section 7.1.2.

The error is $|\hat{x} - x_{\text{true}}|$ at the row used by the controller, $y_{\text{ref}} = 156$ (Chapter 6), and is computed only on the frames not classified as `lost`. A gross error is an error above 15 px (13% of the half-width of the image). For the masks without a corridor the expected status is `lost`. A failure is a frame that is `lost` or has a gross error. All five extractors are deterministic: RANSAC edges draws its samples from a generator with a fixed seed, and the others do not sample. Their times on the Jetson AGX Orin are in Section 6.4.1.

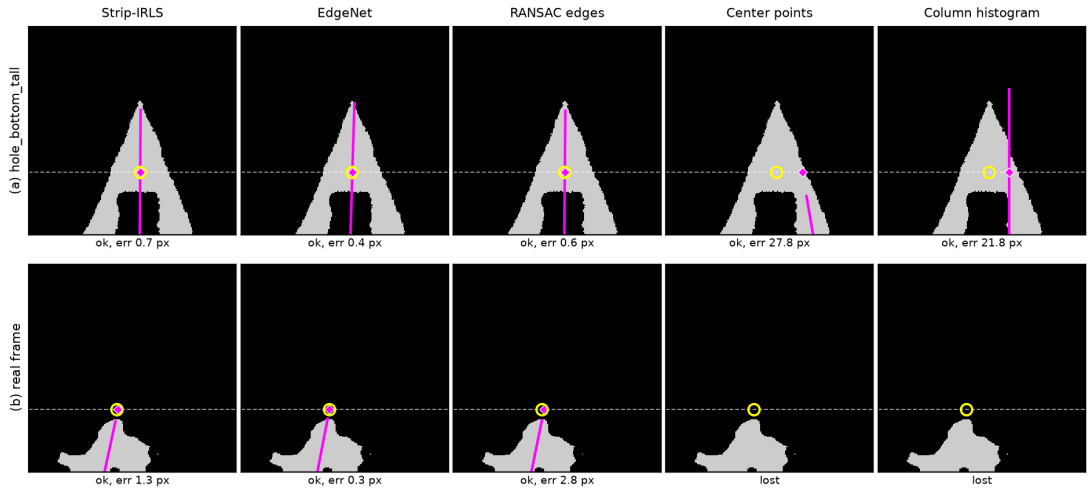


Figure 7.2: Lines estimated by the five extractors on two masks with disturbances, chosen among the masks where Strip-IRLS has an error below 4 px and at least two other extractors fail. (a) is a mask of the synthetic scenario `hole_bottom_tall`, and (b) is the real frame `pomodori2_01051`, whose true centerline comes from the hand annotation. The magenta line is the estimated centerline, the diamond its position at y_{ref} (dashed line), and the yellow circle the true centerline at y_{ref}. Below each panel are the status and the error at y_{ref}. A panel without a line is `lost`.

Table 7.2: Scenarios of the synthetic set (100 masks each). Unless stated otherwise, the corridors have a bottom half-width of 45–70 px and the apex at row 40–90.

Scenario	Content
Base set (21 scenarios)	
`clean`	Clean corridor, robot almost centered
`offset`	Robot offset laterally by 20–55 px at the bottom
`heading`	Robot rotated, vanishing point moved by 30–90 px
`offset+heading`	Offset and rotated
`narrow`, `wide`	Bottom half-width 18–28 px or 85–105 px
`blobs`	5–14 false-path spots in the ROI
`holes`	4–11 holes inside the corridor
`hole_bottom`	Central hole at the bottom, 10–35 rows high
`hole_bottom_tall`	Central hole at the bottom, 45–60 rows high
`bulge_one_side`	Widening on one side, 20–60 px over 15–40 rows
`bite_one_side`	Intrusion on one side, 10–30 px over 15–40 rows
`jagged`	Jagged edges, 3–10 px per row
`shadow_gap`	Empty horizontal band 8–28 rows high
`bottom_full`	Last 10–40 rows all path
`row_end`	End of row, no path above row 112–145
`edge_out`	Robot far from the center, one edge leaves the image
`speckle`	Salt-and-pepper noise, 2–8% of the pixels inverted
`combined`	Spots and holes, robot offset and rotated
`empty`, `full`	Empty mask, mask all path (expected `lost`)
Adjacent corridors and curve (4 scenarios)	
`adjacent`	Two copies of the corridor on the left and right, with the same vanishing point, separated at the bottom by a row 40–110 px wide
`adjacent+offset`	As `adjacent`, robot offset by 25–50 px at the bottom
`adjacent_narrow_crop`	As `adjacent`, row 15–35 px wide
`curved`	Centerline bent by a quadratic term, 9–27 px at y_{ref}

Results on the synthetic masks

Table 7.3 reports the results on the synthetic test masks, and Table 7.4 the gross error rate in the scenarios where at least one extractor exceeds 5%.

Table 7.3: Results of the five extractors on the synthetic test masks (seed 2027). Gross errors are errors above 15 px at $y_{\text{ref}} = 156$ on the non-`lost` frames.

	This work		Baselines		
	Strip-IRLS	EdgeNet	RANSAC edges	Center points	Column histogram
Base scenarios, 1900 masks with a corridor					
Gross errors (%)	0.3	0.1	0.1	4.3	23.2
`lost` (%)	0.4	0.0	0.0	3.4	1.1
Gross errors classified `ok`	4/6	1/2	1/1	25/78	406/436
`empty` and `full` rejected (%)	100	100	100	50	50
Adjacent corridors and curve, 400 masks					
Gross errors (%)	2.3	0.0	18.5	0.5	42.6
`lost` (%)	1.0	6.2	0.0	0.2	0.2
Gross errors classified `ok`	5/9	–	74/74	1/2	170/170

Table 7.4: Gross error rate (%) on the synthetic test masks (seed 2027) in the scenarios where at least one extractor exceeds 5%.

	This work		Baselines		
Scenario	Strip-IRLS	EdgeNet	RANSAC edges	Center points	Column histogram
`adjacent`	3.0	0.0	13.0	0.0	0.0
`adjacent+offset`	4.2	0.0	61.0	2.0	71.0
`curved`	0.0	0.0	0.0	0.0	99.0
`offset`	0.0	0.0	0.0	0.0	64.0
`heading`	0.0	0.0	0.0	0.0	92.0
`offset+heading`	0.0	0.0	0.0	0.0	59.0
`edge_out`	0.0	0.0	0.0	0.0	74.0
`row_end`	0.0	0.0	0.0	0.0	21.6
`combined`	0.0	0.0	1.0	0.0	44.3
`holes`	0.0	0.0	0.0	0.0	5.2
`hole_bottom`	0.0	0.0	0.0	0.0	9.2
`hole_bottom_tall`	2.0	0.0	0.0	71.9	70.4
`bulge_one_side`	3.1	2.0	0.0	11.5	0.0

Each extractor fails in different scenarios:

- **Strip-IRLS.** In `hole_bottom_tall` a central hole 33 to 50 px wide splits the bottom of the corridor. Strip-IRLS merges columns separated by at most 30 px, so it does not join the two parts where the hole is wider. A hole wider

than the gap and taller than the seed window (six strips of 10 rows) puts every seed in one of the two parts. The chain that starts from it stops where the parts join, because there the interval widens beyond the jump limit, and its edges are the two edges of that part. This causes gross errors in 2.0% of the masks of this scenario. In `adjacent+offset` the merging gap joins the narrow upper part of the row between two corridors with the corridors beside it, and 4.2% of the masks have a gross error.

- **EdgeNet.** It has two gross errors in the base scenarios, both in `bulge_one_side`. With adjacent corridors it has no gross errors, but it classifies only 31.5% of these masks as `ok`, because the mask also contains the adjacent corridors and the intersection over union in its confidence is low. A first model, trained with an earlier generator without adjacent corridors and with the apex only between rows 30 and 100, had 95.7% gross errors in `adjacent+offset`, so the accuracy of the network depends on the coverage of its generator. In separate tests on synthetic masks with the shapes of real tomato masks, which are not reported here, most of its gross errors were in masks with the apex below y_{ref} and the true centerline near the image border. There the value at y_{ref} is an extension of the two lines, and a small error on one edge becomes a large error. This is a possible cause of its failures on the annotated test frames, but it was not checked on those frames.

- **RANSAC edges.** It takes the first and last path pixel of each row of the selected component, so when an adjacent corridor belongs to that component its edge replaces the edge of the central corridor in those rows.

- **Center points.** It forms the midpoints of the intervals before the fit, so a disturbance on one side or in the middle of the corridor moves the midpoints and enters the centerline (`bulge_one_side`, `hole_bottom_tall`). It returns a line on the mask that is all path. With adjacent corridors it follows one interval per strip and has almost no gross errors.

- **Column histogram.** It returns a vertical line, so its error grows with the offset and the rotation of the robot and on the curved corridor.

On the curved corridor, the straight-line extractors keep the error below 15 px at y_{ref}, but their lines do not follow the curve. Most gross errors of Strip-IRLS, RANSAC edges and EdgeNet are classified as `ok`.

Results on the annotated real frames

The annotated frames give a ground truth on real data. From each of the 14 sequences, 22 frames evenly spaced in the sequence were selected (308 frames). In a

browser tool written for this purpose, the author marked two points on each corridor edge in the 640 × 480 photo, with the predicted mask shown as a semi-transparent overlay, along the line where the plants meet the ground. Each edge is the line through its two points, scaled to the 224 × 224 mask as in the controller node, and the true centerline is the mean of the two edge lines, as in Eq. (5.2). The error is measured at y_{ref} as for the synthetic masks.

The sequences were split before the evaluation into a development part (`cetrioli_grandi`, `insalata_grande1`, `insalata_media1`, `pomodori_maturi1`, `pomodori_maturi3`, `pomodori_maturi5` and `pomodori2`) and a test part (the other seven), each with frames of the three crops. `pomodori2` is in the development part because the thresholds of RANSAC edges were chosen on its frames. The development frames were used to choose the version of Strip-IRLS. The test frames were evaluated once for the other extractors, after they had been fixed, and four times for Strip-IRLS: with $\rho = 0.50$ and gap 30 px (4.1%, 5 of 122 gross errors), with $\rho = 0.30$ and gap 30 px (2.5%, 3 of 122), with $\rho = 0.30$, gap 40 px and the strips of 9 rows of an earlier version (3.3%, 4 of 122), and with the final version, reported below. The final version tiles the ROI exactly with strips of 10 rows (Section 5.2.2) and uses ten reweighting iterations. Both changes were checked on the development data, where the gross errors on the annotated frames stayed at 3 frames, and the test frames were evaluated after them. After this evaluation the merging gap was changed from 40 to 30 px, the value used on the robot, and no result on the annotated frames changed. The parameters were chosen on the development data. The annotator skipped 12 development frames and 22 test frames, mostly of tomato, where the edges could not be read. The tomato sequences of the SAM 3 dataset were recorded with a handheld camera, and in many of their frames the image is shaken or the segmentation is poor. In 10 development frames and 9 test frames one annotated edge is inclined by more than 60° from the vertical. There the mean of the two lines in the form of Eq. (5.1) is not a usable centerline, and its value at y_{ref} can lie hundreds of pixels outside the image, so these frames are excluded from the error statistics. The threshold of 60° was chosen on the development frames. 132 development frames and 123 test frames remain. None of them was annotated as a frame without a corridor.

Strip-IRLS and RANSAC edges have the fewest failures on both parts. Their difference is a few frames and their order changes between the two parts, so these frames do not separate them. Center points and Column histogram fail on 24 to 28 frames per part. The network has few failures on the development part and 10 on the test part. The three gross errors of Strip-IRLS on the test frames are all in the tomato sequence `pomodori_maturi6`, as is the one of RANSAC edges. As on the synthetic masks, most gross errors of Strip-IRLS, RANSAC edges and EdgeNet are classified as `ok`, so their status does not warn the controller.

These results have four limits. The 22 frames of a sequence are not independent.

Table 7.5: Results on the annotated real frames. Gross errors are errors above 15 px at y_{ref} on the non-`lost` frames, given as a percentage and as a count. Failures are the frames that are `lost` or have a gross error.

	This work		Baselines		
	Strip-IRLS	EdgeNet	RANSAC edges	Center points	Column histogram
Development, 132 frames					
Gross errors (%)	2.3 (3/128)	3.0 (4/132)	2.3 (3/129)	5.5 (6/110)	12.9 (16/124)
`lost` (%)	3.0	0.0	2.3	16.7	6.1
Failures	7	4	6	28	24
Gross errors classified `ok`	3	3	3	1	15
Median error (px)	1.69	1.69	1.49	2.13	5.08
Test, 123 frames					
Gross errors (%)	2.5 (3/122)	6.6 (8/121)	0.8 (1/121)	7.6 (8/105)	18.8 (22/117)
`lost` (%)	0.8	1.6	1.6	14.6	4.9
Failures	4	10	3	26	28
Gross errors classified `ok`	3	5	1	0	22
Median error (px)	2.32	2.00	1.29	2.45	5.65

On the test frames the gross errors of Strip-IRLS and RANSAC edges are all in one tomato sequence, and Strip-IRLS, RANSAC edges and EdgeNet have no gross errors on lettuce, so the comparison rests on about 100 frames. The annotation was made once by one person, with the predicted mask shown in overlay, which can draw the annotation toward the mask, and its error was not measured. The annotated centerline is therefore not exact like the synthetic one, and differences of a few pixels in the median error, which are 1.3 to 2.3 px for Strip-IRLS, RANSAC edges and EdgeNet, are not interpreted: the comparison relies on the gross errors. The ground truth is marked on the photo, so the measured error also contains the segmentation error, which is the same for all extractors. The excluded frames, those skipped and those with a nearly horizontal edge, are among the most difficult, so the reported rates are lower than on all frames.

Choice of the parameters of Strip-IRLS

The ROI start and the merging gap of Strip-IRLS were chosen on the development data, the seed 0 masks and the development part of the annotated frames. Moving the requested ROI start from $\rho = 0.50$ to $\rho = 0.30$ at a gap of 30 px lowered the gross errors on the base scenarios from 6.0% to 0.5% and on `hole_bottom_tall` from 100% to 1.0%. On the development part of the annotated frames it moved them from 1.6% (2 of 126) to 2.3% (3 of 128). Table 7.6 shows the effect of the merging gap with $\rho = 0.30$: a larger gap joins more holes inside the corridor but also more rows between adjacent corridors, and on the annotated frames it makes no difference from 20 to 40 px. A gap of 40 px was chosen with the strip geometry of the earlier version. With the final geometry the table shows that 30 px has fewer errors on `adjacent+offset` (3.1% against 15.0%) and 1.0% instead of 0.0% on `hole_bottom_tall`, so the final version uses 30 px, the value used on the robot. Variants with an MSAC fit of the edges [22] and with parameters chosen on the synthetic errors had fewer synthetic errors, but they were not better on the development part of the annotated frames, and they were not kept.

Table 7.6: Effect of the merging gap of Strip-IRLS ($\rho = 0.30$) on the development data: gross error rate (%) on the seed 0 masks and on the development part of the annotated frames.

Gap (px)	Base scenarios	Adjacent and curve	`hole_bottom_tall`	`adjacent+offset`	Annotated frames
20	0.6	2.5	4.0	3.0	2.3
30	0.5	1.3	1.0	3.1	2.3
40	0.7	5.1	0.0	15.0	2.3
50	0.8	10.8	0.0	40.9	3.1
60	1.1	18.8	0.0	74.2	3.1

7.2 Field validation

The five extractors were tested in closed loop on the robot on 4 November 2026. The robot ran the complete chain of Chapter 6, from the camera to the steering commands, with the segmentation network deployed on the robot and with the debug video enabled. Its trajectory was measured with a total station.

7.2.1 Test conditions

The tests were made in a row of tomato plants. It was a difficult case for the segmentation: the plants were sparse, so the predicted path often widened into the gaps between them, and the lighting was poor. No image of this field is in the training data of the segmentation network. Figure 7.5a shows a frame of this row.

7.2.2 Trajectory measurement

The position of the robot was measured with a total station (model to be specified) that follows a prism automatically. The prism was placed by hand at the lateral center of the robot. The station records about four positions per second, as coordinates (X, Y) in a horizontal frame with the origin at the station, in which Y runs approximately along the row.

The reference for the center of the row is a set of five points measured with the same station, with the prism placed by hand on the center of the row, at $Y = 1.55$, 18.09, 27.09, 32.38 and 39.88 m. The centerline is the polyline through these points, with the first and the last segment extended. The lateral distance of the robot is its perpendicular distance from this polyline, positive when the robot is to the left of the centerline with respect to its direction of travel.

This reference has two limits. The points were placed by eye, so their position across the row is not exact, and their error was not measured. The row is not straight: the three intermediate points lie 4.0 to 6.3 cm from the line through the first and the last point, and between two points the polyline does not follow the row. The short runs described below lie entirely between the first two points, 16.5 m apart, where the reference is a single straight segment. The measured lateral distance therefore contains the error of the reference as well as the error of the robot. If the two errors are independent, their squares add on average, so the root mean square of the measured distance is larger than that of the real distance of the robot from the center of the row. At a single position the two errors can also partly cancel.

7.2.3 Protocol

Each extractor made three separate short runs toward the station. Fourteen of them start at about $Y = 13.9$ m. In RANSAC edges Run 3 the robot started at $Y = 12.97$ m, moved forward to $Y = 12.59$ m and then backward for 0.85 m, and only the part after this movement is used. The robot was placed at the start by hand. At the first position tracked by the station its lateral distance from the centerline is between -5.6 and $+7.3$ cm in the runs of Strip-IRLS, EdgeNet, RANSAC edges and Center points, and -9.7, -5.8 and -3.7 cm in the three runs of Column histogram. At $Y = 12.0$ m the twelve runs of the first four extractors lie within 1.8 cm of each other. The short runs are compared on the same stretch, from $Y = 12.0$ to $Y = 4.0$ m (8.0 m), so that the start position does not enter the comparison. After the short runs, each extractor made one run along the full row, away from the station. The full-row runs are compared on the stretch from $Y = 14$ to about $Y = 39.7$ m, where the station tracked the prism in all five runs. In the first attempt along the full row, EdgeNet and Column histogram stopped against the plants. Both runs were restarted and only the restarts are reported. The station picked up the prism again at $Y = 9.96$ m and $Y = 13.94$ m. Other attempts that did not go well were also repeated, and only the repeated runs are reported, so these results do not give a failure rate. The station also recorded points 500–979 and 8500–8769, which are not used: specify which extractor and why.

7.2.4 Results

Table 7.7 summarizes the lateral distance of all runs. Figure 7.3 shows it along the short runs and Figure 7.4 along the full row.

Table 7.7: Lateral distance from the row centerline in the field tests (cm), positive to the left of the robot, on $Y = 4.0$–12.0 m for the short runs and on $Y = 14$–39.7 m for the full row. For the short runs, the mean and the standard deviation are given as the range over the three runs, and the maximum absolute value is the largest of the three runs. The difference is the root mean square difference from the mean of the runs of the other four extractors at the same position.

	Short runs (3 per extractor)				Full row (1 per extractor)			
Extractor	Mean	Std	Max $\lvert\cdot\rvert$	Difference	Mean	Std	Max $\lvert\cdot\rvert$	Difference
Strip-IRLS	+3.5 to +4.1	3.9–4.3	11.0	1.6	−6.7	6.1	20.4	2.1
EdgeNet	+3.8 to +4.0	3.5–3.6	10.5	1.1	−6.2	5.9	16.3	1.9
RANSAC edges	+2.9 to +3.5	3.9–4.7	10.1	1.9	−7.2	6.1	18.4	1.5
Center points	+3.8 to +4.2	3.9–4.3	10.8	1.6	−7.1	6.1	17.5	1.8
Column histogram	−0.8 to +5.5	2.4–5.1	13.6	6.8	−8.0	4.5	17.4	4.1

Strip-IRLS, EdgeNet, RANSAC edges and Center points give almost the same trajectory. On the short runs their mean lateral distance is between +2.9 and +4.2 cm, and the largest absolute value of each extractor is between 10.1 and 11.0 cm. On the full row the mean is between −6.2 and −7.2 cm and the largest absolute value between 16.3 and 20.4 cm. The root mean square difference of each of them from the mean of the other extractors is 1.1 to 2.1 cm. The mean of the runs of these four extractors at the same position varies from −1.5 to +10.4 cm along the short stretch and from −17.9 to +6.5 cm along the full row. The differences between these four extractors are therefore much smaller than the part of the lateral distance they have in common, and these runs do not rank them.

On the stretch from $Y = 4.0$ to $Y = 12.0$ m, covered by the short runs and also by the full-row runs of these four extractors (by EdgeNet from $Y = 9.96$ m and by RANSAC edges from $Y = 4.28$ m), the robot is on average on the same side of the reference line, toward positive X, although the short runs and the full-row runs go in opposite directions: by 2.9 to 4.2 cm in the short runs and by 5.4 to 9.6 cm in the full-row runs. An offset of the prism or of the camera on the robot would change side with the direction of travel. If the offset is split into a part fixed to the field and a part fixed to the robot, the two directions give 5.2 to 5.6 cm fixed to the field and 1.4 to 1.8 cm fixed to the robot, to its right. The range depends on whether the full-row run of EdgeNet, which covers only the part from $Y = 9.96$ to $Y = 12.0$ m, is included. The offset is therefore mostly fixed to the field, in the plants seen by the camera or in the reference.

Column histogram differs from the other extractors. The root mean square difference from their mean is 6.8 cm on the short runs and 4.1 cm on the full row. Its three short runs differ from each other, with means from −0.8 to +5.5 cm, and their correlation with the mean of the other extractors is between −0.88 and 0.40, against 0.94 to 0.98 for the runs of the other four extractors. Run 1 starts about 4 and 6 cm further to the right than the other two runs, and it comes within 2 cm of them only at about $Y = 6.3$ m, after more than 7 m.

The data recorded by the controller show the cause. Between $Y = 12.0$ and $Y = 5.0$ m, Column histogram classified every frame of its three runs as `ok`, also when its centerline was at the image border. Where the predicted path covers most of the width of the mask, the column with the most path pixels can lie at the border, and the measured lateral error reaches about ±110 px, almost the half-width of the mask (112 px). Figure 7.5b and c shows two of these frames. The filtered error is still 28 and 32 px in these two frames. In Run 2 the measured error is on average about +20 px between $Y = 5.25$ and $Y = 4.75$ m, and the robot turns to the right. This gives the largest distance of Column histogram in Table 7.7, 13.6 cm, at the end of Run 2.

Near the end of the short runs the camera sees the end of the row, the station and the operator (Figure 7.5d). The robot is still in the row there, about 2.5 m

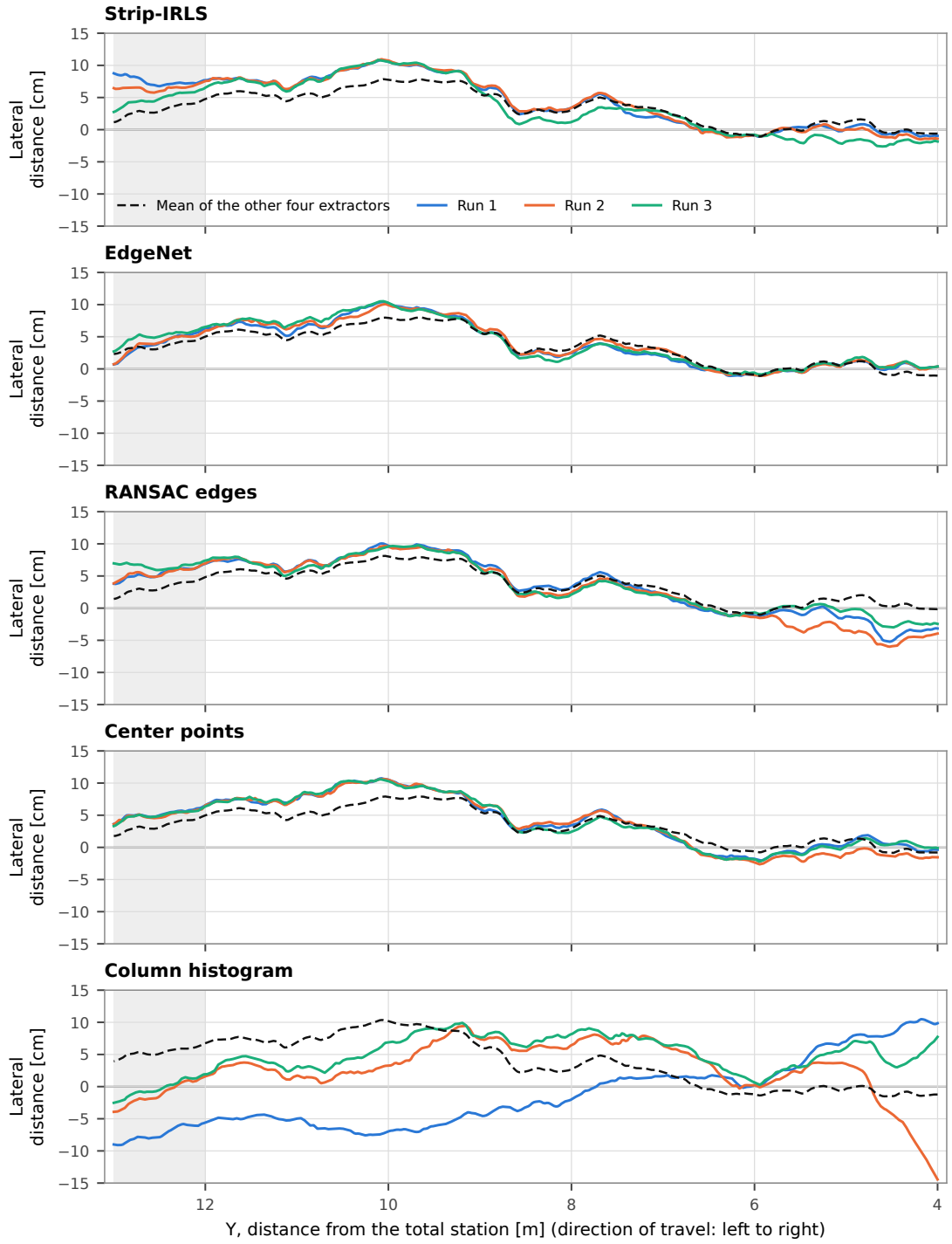


Figure 7.3: Lateral distance from the row centerline in the three short runs of each extractor, positive to the left of the robot, from $Y = 13.0$ to $Y = 4.0$ m. The gray band is not used in Table 7.7: there the runs still depend on the start position. The robot moves toward the station, from left to right in the plot. The black dashed line is the mean of the runs of the other four extractors. The vertical axis is centered on zero, with the same scale in all panels. In this stretch the reference is the straight segment between the points measured by hand at $Y = 1.55$ m and $Y = 18.09$ m.

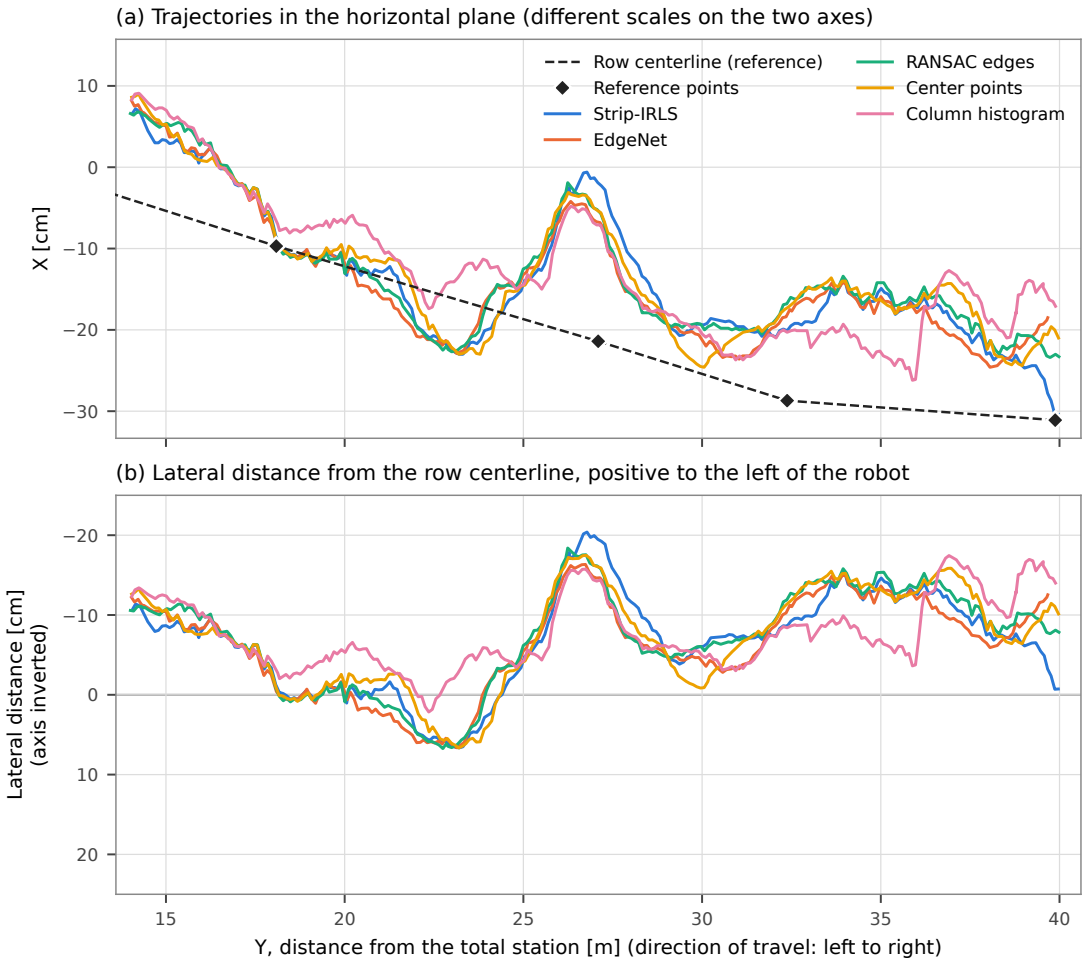


Figure 7.4: Runs along the full row, one per extractor. (a) Trajectories measured by the total station and centerline through the reference points measured by hand. The robot moves away from the station, so its left is toward negative X. (b) Lateral distance from the centerline, positive to the left of the robot. The vertical axis is centered on zero and inverted, so that the curves have the same orientation as in (a).

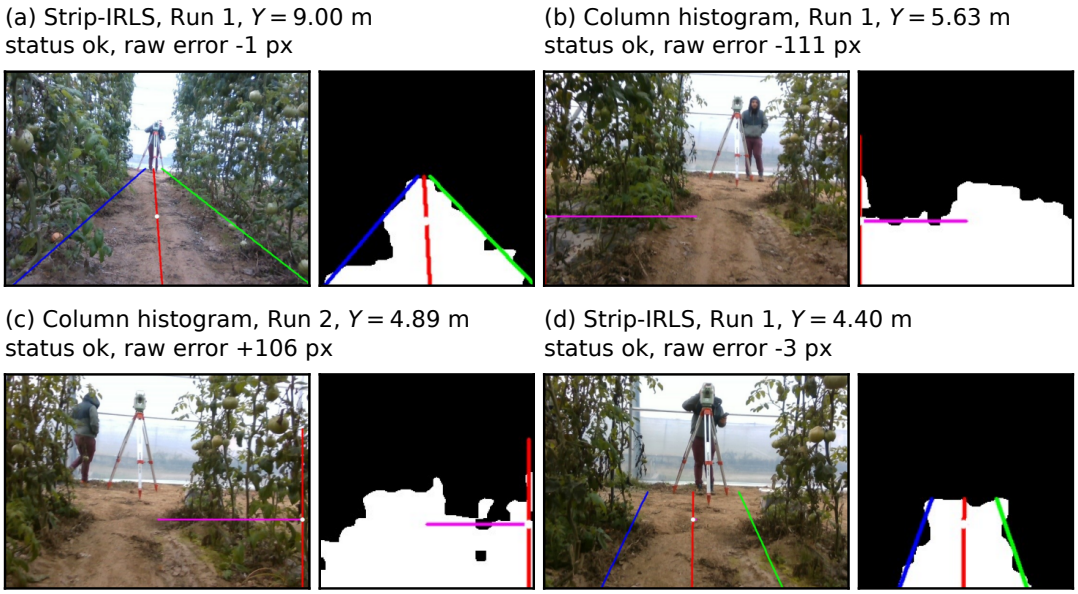


Figure 7.5: Frames recorded by the controller during the short runs. In each example the left image is the camera image with the edges (blue and green) and the centerline (red) estimated by the extractor, and the right image is the region of interest, the cleaned mask on which the extractor operates (Chapter 6), with the same lines. Column histogram estimates only a vertical centerline. The magenta segment is the lateral error. The status and the measured lateral error e_k (positive to the right) are those recorded by the controller for the same frame, and Y is interpolated from the positions measured by the total station. The text that the controller writes on these images was removed. (a) Sparse plants in the middle of the short runs. (b), (c) Column histogram with the centerline at the image border and status `ok`. (d) Strip-IRLS near the end of the stretch, with the station and the end of the row in view.

from the first reference point at $Y = 1.55$ m. Between $Y = 5.0$ and $Y = 4.0$ m Strip-IRLS classified 232 of 241 frames as `ok`, and the mean absolute measured error is 1.5 to 4.8 px in each half meter of its three runs. In the same stretch the confidence of EdgeNet falls. Between $Y = 4.5$ and $Y = 4.0$ m, 46 of the 84 frames recorded by the controller are `degraded`, in the two runs of EdgeNet that were recorded. In the masks computed offline from the recorded images, the predicted path ends at about row 133, below the station and the operator, while the edge lines of EdgeNet continue upward. The IoU of Eq. (5.9) falls to about 0.6, while p_{valid} stays at 1 (Figure 5.6f). A lower confidence is expected there, since the station and the operator stand on the path.

7.3 Discussion

7.3.1 Segmentation performance

The architecture comparison supports LR-ASPP as an accuracy–speed compromise: on the manual lettuce test images it loses about 1.1 percentage points of mIoU with respect to FCN and DeepLabV3, with much shorter recorded inference times. IoU alone does not establish that a segmentation is sufficient for navigation. This is assessed through the geometric extraction and the robot tests.

The supervision comparison tested labels generated by SAM 3 and used without any manual correction. On lettuce, networks trained on these labels remain below the manual model (path IoU 0.9013 against 0.9587 for the binary network in the realistic scenario) and lose most of their accuracy on the complete test set. Part of this gap may come from a difference in label definition: the manual masks mark only the central traversable corridor as path, whereas networks trained on SAM 3 labels can also segment adjacent or non-traversable corridors, which the IoU counts as false positives. This behavior was observed visually for the deployed network but was not quantified. The deployed network required no manual annotation of its training data, but the time saved with respect to manual labeling was not measured.

7.3.2 Geometric extraction trade-offs

No extractor is best on all the test sets. The results give the following picture of each geometric extractor:

- **Strip-IRLS** has few failures on the annotated real frames. On the synthetic masks it fails when a hole wider than the merging gap and taller than the seed window splits the bottom of the corridor, and when the merging gap joins a narrow row between adjacent corridors.

- **RANSAC edges** has few failures on the annotated real frames. It fails with adjacent corridors, because the row extremes of the selected component can belong to the adjacent corridor.
- **Center points** follows one interval per strip and is accurate with adjacent corridors, but the midpoints formed before the fit carry the disturbances on one side into the centerline, and it fails more often on the annotated frames.
- **Column histogram** is the fastest extractor on the Jetson, but it returns a vertical line without heading, its error grows with the offset and the rotation of the robot, and it has the most failures on the annotated frames.

The variants of Strip-IRLS tuned on the synthetic development masks had fewer synthetic errors, but they were not better on the development part of the annotated real frames. Synthetic test sets written by the same author as the extractors find failure cases, but they can favor changes that do not carry over to real masks, so the choice between versions was made on real annotated frames.

For Strip-IRLS, RANSAC edges and EdgeNet, most gross errors, synthetic and real, are classified as `ok`. A calibrated confidence or a temporal consistency check between frames would give the controller a further protection besides the `lost` status.

7.3.3 EdgeNet: first results

EdgeNet is a first experiment, and the results above are those of the second trained model (Section 5.3). It has no geometric parameters in the extraction. On the development part of the annotated frames it has 4 failures, against 7 for Strip-IRLS and 6 for RANSAC edges, and on the test part 10, against 4 and 3 (Table 7.5). On the synthetic masks with adjacent corridors and the curve it has no gross errors, where Strip-IRLS has 2.3% and RANSAC edges 18.5%. Its accuracy depends on how well its generator covers the masks met in the field (Section 7.1.2).

Adapting the network to a new crop or camera mounting means changing the generator and retraining it, not tuning thresholds in pixels. The design choices are not removed but moved into the generator, which has 16 disturbance probabilities and many hand-chosen ranges, and the status still depends on the thresholds 0.5, 8 px, 0.38 and 0.60. The loss gives double weight to row 156, the row of the evaluation. The geometric extractors also use the same parameters for all three crops, so one model for all crops is not an advantage of the network alone. The network was not tested with another camera or mounting. The second generator was written after the errors of the first model had been inspected on masks predicted on the real sequences, which include the sequences of the annotated test frames, although not their annotations.

At this early stage, with one model for the three crops, the network is close to the geometric extractors on the annotated real frames but not better. These first results make learning the extraction from the mask a promising direction. The failures above indicate what further work has to address: the extension of the lines when the apex lies below the control row, a confidence calibrated on real frames instead of the thresholds of Strip-IRLS, and a test on frames recorded after the generator was written.

7.3.4 Field tests

In the field tests the extractor changes between runs, while the segmentation network, the controller and the row are the same. Strip-IRLS, EdgeNet, RANSAC edges and Center points differ from the mean of the other extractors by 1.1 to 2.1 cm, while the lateral distance they have in common varies by about 24 cm along the full row. This common part repeats at the same positions of the row in all runs of these four extractors, so it comes from something fixed along the row: the masks predicted in this field, with sparse plants, a predicted path that widened into the gaps between them and poor light, or the reference, which is placed by eye and does not follow the row between its five points. These measurements cannot separate the two causes. If the common part comes mainly from the masks, as the conditions of this field suggest, a better segmentation of such fields would change the trajectory more than a change among these four extractors. A reference measured at more points along the row would reduce the part of the measured distance that does not come from the robot. Column histogram is the only extractor whose runs differ from the others and from each other.

Chapter 8

Conclusions and Future Work

This chapter summarizes the main findings, discusses the contributions and limitations of the work, and identifies directions for future research.

8.1 Main contributions

This thesis investigated vision-based local guidance for agricultural row navigation through segmentation model selection, dataset preparation with automatic SAM 3 annotation, geometric reference extraction, and the integration and test of these stages on a robot. The main contributions are:

- **Architecture comparison for robotic segmentation:** FCN, DeepLabV3, and LR-ASPP were compared on manually annotated lettuce images. LR-ASPP was selected for its accuracy–speed compromise: on the test images it reached a path IoU of 0.9532 with a recorded inference time of 8.21 ms, against 59.10 ms and 86.90 ms for the other architectures, at the cost of an mIoU about 1.1 percentage points lower.
- **Automatic dataset annotation with SAM 3:** Training labels were generated with SAM 3 text prompts and used without any manual correction. Networks trained on these labels for lettuce were compared with a network trained on manual masks, and the binary labeling was then extended to the complete lettuce, tomato, and cucumber collection to train the network deployed on the robot.
- **Evaluation of output label configurations:** Networks with four-class, three-class, and binary path outputs, trained on SAM 3 lettuce labels, were

compared against the manual ground truth. Their path IoU values differ by less than 0.5 percentage points in the realistic scenario. The binary configuration was adopted for the robot because the path is the only class used by the geometric extraction.

- **Edge-based geometric extractor and its evaluation:** An extractor that converts corridor masks into image-space navigation references was implemented (Strip-IRLS). It uses horizontal strip extraction, bottom-up interval association with selection of the chain that gives the most edge samples, exclusion of the edge samples clipped by the image border, and separate fits of the left and right edges with iteratively reweighted least squares. It was compared with three baseline extractors (Center points, Column histogram, RANSAC edges) on synthetic masks with exact ground truth and on real frames of three crops with manually annotated corridor edges.
- **Edge regression network, a first experiment:** A small network that takes the binary mask and regresses the two corridor edges was designed and trained on synthetic masks. It has the same interface as the geometric extractors and was compared with them in the same tests. It is at an early stage.
- **Integration on the robot and field tests:** The segmentation network, the five extractors and a controller with an exponential moving average filter, a rejection of isolated jumps, a deadband and a proportional steering law run in one ROS 2 node on the Jetson AGX Orin of the robot (Chapter 6). Without the debug video, the callback takes 11.3 to 26.2 ms depending on the extractor, less than the 33.3 ms frame period of the camera. All five extractors were tested in closed loop in a tomato row, with the trajectory of the robot measured by a total station (Section 7.2).

8.2 Conclusions on segmentation and annotation

The architecture comparison supports LR-ASPP as a compromise between segmentation accuracy and inference time, measured on the training GPU. Whether its segmentation is sufficient for navigation is assessed through the geometric extraction and the robot tests, not from IoU alone.

On lettuce, the manual model remains the best: path IoU 0.9532 on the complete test set and 0.9587 in the realistic scenario. Networks trained only on uncorrected SAM 3 labels reach a path IoU of about 0.90 in the realistic scenario (0.9013 for the binary network), but only 0.39–0.41 on the complete test set, and the 4-class network reaches an mIoU of 0.5925. Fully automatic labels therefore provide path supervision of about 0.90 IoU in the operational conditions, but they are not a replacement for manual annotation when all classes are needed or when the scene

departs from those conditions. The multi-crop binary configuration used on the robot, trained on the complete SAM 3 collection without any manual annotation, reaches 0.9071 in the realistic scenario and 0.6369 on the complete test set when the lettuce test images are excluded from its training data. The deployed weights were assessed visually, in simulation, and on the robot.

The choice of output label configuration affects both annotation effort and downstream geometric extraction. Binary path segmentation focuses the model directly on the navigation target, and it makes both the labeling task and the geometric stage simpler. This comes at the cost of discarding plant and covering-material information that might be useful for other tasks.

8.3 Conclusions on path extraction

The geometric extraction stage converts binary corridor segmentation masks into navigation setpoints. The offline comparison of Chapter 7 gives the following results:

- On real frames with manually annotated edges, Strip-IRLS and RANSAC edges have the fewest failures, with 2.3% and 2.3% gross errors on the development frames and 2.5% and 0.8% on the test frames. The difference is a few frames and the order changes between the two parts. The network has 3.0% and 6.6%, Center points and Column histogram 5.5 to 18.8%.

- The synthetic masks show a different failure case for each extractor: for Strip-IRLS a hole wider than its merging gap and taller than its seed window, and a narrow row between adjacent corridors, for RANSAC edges adjacent corridors, for Center points disturbances on one side, for Column histogram an offset or rotated robot, and for EdgeNet a bulge on one side of the corridor, where it has its only two gross errors in the base scenarios.

- Variants of Strip-IRLS tuned on the synthetic development masks were not better on the real annotated frames. Synthetic benchmarks written by the same author as the extractors are useful to find failure cases, but the choice between versions needs real annotated data.

- The parameters of Strip-IRLS were set by hand for the camera, its mounting and the fields of this work, and in a new scenario they may need to be adjusted to get the best results. EdgeNet needs no geometric parameters, so adapting it means changing its generator instead of tuning thresholds. With one model for three crops it is close to the geometric extractors but not better, and its accuracy depends on how well the generator covers the field masks.

- For Strip-IRLS, RANSAC edges and EdgeNet the status is a weak indicator of a correct line, because most of their gross errors are classified as `ok`.

No extractor is best on every test set.

8.4 Conclusions on the field tests

The five extractors were tested in closed loop in a row of tomato plants. The plants were sparse, so the predicted path often widened into the gaps between them, and the light was poor. No image of this field is in the training data of the segmentation network (Section 7.2.1).

Strip-IRLS, EdgeNet, RANSAC edges and Center points give almost the same trajectory. On the short runs their mean lateral distance from the row centerline is between +2.9 and +4.2 cm, and the largest absolute value between 10.1 and 11.0 cm. On the full row the mean is between −6.2 and −7.2 cm and the largest absolute value between 16.3 and 20.4 cm. The root mean square difference of each of them from the mean of the other extractors is 1.1 to 2.1 cm, while the lateral distance they have in common varies by about 24 cm along the full row. These runs therefore do not rank the four extractors. The common part repeats at the same positions of the row, so it comes from the masks predicted in this field or from the reference, which was placed by eye at five points. The measurements cannot separate the two causes.

Column histogram is the only extractor whose runs differ from the others and from each other, with a root mean square difference of 6.8 cm on the short runs and 4.1 cm on the full row. It classified as `ok` also the frames in which its centerline was at the image border.

8.5 Limitations

The main limitations of the work are:

- **Structural limits of Strip-IRLS:** The dependence on the seed in the bottom strips, the number of parameters and the effect of missing plants on one side are described in Section 5.5. On the synthetic masks the seed and the merging gap cause the gross errors in `hole_bottom_tall` and `adjacent+offset` (Section 7.1.2).

- **Geometric model constraints:** Strip-IRLS, RANSAC edges, Center points and the edge network model the corridor with straight lines. They cannot follow curved crop rows or navigate headland turnaround maneuvers where crop boundaries terminate or diverge.

- **Dataset annotation coverage:** Segmentation accuracy was measured against manual ground truth only on lettuce. The deployed weights have no independent IoU, because their training data include the lettuce test images. The multi-crop configuration was evaluated through a separate network trained without them. Tomato and cucumber have no manual ground truth. The random image-level split also takes training and test images from the same lettuce folders, each recorded along one path. No test image is used for training, but the reported IoU values may be optimistic for paths not seen during training.
- **Label definition mismatch:** The manual masks mark only the central traversable corridor as path, whereas networks trained on SAM 3 labels can also segment adjacent or non-traversable corridors. This lowers their IoU against the manual masks, but the size of the effect was not measured.
- **Quality of the SAM 3 labels:** Visual inspection showed that SAM 3 segmented the plastic mulch film poorly and also assigned the greenhouse plastic cover to the covering class, and that its path masks were less accurate in images taken from unusual viewpoints or where the path between the rows is not clearly visible. Because the labels were used without correction, these errors enter the training data, and their extent was not measured.
- **Annotation effort measurement:** The time saved by automatic labeling with respect to manual annotation was not measured.
- **Offline test data:** The synthetic masks are straight corridors whose edges have irregularities of a few pixels, while the real predictions often have rounded boundaries. Only 255 real frames have an annotated ground truth, marked once by one person, and the frames skipped or with a nearly horizontal edge, which are among the most difficult, are excluded. The annotated frames of the SAM 3 dataset come from images used to train the network, so their masks are probably cleaner than predictions on new images.
- **Parameters of the geometric extractors:** The parameters of Strip-IRLS and of the baselines were chosen for the camera, its mounting and the fields of this work (Section 5.2.6). With another camera, mounting or crop, they may need to be adjusted to get the best results.
- **Confidence of the extractors:** The confidence scores are heuristic and not calibrated, and most gross errors of Strip-IRLS, RANSAC edges and the edge network are classified as `ok`.
- **Field tests:** The tests were made in one row of one crop on one day, with three short runs of 8 m and one run along the full row per extractor. The

reference of the row center was placed by eye at five points, so the measured lateral distance also contains its error. Attempts that did not go well were repeated and only the repeated runs are reported, so the tests do not give a failure rate.

8.6 Future work

Several directions for future research emerge from this work:

8.6.1 Dataset and annotation

- Measure the annotation time of manual and automatic workflows, and test whether a light correction of the SAM 3 labels improves the trained networks.
- Split the data by folder rather than by image, so that the test paths are not represented in training, and repeat the evaluation of all networks.
- Prepare manual ground-truth masks for tomato and cucumber to evaluate the deployed network on those crops directly.
- Add images of fields like the one of the field tests, with sparse plants and poor light, to the training data.
- Investigate active learning strategies to select images for manual annotation that improve model generalization most effectively.
- Explore other foundation models or fine-tuned SAM variants specialized for agricultural segmentation.

8.6.2 Segmentation and model deployment

- Evaluate additional lightweight architectures (e.g., BiSeNet, FastSCNN) for further latency reduction.
- Investigate quantization and model compression to reduce the inference time on the Jetson AGX Orin further.
- Test temporal consistency methods that use previous frames to improve segmentation and reduce flickering.
- Explore uncertainty estimation to detect when segmentation confidence is low and fallback strategies are needed.

8.6.3 Geometric extraction

- Extend the geometric representation from linear models to piecewise linear or polynomial formulations to support curved rows and headland turns.
- Implement top-down recovery logic to maintain guidance when bottom-up seeding fails, and handle a corridor that rises from a full-width band of path.
- Annotate more real frames, from new recordings and by more than one annotator, so that the differences of a few frames between the best extractors can be resolved and the annotation error can be measured.
- Calibrate the confidence of the extractors, or add a temporal consistency check, so that wrong lines can be rejected before they reach the controller.
- Extend the generator of the edge network to other cameras, mountings and crops, so that a new scenario needs a new training set and not a new tuning of geometric thresholds.
- Evaluate the edge network on annotated frames of new recordings, and calibrate its status thresholds for its own confidence.
- Adapt spatial gate thresholds dynamically using vehicle attitude telemetry from an onboard IMU to prevent false tracking loss during pitch and roll variations.
- Incorporate temporal Bayesian tracking (such as an Extended Kalman Filter) to smooth reference parameters across frame dropouts.

8.6.4 System integration and field validation

- Repeat the field tests in more rows, crops, growth stages and lighting conditions, with longer runs and a reference measured at more points along the row, so that the error of the reference can be separated from that of the robot and a failure rate can be measured.
- Predict the reference from the motion of the robot, so that guidance continues during reference losses longer than the 0.5 s hold of the controller.
- Investigate fusion with other sensors (e.g., depth, IMU) to keep a valid reference when the camera view is degraded.
- Develop a complete autonomy system including global path planning, obstacle avoidance, and task-specific behaviors.

8.7 Closing remarks

This thesis addressed the segmentation, dataset preparation, geometric extraction and on-robot integration stages of vision-based agricultural navigation. The selection of LR-ASPP balances accuracy and speed for robotic deployment. Labels generated by SAM 3 without manual correction were used to train the network deployed on the robot, and the geometric extractor converts masks into navigation references using horizontal strip selection and independent edge fitting. In the field tests, the trajectories of four of the five extractors differ by 1.1 to 2.1 cm (root mean square) from the mean of the others.

Future research should extend the geometric model to handle curves, evaluate the deployed network on independent, sequence-level test data, and repeat the field tests in more fields with a more accurate reference.

Bibliography

[1] Yuhao Bai, Baohua Zhang, Naimin Xu, Jun Zhou, Jiayou Shi, and Zhihua Diao. «Vision-based navigation and guidance for agricultural autonomous vehicles and robots: A review». In: *Computers and Electronics in Agriculture* 205, 107584 (Feb. 2023). DOI: 10.1016/j.compag.2022.107584 (cit. on pp. 1, 2, 5, 7).

[2] Rajitha de Silva, Grzegorz Cielniak, Gang Wang, and Junfeng Gao. «Deep Learning-Based Crop Row Detection for Infield Navigation of Agri-Robots». In: *Journal of Field Robotics* 41.7 (2024), pp. 2299–2321. DOI: 10.1002/rob.22238 (cit. on pp. 1, 2, 5).

[3] Alessandro Navone, Mauro Martini, Marco Ambrosio, Andrea Ostuni, Simone Angarano, and Marcello Chiaberge. «GPS-Free Autonomous Navigation in Cluttered Tree Rows with Deep Semantic Segmentation». In: *Robotics and Autonomous Systems* 183, 104854 (Jan. 2025). DOI: 10.1016/j.robot.2024.104854 (cit. on pp. 1, 2, 5–8, 47).

[4] Enrico Mendez, Javier Piña Camacho, Jesús Arturo Escobedo Cabello, and Alfonso Gómez-Espinosa. «Autonomous Navigation and Crop Row Detection in Vineyards Using Machine Vision with 2D Camera». In: *Automation* 4.4 (Sept. 2023), pp. 309–326. DOI: 10.3390/automation4040018 (cit. on pp. 2, 5, 7, 8).

[5] Alessandro Navone, Mauro Martini, Andrea Ostuni, Simone Angarano, and Marcello Chiaberge. «Autonomous Navigation in Rows of Trees and High Crops with Deep Semantic Segmentation». In: *2023 European Conference on Mobile Robots (ECMR)*. IEEE, 2023, pp. 1–6. DOI: 10.1109/ECMR59166.2023.10256334 (cit. on p. 5).

[6] Xia Li, Junhao Su, Zhenchao Yue, and Fangtao Duan. «Adaptive Multi-ROI Agricultural Robot Navigation Line Extraction Based on Image Semantic Segmentation». In: *Sensors* 22.20, 7707 (Oct. 2022). DOI: 10.3390/s22207707 (cit. on pp. 5–7).

[7] Jonathan Long, Evan Shelhamer, and Trevor Darrell. «Fully Convolutional Networks for Semantic Segmentation». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 3431–3440. DOI: 10.1109/CVPR.2015.7298965 (cit. on pp. 6, 9–11).

[8] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. «Rethinking Atrous Convolution for Semantic Image Segmentation». In: *arXiv preprint arXiv:1706.05587* (2017) (cit. on pp. 6, 9–12).

[9] Andrew Howard et al. «Searching for MobileNetV3». In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 1314–1324 (cit. on pp. 6, 13–15).

[10] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn and PyTorch: Concepts, Tools, and Techniques to Build Intelligent Systems*. 1st ed. O'Reilly Media, 2026. ISBN: 979-8-341-60798-9 (cit. on pp. 6, 9, 10, 13, 24).

[11] Nicolas Carion et al. «SAM 3: Segment Anything with Concepts». In: *International Conference on Learning Representations (ICLR)*. 2026. arXiv: 2511.16719 [cs.CV]. URL: https://proceedings.iclr.cc/paper_files/paper/2026/hash/e0982cbc81401df3430ee1ff780dc7a2-Abstract-Conference.html (cit. on pp. 6, 16).

[12] Letian Wu, Yongzhi Cui, Huifeng Shi, Xiaoli Sun, Jiayan Yang, Xinwei Cao, Ping Zou, and Ya Liu. «An Improved DeepLabV3+-Based Method for Crop Row Segmentation and Navigation Line Extraction in Agricultural Fields». In: *Sensors* 26.10, 3142 (May 2026). DOI: 10.3390/s26103142 (cit. on p. 7).

[13] Johannes Kneip, Patrick Fleischmann, and Karsten Berns. «Crop Edge Detection Based on Stereo Vision». In: *Robotics and Autonomous Systems* 123, 103323 (Jan. 2020). DOI: 10.1016/j.robot.2019.103323 (cit. on p. 7).

[14] TorchVision Maintainers and Contributors. *TorchVision v0.20.1: FCN segmentation models, source code* ***torchvision/models/segmentation/fcn.py***. URL: https://github.com/pytorch/vision/blob/v0.20.1/torchvision/models/segmentation/fcn.py (visited on 11/14/2026) (cit. on pp. 10, 11, 23).

[15] TorchVision Maintainers and Contributors. *TorchVision v0.20.1: DeepLabV3 segmentation models, source code* ***torchvision/models/segmentation/deeplabv3.py***. URL: https://github.com/pytorch/vision/blob/v0.20.1/torchvision/models/segmentation/deeplabv3.py (visited on 11/14/2026) (cit. on pp. 10, 12, 23).

[16] TorchVision Maintainers and Contributors. *TorchVision v0.20.1: LR-ASPP segmentation model, source code* `torchvision/models/segmentation/lraspp.py`. URL: `https://github.com/pytorch/vision/blob/v0.20.1/torchvision/models/segmentation/lraspp.py` (visited on 11/14/2026) (cit. on pp. 10, 15, 23).

[17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. «Deep Residual Learning for Image Recognition». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. DOI: `10.1109/CVPR.2016.90` (cit. on p. 11).

[18] TorchVision Maintainers and Contributors. *TorchVision: MobileNetV3 models, source code* `torchvision/models/mobilenetv3.py`. URL: `https://github.com/pytorch/vision/blob/v0.20.1/torchvision/models/mobilenetv3.py` (visited on 11/13/2026) (cit. on p. 15).

[19] Devansh Arpit et al. «A Closer Look at Memorization in Deep Networks». In: *Proceedings of the 34th International Conference on Machine Learning*. Ed. by Doina Precup and Yee Whye Teh. Vol. 70. Proceedings of Machine Learning Research. 2017, pp. 233–242. URL: `https://proceedings.mlr.press/v70/arpit17a.html` (cit. on p. 27).

[20] Rosanne Liu, Joel Lehman, Piero Molino, Felipe Petroski Such, Eric Frank, Alex Sergeev, and Jason Yosinski. «An Intriguing Failing of Convolutional Neural Networks and the CoordConv Solution». In: *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*. 2018 (cit. on p. 44).

[21] Martin A. Fischler and Robert C. Bolles. «Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography». In: *Communications of the ACM* 24.6 (June 1981), pp. 381–395. DOI: `10.1145/358669.358692` (cit. on p. 47).

[22] Philip H. S. Torr and Andrew Zisserman. «MLESAC: A New Robust Estimator with Application to Estimating Image Geometry». In: *Computer Vision and Image Understanding* 78.1 (2000), pp. 138–156. DOI: `10.1006/cviu.1999.0832` (cit. on p. 73).